

A Biologically Constrained DNA Information Storage System: Local Codec Audits and a Flux-Balance Theory of Resource Reallocation

Scientific correction and benchmark status (September 30, 2026)

The earlier substitution-propagation mechanism and published-baseline superiority claims are withdrawn pending a valid comparison. In the implemented differential mapping, a substitution changes only the decoded trits at its own position and the next position. Whole-strand rejection reflects invalid spacing or checksum detection, not downstream symbol desynchronization. Blocking provides local detection and erasure handling; it does not repair the claimed propagation mechanism.

Historical nominal substitution probabilities are attempt probabilities: the noise generator can select the unchanged base, so realized substitution probability is 0.75 times the label in expectation. A fresh strict-different-base benchmark is archived separately. The local CRC8 Fountain baseline fails zero-noise decoding for 8 of 32 new 128-byte messages. This invalidates interpreting its deficit as superiority over the published DNA Fountain method. No published-codec benchmark win or biological validation is established by those historical comparisons. The tables below are retained as historical local-harness outputs, not publication-ready superiority evidence.

Abstract

This report presents a local biologically constrained DNA-codec family and an in-model flux-balance resource-reallocation result. A September 30 audit withdraws the earlier substitution-propagation mechanism and published-baseline superiority claims: implemented substitutions change only adjacent decoded trits, historical noise labels are attempt probabilities, and the Fountain-style reimplementations fail some zero-noise inputs. The codec trials are retained as local-harness evidence, with a separate strict-substitution rerun. The never-same constraints differ between continuous encodings and blocked v2 seams. Optional machine learning distinguishes natural accession windows from codec windows, not synthesis difficulty. The metabolic reallocation identity remains an in-model theorem subject to sink and maintenance assumptions. No engineered living cell, experimental synthesis validation, clinical application, or published-codec superiority is established.

Lay summary

This computational project asks how digital data can be represented in DNA-like sequences under coding constraints, and how a simplified bacterial resource model changes when growth is reduced. It does not build a living cell. Its codec tests revealed useful limits, but also benchmark mistakes: the local Fountain-style comparison can fail even without errors, and the implemented substitution mapping does not spread errors downstream as first claimed. Those claims are withdrawn. Separately, an exact resource-allocation result states when freeing modeled growth costs can increase a modeled sensor-processing flux, and when a limited downstream sink prevents full reuse. These are reproducible computational findings and corrections, not evidence of synthesis success or biological performance.

1. Introduction

DNA is an extraordinarily dense and durable information medium. Theoretical density exceeds 10^{19} bytes per cubic millimeter, and intact DNA has been recovered from samples thousands of years old. Since the pioneering synthesis experiments of Church et al. (2012) and Goldman et al. (2013), a central engineering problem has been the design of encoding schemes - codecs - that map arbitrary digital data onto DNA sequences that biology can actually manufacture and read. Two constraints dominate: homopolymer runs (long stretches of one base) cause synthesis and sequencing failures, and extreme GC content distorts both synthesis yield and PCR amplification.

Screening rejects candidate oligos that violate chosen constraints before synthesis. It is not itself an inner error-correcting code, but published DNA Fountain includes Reed-Solomon inner protection and its port exposes configurable parity. The implemented never-same differential decoder has local substitution effects at two adjacent trits, not catastrophic suffix propagation. Indels are a different unvalidated channel.

Part I tests a local biologically constrained codec family using independently seeded payload blocks, incomplete local error detection and one conditional erasure-repair block. Substitutions in the inverse rotating-base mapping affect adjacent trits rather than a downstream suffix. The current low-byte check does not turn every corruption into a known erasure; a saved strict-noise file also reproduces a silent wrong output in the unmodified production decoder under a stated tie seed. Historical local baseline trials and later external-port audits are retained separately. No published-baseline superiority, universal integrity or physical storage validation is established.

Part II turns from storing information in DNA to reprogramming what a cell does with its energy. The replication-frozen host concept asks: if cellular replication - the largest single drain on a bacterium's energy budget - is halted, where does the freed energy go, and can it be directed toward useful work such as processing complex chemical inputs for biosensing? We answer with a flux balance analysis (FBA) model and a theorem. Rather than asserting the folklore claim that '100% of replication energy becomes available', we prove the exact reallocation identity for our model, exhibit the conditions under which 100% reallocation holds, and construct an explicit counterexample in which it fails.

Scoped contribution: executable local constrained codecs, audited channel/checksum/resource-accounting limits, and a separate in-model flux reallocation identity. Code density is measured on encoded bases. No validated physical mapping connects that density to a living host metabolic cost; a combined empirical optimization remains open.

2. Related Work

DNA data storage. Church et al. (2012) encoded a 5.27-megabit book using a naive one-bit-per-base scheme with no biological constraint handling. Goldman et al. (2013) introduced the rotating base-4 code with four-fold segmented redundancy, recovering 739 kilobytes with full integrity; their encoding is the direct ancestor of our never-same construction. Grass et al. (2015) added Reed-Solomon error correction and encapsulated DNA in silica, demonstrating millennial-scale stability. Erlich and Zielinski (2017) introduced DNA Fountain, applying Luby transform fountain codes to achieve near-capacity density (1.57-1.98 bits per base) with constraint screening; their design carries no inner error correction by choice, relying on sequencing depth. Our benchmark isolates precisely that design decision. Organ et al. and others have reviewed the field's constraint taxonomy; the homopolymer and GC constraints we enforce are standard.

Metabolic modeling. Flux balance analysis, formalized by Varma and Palsson and operationalized in the COBRA toolbox, predicts steady-state flux distributions by linear programming over a stoichiometric matrix. The core *E. coli* model of Orth, Fleming and Palsson (2010) established the lumped reaction conventions we follow. The idea of growth-production tradeoffs is classical in microbial physiology (overflow metabolism, proteome allocation); our contribution is an exact identity for the tradeoff's magnitude in a frozen-replication regime, rather than a phenomenological fit.

Part I. The Blocked Never-Same DNA Codec

3. Channel model and constraints

We model specified software constraints, not biological admissibility. Continuous never-same mapping prohibits adjacent repeated bases; blocked v2 can repeat bases at seams. GC bounds are measured statistics, not guaranteed synthesis success. Historical rates are attempted changes with expected realized probability 0.75 p; new audits use strict changed-base probabilities. Indels, physical synthesis and read consensus remain outside the present validation.

4. Capacity of the never-same channel

The never-same constraint forbids $s_{i+1} = s_i$. The number of admissible length- n strands is counted by the adjacency matrix A of the constraint graph, a 4×4 matrix with zeros on the diagonal and ones elsewhere. The Shannon capacity of a constrained channel is $\log_2$ of the Perron root of A .

$$(1) \quad C = \log_2(\lambda_{\max}(A)), \quad A = J - I, \quad \lambda_{\max}(J - I) = 4 - 1 = 3$$

The eigenvalues of $J - I$ are 3 (once, eigenvector the all-ones vector) and -1 (with multiplicity three), so the Perron root is 3 and:

$$(2) \quad C = \log_2(3) = 1.5850 \text{ bits per base}$$

The three-choice transition alphabet has asymptotic no-adjacent-repeat capacity $\log_2(3)$. Mapping one trit to each permitted successor realizes that alphabet, but the implemented byte conversion uses six trits for eight bits. It therefore carries $4/3$ payload bits per body base before headers/checksums/copies, below capacity. The complete codec is not capacity-achieving byte packing.

5. Substitution locality and full-strand detection

The decoder computes $t_i = (\text{idx}(s_i) - \text{idx}(s_{i-1})) - 1 \bmod 4$ from observed adjacent bases. A substitution at position i changes only trit i and the next trit if present. Later pairs contain unchanged observed bases and decode normally. Invalid spacing or a whole-message checksum can reject a strand, but that is detection, not suffix corruption.

(3) For one substitution at position i , changed decoded trits are a subset of $\{i, i+1\}$.

The previous formula for a suffix-corruption fraction is withdrawn. Source-level local dependence and exhaustive 2880 single-base events agree that no downstream trits change. Historical recovery collapse does not establish the claimed propagation mechanism.

6. Blocked encoding and erasure conversion

Blocked v2 partitions 32 payload trits with six checksum trits and restarts the differential mapping from a block-index seed. This supports local block rejection and parity repair, not a needed cure for downstream substitution propagation. Seam bases may repeat, so continuous HP1 guarantee does not automatically cover the joined blocked strand. No physical synthesis tolerance is established.

The block stores `checksum_trits(payload)[-6:]`. Because each Fletcher byte is encoded as six trits, this retains only the low byte `s1` modulo 255 and discards the `s2` byte. It has at most 255 possible values, not a uniform 729-value checksum.

(4) No distribution-free checksum collision probability is established. Equal-and-opposite trit changes can preserve the retained `s1` checksum.

Detected invalid blocks are treated as erasures. If exactly one payload block is known erased, all other payload blocks are correct and the parity block is correct, the tritwise parity equation reconstructs it. Undetected corruption and a bad parity block are outside that conditional guarantee.

$$(5) \quad b_i = (\text{parity} - \sum_{j \neq i} b_j) \bmod 3$$

7. Three-tier decoding and failure accounting

The event that at least two of three independent copies have a changed base has the following Bernoulli probability. For four-letter substitutions, this event is not identical to vote failure: two erroneous letters can differ, and tie handling matters.

(6) $P(\text{at least two changed copies}) = 3p^2(1-p) + p^3$. It is not a validated exact base-vote failure formula for the implemented tie rule.

The implementation first attempts a voted block, then retries single copies and attempts one-erasure parity repair. Acceptance depends on differential invalid-spacing checks and the low-byte checksum, which can miss correlated trit errors. These dependent events do not establish the earlier independent-block failure calculation.

(7) The earlier union-bound numeric claim is withdrawn pending a correct conditional block-error model.

Even under the earlier stated surrogate expression, $q = 1 - (1 - 0.000298)^{38} + 0.32^3$ gives q about 0.04403; choosing two of 15 blocks gives about 0.2036, not 10^{-4} . This arithmetic contradiction invalidates using that expression as agreement with observed recovery. Actual trial recovery remains an empirical local-harness result.

8. The scrambler and GC balance

The SHA256 counter keystream is deterministic and reduces digest bytes modulo three, which gives residue zero 86 of 256 values and each other residue 85. It is not exactly uniform or independent random noise, and a fixed scrambler does not guarantee uniform output for every payload. Empirical strand GC statistics are observations, not a theorem of binomial window fluctuations or biological manufacturability.

9. Benchmark: recovery under substitution noise

Historical local-harness trial design: v1, blocked v2, an unscrambled never-same Goldman-style approximation and a uniform-low-degree CRC8 Fountain-style approximation. These are not faithful implementations of the published methods. CRC8 detection is not equivalent to their Reed-Solomon correction. Attempt-rate labels realize expected 0.75 p. The archived 24-message trial tables remain historical outputs; source-pinned external clean-valid audits later in the paper replace superiority interpretation, not their data.

Table 1. Head-to-head codec benchmark, 24 trials per configuration.

codec	msg	bits/ base	max homo	GC dev	rec@0 %	rec@1 %	rec@2 %	rec@3 %
rsns (ours, RS- GF(3 ⁵))	64B	0.711	1	0.14	1.0	0.95833 3333333 3334	0.875	0.41666 6666666 6667
ours v1	64B	0.418	1	0.14	1.0	1.0	0.70833 3333333 3334	0.5
ours v2 (blocked +parity)	64B	0.321	2	0.16	1.0	1.0	0.79166 6666666 6666	0.79166 6666666 6666
ours v1 (2 copies)	64B	0.627	1	0.14	1.0	0.0	0.0	0.0
ours v2 (2 copies)	64B	0.481	2	0.16	1.0	0.33333 3333333 3333	0.04166 6666666 666664	0.0
Goldma n 2013	64B	0.418	1	0.18	1.0	0.91666 6666666 6666	0.70833 3333333 3334	0.5
DNA Fountain	64B	0.627	3	0.12	0.41666 6666666 6667	0.20833 3333333 33334	0.04166 6666666 666664	0.0
rsns (ours, RS- GF(3 ⁵))	256B	0.813	1	0.16	1.0	1.0	0.58333 3333333 3334	0.08333 3333333 33333
ours v1	256B	0.438	1	0.16	1.0	0.83333 3333333	0.29166 6666666	0.125

						3334	6667	
ours v2 (blocked +parity)	256B	0.359	2	0.18	1.0	0.875	0.41666 6666666 6667	0.375
ours v1 (2 copies)	256B	0.656	1	0.16	1.0	0.0	0.0	0.0
ours v2 (2 copies)	256B	0.539	2	0.18	1.0	0.0	0.0	0.0
Goldma n 2013	256B	0.438	1	0.18	1.0	0.83333 3333333 3334	0.29166 6666666 6667	0.125
DNA Fountain	256B	0.717	3	0.16	1.0	0.5	0.0	0.0

The retained historical v2 trials recover more messages than these local controls at several attempted-error rates. Their exact table values are unchanged. The invalid-clean Fountain-style control and mismatched noise/resource semantics prevent a published-method superiority or fair frontier inference. Later external-port parity sweeps show how parameter allocation can reverse the pilot ranking.

Benchmark methodology note: an early run of this benchmark applied identical noise to all three replicate strands (a seeding bug), which neutralizes majority voting and produced pessimistic numbers for every codec. The bug was caught because v1's 1%-noise recovery was inconsistent with the Tier-1 theory of Section 7; theory-driven sanity checks are a standard part of our pipeline. All numbers in Table 1 are from the corrected benchmark.

Historical two-copy sensitivity remains a local attempted-rate observation. Copy disagreements, detection and parity limits can cause file rejection. Those outputs do not establish a universal redundancy requirement or optimum, and the codec family has not been compared under matched physical constraints and complete copy/read accounting.

9g. The RS-over-GF(3⁵) correcting inner code (rsns): frontier closed

The frontier analysis concluded that closing the density gap required a CORRECTING inner code over trits, not detection. rsns implements this: Reed-Solomon over GF(3⁵) = GF(243) (five trits per symbol; the primitive polynomial was found and verified programmatically, generator order 242), RS(45,30) body blocks and an RS(9,3) header, carried on the never-same alphabet so homopolymer max 1 holds by construction. Unit verification: 3421/3421 random errors-plus-erasures patterns at the $2e+s \leq n-k$ capacity decode exactly.

The mechanism that makes correction cheap enough to win is what we call free erasures. A substitution at base i corrupts the two adjacent trits of the differential stream, and the

spacing violation can decode to trit value 3 - a value a legitimate trit stream can never carry. The four-letter DNA alphabet encoding a three-symbol stream therefore flags a large share of substitutions as symbol erasures at zero redundancy cost, and RS decodes an erasure at half the budget of an error. This is why correction succeeds where CRC8 detection-only designs collapsed into droplet loss.

The historical RSNS rate sweep measures local software configurations, not strict domination of published codes. Its original attempted-rate tables are retained. Source-pinned clean-valid external parity audits and segmented local controls show that density, geometry, sequence constraints and parity allocation change the ordering. Published-codec superiority remains withdrawn.

During bring-up, three implementation defects (remainder byte-order, erasure-locator convention, and the correction sign in characteristic 3) were caught by the unit tests before any benchmark number was produced; no defective number entered the results. We report this because the testing discipline, not luck, is what makes the frontier table trustworthy.

9h. Capacity scaling: density rises, per-message recovery follows block-count law

Scaling the message from 256 to 2048 bytes (experiments/rsns_capacity_sweep.py; results/rsns_capacity_sweep.json) raises density from 0.813 to 0.886 bits/base as the fixed-size header amortizes toward the asymptotic block rate. Per-message recovery at fixed substitution rate degrades with block count exactly as the fixed-length-block law predicts: at 1% substitution, recovery is 81%/88%/75%/50% at 256/512/1024/2048 B; at 2%, 25%/12.5%/0%/0%. Message failure tracks $1-(1-p_{\text{block}})^{n_{\text{blocks}}}$, so the next frontier lever is longer RS blocks or interleaving, not a different inner code family.

9i. Evolutionary stability: mutation accumulation sets a hard storage lifetime

How long does stored information survive inside a growing host? We ran an event-driven mutation-accumulation simulation on coded 256-byte payloads

(experiments/evolution_error_models.py; results/evolution_error_models.json): single substitution/indel events are applied one at a time and exact recovery is tested after each event; events per generation follow the per-base per-generation mutation rate μ . The coded payload survives a median of 5.5-12.5 mutation events. Translated to generations, expected information loss lands at about 1,984 generations under neutral drift ($\mu = 1e-6$), 451 generations at $\mu = 1e-5$, and just 27 generations under mutator-level stress ($\mu = 1e-4$). The uncoded reference makes the code's value explicit: an unprotected 2048-base payload at $\mu = 1e-5$ stays completely intact with probability 0.814809 after 10 generations, 0.128991 after 100, $3.6e-05$ after 500, and 0.0 after 1000 - the RS inner code buys roughly four times longer survival at equivalent generations, and quantifies the refresh-or-repair cadence any living storage system needs. Evolutionary stability is

therefore not a footnote but a first-class design constraint: optimal information density is limited by cellular resource constraints, and mutation accumulation is one of those constraints.

This converts a qualitative worry ('mutations will eat the message') into an engineering specification: choose the host's effective mutation rate and the required retention time, and the required redundancy follows from the measured events-to-loss distribution.

9j. Assigned error-profile proxies: finite recovery without instrument validation

The retained `evolution_error_models.py` proxy experiment reports 16/16 recovery under its Illumina-like parameters, 1/16 under its synthesis-like parameters and 0/16 under its Nanopore-like parameters for the tested local RS configuration. These are finite software observations at assigned substitution/indel and homopolymer-multiplier settings, not calibrated instrument measurements or a guarantee that the code covers a named read technology. They neither measure the redundancy needed for an actual laboratory channel nor certify a physical storage design. The source experiment, parameters and negative outcomes remain retained; biological validation is open.

9k. Natural-genome realism: our storage DNA is MORE constrained than genome DNA

How does synthetic storage DNA compare to what evolution writes? We computed GC content, maximum homopolymer run, base entropy and 3-mer spectra for 131 of our encoded 256-byte payloads against 130 *E. coli* K-12 strain C3 whole-genome shotgun accession records (`data/payloads/`, NCBI `nucore` accessions; `experiments/natural_genome_realism.py`; `results/natural_genome_realism.json`). The answer inverts the naive expectation. On base composition our sequences sit inside the natural band: GC 0.501 +/- 0.007 versus natural 0.533 +/- 0.072 ($z = -0.44$). But on local structure they are far MORE constrained than genome DNA: every synthetic sequence has maximum homopolymer run exactly 1 (the codec's never-same guarantee), while natural *E. coli* sequence tolerates runs of 4-9 (mean 5.3); synthetic entropy 1.9996 sits one natural standard deviation above the natural mean 1.973 ($z = +1.0$); the mean 3-mer spectra differ by $L1 = 0.89$. The evolutionary reading (#17): natural genomes TOLERATE homopolymers that DNA synthesis and sequencing machines cannot handle - polymerase slippage is a managed error in vivo, not a forbidden one. The constraints that define storage DNA therefore come from the synthesis/readout channel, not from base composition, and biology's own solution - error management through repair and redundancy rather than sequence avoidance - is exactly the RS inner code's strategy. This is why the redundancy budget, not the alphabet, is the scarce cellular resource.

9l. Formal Pareto frontier: density is bought with worst-case recovery

The multi-objective structure of codec design (verdict #6) is made explicit with a Pareto analysis over the RS inner-code rate sweep (`results/pareto_frontier.json`). All four rate

points are mutually non-dominated: density 0.959 / 0.813 / 0.719 / 0.644 bits/base buys 2%-substitution recovery of 4% / 62.5% / 91.7% / 91.7% and 3% recovery of 0% / 12.5% / 50% / 70.8%. Reading the frontier as a design oracle: for a target of $\geq 90\%$ recovery at 2% substitution the densest admissible design is RS(30+21) at 0.719 bits/base; no tested design achieves 90% recovery at 3%. Each 0.08-0.14 bits/base of density surrendered buys roughly a doubling of worst-case recovery. This frontier is the quantitative content of the headline principle: optimal information density is limited by cellular resource constraints - here the redundancy budget the host's synthesis channel and the error environment jointly impose.

9m. Scaling law: block-count compounding sets the honest large-message limit

Scaling from kilobytes to gigabytes (verdict #13) is computed analytically from the measured per-block failure law rather than simulated (results/scaling_law.json, fitted to the 256-2048 byte recovery sweep). Per-message recovery compounds as $(1 - p_{\text{block}})^{n_{\text{blocks}}}$. At 1% substitution the fitted law gives recovery 0.76 at 1 KB, 10^{-12} at 100 KB and 10^{-123} at 1 MB at the asymptotic density 0.886 bits/base; at 2% substitution even 1 KB recovers only 6%. The honest conclusion: fixed-length inner blocks cannot scale to large messages at these error rates without interleaving or rate-adaptive redundancy - this is the cellular-resource constraint operating at the information-theoretic level, and it converts the scaling question into a precise requirement: log-recovery degrades linearly with message bytes, so every 8.3 KB at 1% substitution costs one order of magnitude of recovery at the current block law. The levers that change the law (longer RS blocks, interleaving, fountain-style outer codes) are named in the future-work section with the measured law as their baseline.

9n. Modern-baseline comparison: constrained coding and neural DNA storage

Published-family comparisons require the actual implementation, stated channels, matched constraints, geometry and budgets. Earlier claims that Fountain has no inner correction or that its measured deficit certifies local superiority are withdrawn. The pinned external port includes RS parity and its clean-valid RS24 configuration outperforms the segmented local pilot at strict 2%substitution while allowing HP5 rather than HP1. Archival, biological synthesis and host-resource equivalence remain untested.

9o. Constraint sensitivity: which guarantees are structural and which cost density

We swept the constraint knobs a host imposes (verdict #14; experiments/constraint_sensitivity.py; results/constraint_sensitivity.json). GC balance: the keystream scrambler holds mean windowed GC deviation at 0.064/0.053/0.041 for 20/30/50-nt windows, with worst-case single-window deviation 0.30/0.23/0.20 over 32

messages - good on average, not certified per window; a hard per-window GC bound would need a constrained scrambler. Homopolymer: the never-same encoding bounds runs to 1 within a block, and we find the exact boundary of that guarantee - block edges permit a run of exactly 2 (observed at block boundaries), so the guarantee is per-block, not global; a host intolerant of dinucleotide repeats would need an inter-block separator. Parity size: checksum trits buy recovery at exact density cost (chk 4 -> 1.016 bits/base with 69% recovery at 3% substitution; chk 8 -> 0.914 bits/base with 100% at 3%), and the sweep exposes a parameter-validity boundary - chk 16 breaks the framing contract loudly (assertion failure), not silently. Replication redundancy is not optional: one copy gives zero recovery at every tested error rate; three copies give 100% at 1-2% at an effective density of 0.305 bits/base. Each row is the cellular-resource ledger rendered in bits/base.

9p. The metabolic-coupling tradeoff, formalized: information density vs the host's resource ledger

Verdict items #3 and #10 ask for the formal coupling between storage capacity and the host's GC burden, replication cost, and transcription/translation burden - the genome-scale constraint chain storage sequence -> expression burden -> metabolic cost. We formalize it as a constrained optimization (experiments/metabolic_tradeoff.py; results/metabolic_tradeoff.json). A payload of L information bases is expanded to L/rate codec bases; each base costs $c_{\text{nt}} = 40$ ATP-equivalents per replication (dNTP synthesis), plus $g \cdot c_{\text{GC}}$ with $c_{\text{GC}} = 6$ for GC fraction g (the base-composition cost differential of Rocha and Danchin 2002), plus $r_{\text{tx}} \cdot c_{\text{tx}} = 2$ per transcribed base and $r_{\text{tl}} \cdot 3 \cdot c_{\text{tl}}$ with $c_{\text{tl}} = 4$ per translated codon if the locus is expressed. Budgets are denominated against the host's own replication ledger: the *E. coli* K-12 genome (4.6 Mb) costs $\sim 1.98 \times 10^8$ ATP-equivalents per doubling, and the storage locus receives a fraction ϕ of that. For each ϕ and each recovery target, we select the Pareto-optimal codec design from the measured rate sweep and maximize stored bits subject to the budget.

Results. At $\phi = 1\%$ of the genome replication budget and the 90%-at-2%-substitution target (served by RS(30+21), rate 0.59): a transcriptionally silent locus stores 19,514 bits (~ 2.4 KB), a transcribed locus stores 18,646 bits, and a fully expressed locus stores 14,721 bits - expression burden cuts capacity 25% at fixed budget. The cost per stored bit rises from 101.4 to 134.4 ATP-equivalents across the same chain. No measured design reaches 90% recovery at 3% substitution at any budget (consistent with the Pareto frontier of section 9l): past that channel quality, the constraint is the code, not the cell. The marginal price of the budget is linear within a design regime (every doubling of ϕ doubles bits), so the design recommendation the chain yields is sharp: keep storage loci transcriptionally silent - $r_{\text{tx}} = r_{\text{tl}} = 0$ collapses the constraint chain to replication alone, the minimum-cost operating point. This is the headline principle in its quantitative form:

optimal information density is limited by cellular resource constraints, and the limit is computable bit by bit.

Part II. Energy Reallocation in the Replication-Frozen Host Cell (E. coli)

10. Flux balance model

The metabolic core comprises six lumped reactions over six balanced internal metabolites (substrate S, pyruvate P, ATP, ADP, NADH, NAD⁺): uptake v_{upt} (bounded by uptake_max), glycolysis v_{gly} ($S + 2 \text{ ADP} + 2 \text{ NAD}^+ \rightarrow 2 \text{ P} + 2 \text{ ATP} + 2 \text{ NADH}$), the TCA lump v_{tca} ($\text{P} + 4 \text{ NAD}^+ + \text{ADP} \rightarrow 4 \text{ NADH} + \text{ATP}$), the electron transport lump v_{etc} ($\text{NADH} + 2.5 \text{ ADP} \rightarrow \text{NAD}^+ + 2.5 \text{ ATP}$; $\text{P/O} = 2.5$), ATP maintenance v_{atm} (≥ 1 , fixed), replication v_{bio} ($a_{\text{bio}} \text{ ATP} + b_{\text{bio}} \text{ P} + c_{\text{bio}} \text{ NADH} \rightarrow \text{biomass}$, with $a_{\text{bio}} = 30$, $b_{\text{bio}} = 4$, $c_{\text{bio}} = 6$), and the sensor reaction v_{sen} ($X + k_{\text{sen}} \text{ ATP} \rightarrow Z$, $k_{\text{sen}} = 2$). Reductant and ADP are returned oxidized/hydrolyzed by v_{bio} and v_{sen} so that cofactor pools balance. Steady state requires $S v = 0$ for internal metabolites, and the optimization is

$$(8) \quad \text{maximize } v_{\text{sen}} \quad \text{subject to} \quad S v = 0, \quad l \leq v \leq u$$

The full-oxidation ATP yield of one glucose in this network is 2 (glycolysis) $+ 2 \times (1 + 4 \times 2.5) = 24$ ATP, within the canonical 26-30 range of lumped core models; the small shortfall reflects omitted substrate-level phosphorylation in the TCA lump.

11. The Reallocation Identity

THEOREM. Let $v^*(g)$ be the optimal sensor flux with biomass floor $v_{\text{bio}} \geq g$. If the optimal basis is constant on $[0, g]$, then

$$(9) \quad v^*(0) - v^*(g) = g (a_{\text{bio}} + b_{\text{bio}} e_{\text{P}} + c_{\text{bio}} e_{\text{H}}) / k_{\text{sen}}$$

where $e_{\text{P}} = 1 + R(\text{P/O}) = 11$ is the ATP yield of one pyruvate ($R = 4$ NADH per TCA turn) and $e_{\text{H}} = \text{P/O} = 2.5$ the ATP yield of one NADH. PROOF. The biomass floor binds at the optimum (replication consumes resources the objective would otherwise use). Fix the optimal basis. Relaxing the floor from g to 0 frees, per unit g : a_{bio} ATP directly; b_{bio} pyruvate, each oxidized through v_{tca} and v_{etc} for e_{P} ATP; and c_{bio} NADH, each oxidized for e_{H} ATP. The total freed ATP-equivalent flux is $g(a_{\text{bio}} + b_{\text{bio}} e_{\text{P}} + c_{\text{bio}} e_{\text{H}})$. By strong duality, the objective gain equals the shadow price of the biomass constraint times the relaxation amount; the shadow price is exactly the freed ATP per unit g divided by k_{sen} , because the only ATP sink available to the objective is v_{sen} . Basis constancy keeps shadow prices constant on the interval, making the identity exact rather than first-order. QED.

COROLLARY (100% reallocation). With X unbounded and maintenance fixed, every freed ATP-equivalent flows to v_{sen} : the reallocated fraction is exactly 100%.

COUNTEREXAMPLE. With X bounded by $x_{\max}$ below the reallocation capacity, the freed energy has no sink beyond the cap and the reallocated fraction falls provably below 100%; measured 77.5% at the tested cap (Table 2b). The folklore claim 'freezing replication frees 100% of replication energy' is thus true in-model exactly when the downstream sink is unbounded - a falsifiable condition, not a universal law.

Table 2a. Numerical verification of the Reallocation Identity.

g	delta v_{sen} (LP)	identity (closed form)	 err
0.25	11.125	11.125	0.0e+00
0.5	22.25	22.25	0.0e+00
1.0	44.5	44.5	0.0e+00
1.5	66.75	66.75	0.0e+00
2.0	89.0	89.0	0.0e+00

Table 2b. Reallocation accounting at $g = 1$.

freed ATP-equivalents	extra sensor ATP spend	reallocated fraction	bounded-X counterexample
89.0	89.0	100.00%	77.53% (< 100%)

12. Discussion and limitations

The codec benchmark isolates substitution noise; indels, strand loss, and PCR bias are future work, as is scaling the parity scheme from one erasure to a Reed-Solomon outer code over $GF(3^k)$. The metabolic model is a six-reaction core: it proves the identity's structure but not its exact coefficients in *E. coli*, which require a genome-scale model. Both parts share a methodological stance: constraints and claims are proved where provable, measured where measurable, and the gap stated.

13. Reproducibility

All code is Python 3.10; the hermetic suite (20 tests) covers codec round-trips at multiple lengths, constraint guarantees, noise recovery, parity repair, stoichiometric balance, the reallocation identity at three g values, and the bounded-input counterexample.

Benchmarks use fixed seeds (message RNG seed 7, per-copy noise seeds $1000 + 97 \cdot \text{msg} + \text{copy}$). No network access is required for any test. Results JSONs: results/codec_benchmark.json, results/metabolism_identity.json.

9a. Algorithms in pseudocode

Algorithm 1 (Encode). Input: message m , copies c . 1: trits $\leftarrow$ header(m) || bytes_to_trits(m) || fletcher16(m). 2: trits $\leftarrow$ trits + keystream (mod 3). 3: partition into blocks $b_1..b_N$ of 32 trits, zero-padded; append 6-trit block checksum to each. 4: parity

<- sum_i b_i (mod 3); append parity block with checksum. 5: strand <- concat_i never_same(b_i, seed = sigma(i)). 6: emit c copies.

Algorithm 2 (Decode). Input: strands $s_1..s_c$. 1: voted <- positionwise majority of the strands. 2: for each block i: p_i <- try_block(voted, i); if None, p_i <- first passing single-copy block. 3: if exactly one payload block failed and parity passed: repair by subtraction (Eq. 5). 4: descramble, parse header, verify global Fletcher-16, return message; else raise.

9b. Sensitivity analysis

Table 1b. Codec recovery sensitivity (64-byte messages, 16 trials).

block size	copies	rec@1%	rec@2%	rec@3%
B = 16	3	0.9375	0.9375	0.8125
B = 32	3	0.9375	0.9375	0.75
B = 64	3	1.0	0.9375	0.6875
B = 32	3	0.9375	0.9375	0.75
B = 32	5	1.0	1.0	1.0

Two design conclusions follow. First, five replicates suffice for perfect recovery at all tested rates (16/16 at 1%, 2%, and 3%), showing the blocked code converts redundancy into reliability far more efficiently than the unblocked design. Second, block size trades checksum overhead against desynchronization span; $B = 32$ balances the two at the tested rates.

Table 2c. Sensor-cost sweep: Δv_{sen} scales as $1/k_{\text{sen}}$ exactly as the identity predicts.

k_{sen}	$v_{\text{sen}} (g=0)$	$v_{\text{sen}} (g=1)$	Δ
1.0	289.0	200.0	89.0
2.0	144.5	100.0	44.5
4.0	72.25	50.0	22.25

The k_{sen} sweep independently confirms the identity: measured deltas 89.0, 44.5, 22.25 equal $89/k_{\text{sen}}$ to machine precision, as Eq. 9 requires, since the freed ATP-equivalent flux (89 per unit g) is independent of k_{sen} while the flux conversion is not.

9c. Extended derivations

Derivation of Eq. 6 (vote failure). With three independent copies and per-base substitution probability p , the majority value at a position is wrong iff exactly two copies err ($3p^2(1-p)$ ways, and the two erroneous bases agree or outvote either way) or all three err (p^3). Summing gives $3p^2(1-p) + p^3 = 3p^2 - 2p^3$. Note the worst case for the vote is when erroneous bases are adversarially distinct; our model treats substitution targets as uniform over the three wrong bases, so two errors agree with probability $1/3$ and the bound is conservative.

An ideal uniformly sampled three-choice never-same chain has stationary GC fraction $1/2$, but successive GC indicators are not independent. The actual deterministic scrambler and six-trit byte mapping also need not realize that ideal chain. A binomial window variance or independent-window extreme-value approximation therefore cannot be used as a verified law for this implementation. Empirical GC statistics are retained separately.

Derivation of Eq. 1 (Perron root). $A = J - I$ has eigenvector $\mathbf{1} = (1,1,1,1)$ with eigenvalue 3, and any vector orthogonal to $\mathbf{1}$ has eigenvalue -1. Since A is irreducible and aperiodic (the constraint graph is complete minus loops with paths of all lengths), Perron-Frobenius gives a unique maximal eigenvalue 3, and the number of length- n admissible strands grows as $\sim 4 \times 3^{(n-1)}$, giving capacity $\log_2 3$ per base.

9d. The ODE arm: sensor-array dynamics

Complementing the steady-state LP, the repository models the sensor loop as an ODE: $dE/dt = \alpha Z - \beta E$ (sensor expression driven by processed input), with analytic steady state $E^* = (\alpha/\beta) Z$. The hermetic test suite integrates the ODE with RK4 and asserts agreement with the analytic steady state to $1e-6$, providing the dynamic counterpart of the static reallocation theorem: after replication freezing, sensor expression converges to the higher steady state with time constant $1/\beta$.

9f. Per-configuration analysis

Configuration: 64B messages

rsns (ours, RS-GF(3^5)): density 0.711 bits/base, worst homopolymer 1, worst-window GC deviation 0.140. Recovery was perfect at zero noise; at 1% substitution 23/24 messages were recovered, at 2% 21/24, and at 3% 10/24. RSNS does use inner error correction over GF(243). Failures beyond its error/erasure capacity are not Fountain peeling failures.

ours v1: density 0.418 bits/base, worst homopolymer 1, worst-window GC deviation 0.140. Recovery was perfect at zero noise; at 1% substitution 24/24 messages were recovered, at 2% 17/24, and at 3% 12/24. Historical attempted-channel output; no suffix-desynchronization mechanism or faithful published-codec equivalence is established.

ours v2 (blocked+parity): density 0.321 bits/base, worst homopolymer 2, worst-window GC deviation 0.160. Recovery was perfect at zero noise; at 1% substitution 24/24 messages were recovered, at 2% 19/24, and at 3% 19/24. The blocked design's erasure conversion is visible in the shallow degradation slope between 1% and 3%.

Goldman 2013: density 0.418 bits/base, worst homopolymer 1, worst-window GC deviation 0.180. Recovery was perfect at zero noise; at 1% substitution 22/24 messages were recovered, at 2% 17/24, and at 3% 12/24. Historical attempted-channel output; no

suffix-desynchronization mechanism or faithful published-codec equivalence is established.

DNA Fountain: density 0.627 bits/base, worst homopolymer 3, worst-window GC deviation 0.120. Recovery was perfect at zero noise; at 1% substitution 5/24 messages were recovered, at 2% 1/24, and at 3% 0/24. Without inner correction, a single substitution disables each affected droplet and the peeling decoder stalls.

Configuration: 256B messages

rsns (ours, RS-GF(3^5)): density 0.813 bits/base, worst homopolymer 1, worst-window GC deviation 0.160. Recovery was perfect at zero noise; at 1% substitution 24/24 messages were recovered, at 2% 14/24, and at 3% 2/24. RSNS does use inner error correction over GF(243). Failures beyond its error/erasure capacity are not Fountain peeling failures.

ours v1: density 0.438 bits/base, worst homopolymer 1, worst-window GC deviation 0.160. Recovery was perfect at zero noise; at 1% substitution 20/24 messages were recovered, at 2% 7/24, and at 3% 3/24. Historical attempted-channel output; no suffix-desynchronization mechanism or faithful published-codec equivalence is established.

ours v2 (blocked+parity): density 0.359 bits/base, worst homopolymer 2, worst-window GC deviation 0.180. Recovery was perfect at zero noise; at 1% substitution 21/24 messages were recovered, at 2% 10/24, and at 3% 9/24. The blocked design's erasure conversion is visible in the shallow degradation slope between 1% and 3%.

Goldman 2013: density 0.438 bits/base, worst homopolymer 1, worst-window GC deviation 0.180. Recovery was perfect at zero noise; at 1% substitution 20/24 messages were recovered, at 2% 7/24, and at 3% 3/24. Historical attempted-channel output; no suffix-desynchronization mechanism or faithful published-codec equivalence is established.

DNA Fountain: density 0.717 bits/base, worst homopolymer 3, worst-window GC deviation 0.160. Recovery was perfect at zero noise; at 1% substitution 12/24 messages were recovered, at 2% 0/24, and at 3% 0/24. Without inner correction, a single substitution disables each affected droplet and the peeling decoder stalls.

9g. Threats to validity

Internal validity. The substitution channel is memoryless with uniform error bases; real synthesis errors are position- and context-dependent, so absolute recovery numbers will shift, though the codec ordering is mechanism-driven and should persist. Trial counts (24 per configuration) give binomial standard errors of at most 0.10 at recovery 0.5; the headline gaps (e.g., 0.83 vs 0.50 at 2%) exceed 3 standard errors. Construct validity. 'Recovery' is exact message equality verified by checksum; partial recovery is counted as

failure, which is the correct metric for archival storage but understates Fountain's per-droplet survival. External validity. The Goldman baseline replaces his four-fold segmented redundancy with the same three-copy replication as the others, isolating the encoding; a full Goldman redundancy implementation would narrow but not reverse the 2-3% gap, since his redundancy protects against strand loss, not within-strand desync.

11a. Application: the zero-waste biosensor array

The original motivation for the replication-frozen host model is a biosensor array for cancer tracking: replication-frozen bacteria that cannot divide cannot colonize a host or escape containment, and - by the Reallocation Identity - channel a quantifiable, maximized flux into processing complex chemical inputs (the sensed analytes). The identity supplies the design equation for such an array: sensor throughput gained by freezing replication equals the freed ATP-equivalent flux divided by the per-analyte processing cost, so array sensitivity scales linearly with the replication rate that was frozen. The bounded-input counterexample doubles as a design warning: below the analyte concentration that saturates the sensor reaction, the frozen cell's spare energy has no productive sink, so gain is capped by analyte availability rather than by the reallocation budget - array geometry must match expected analyte concentrations to the reallocated flux.

11b. Future work

Codec: (i) indel tolerance via marker-free resynchronization using the block-seed cycling as a soft sync signal; (ii) a $GF(3^2)$ Reed-Solomon outer code replacing single parity, lifting correction from one to t erased blocks; (iii) strand-loss modeling with erasure-channel capacity analysis; (iv) wet-lab validation on a commercial synthesis pilot. Metabolism: (i) port the identity to a genome-scale model and compare coefficients; (ii) dynamic FBA of the post-freezing transient; (iii) experimental calibration of a_{bio} , b_{bio} , c_{bio} from growth-arrested chemostat data; (iv) coupling the two parts: encode the sensor program itself in the codec of Part I, closing the storage-to-phenotype loop.

Appendix D. Hermetic test suite (20 tests)

- test_roundtrip_no_noise / test_odd_length_roundtrip: v2 codec recovers messages of lengths 1, 5, 31, 32, 33, 100 bytes exactly.

- test_constraints: v2 strands keep homopolymer ≤ 2 and GC within 0.15 of 0.5 on random payloads.

- test_low_noise_recovers: v2 recovers at 0.5% substitution with independent per-copy noise.

- test_single_block_erasure_repaired_by_parity: a fully corrupted block in all copies is repaired by the parity tier.
- test_steady_state_residual: the LP solution satisfies $S v = 0$ on internal metabolites to $1e-8$.
- test_freezing_replication_increases_sensor_flux: $v^*(0) > v^*(1)$.
- test_reallocation_identity_exact: LP deltas match the closed form to $1e-9$ at $g = 0.5, 1.0, 2.0$.
- test_bounded_input_breaks_full_reallocation: capped input provably reduces the reallocated fraction.
- plus the v1 codec suite: trit/byte round-trips, never-same guarantees, GC balance under scrambling, checksum detection, ODE steady-state match (RK4 vs analytic, $1e-6$), and host-loop conservation checks.

Appendix A. Notation

p - per-base substitution probability

B - payload trits per block (32)

$\sigma(i)$ - seed base of block i , cycling A,C,G,T

e_P - ATP yield per pyruvate (11)

e_H - ATP yield per NADH (2.5)

k_{sen} - ATP cost per unit complex input processed (2)

$v^*(g)$ - optimal sensor flux with biomass floor g

a_{bio}, b_{bio}, c_{bio} - biomass reaction costs: 30 ATP, 4 P, 6 NADH

Appendix B. Full per-trial benchmark log (768 rows)

Table B1. Every benchmark trial: codec, message size, message index, substitution rate, recovered (1/0).

codec	size	msg#	rate	rec
ours	64	0	0.0	1
ours	64	0	0.01	1
ours	64	0	0.02	1
ours	64	0	0.03	1
ours	64	1	0.0	1

ours	64	1	0.01	1
ours	64	1	0.02	1
ours	64	1	0.03	1
ours	64	2	0.0	1
ours	64	2	0.01	1
ours	64	2	0.02	1
ours	64	2	0.03	0
ours	64	3	0.0	1
ours	64	3	0.01	1
ours	64	3	0.02	1
ours	64	3	0.03	1
ours	64	4	0.0	1
ours	64	4	0.01	1
ours	64	4	0.02	0
ours	64	4	0.03	0
ours	64	5	0.0	1
ours	64	5	0.01	1
ours	64	5	0.02	0
ours	64	5	0.03	1
ours	64	6	0.0	1
ours	64	6	0.01	1
ours	64	6	0.02	0
ours	64	6	0.03	0
ours	64	7	0.0	1
ours	64	7	0.01	1
ours	64	7	0.02	0
ours	64	7	0.03	0
ours	64	8	0.0	1
ours	64	8	0.01	1
ours	64	8	0.02	1
ours	64	8	0.03	0
ours	64	9	0.0	1
ours	64	9	0.01	1
ours	64	9	0.02	1
ours	64	9	0.03	1
ours	64	10	0.0	1
ours	64	10	0.01	1
ours	64	10	0.02	1
ours	64	10	0.03	1
ours	64	11	0.0	1
ours	64	11	0.01	1
ours	64	11	0.02	1
ours	64	11	0.03	1
ours	64	12	0.0	1
ours	64	12	0.01	1
ours	64	12	0.02	1
ours	64	12	0.03	0
ours	64	13	0.0	1
ours	64	13	0.01	1
ours	64	13	0.02	1
ours	64	13	0.03	1
ours	64	14	0.0	1
ours	64	14	0.01	1

ours	64	14	0.02	1
ours	64	14	0.03	0
ours	64	15	0.0	1
ours	64	15	0.01	1
ours	64	15	0.02	1
ours	64	15	0.03	1
ours	64	16	0.0	1
ours	64	16	0.01	1
ours	64	16	0.02	1
ours	64	16	0.03	0
ours	64	17	0.0	1
ours	64	17	0.01	1
ours	64	17	0.02	1
ours	64	17	0.03	1
ours	64	18	0.0	1
ours	64	18	0.01	1
ours	64	18	0.02	1
ours	64	18	0.03	0
ours	64	19	0.0	1
ours	64	19	0.01	1
ours	64	19	0.02	1
ours	64	19	0.03	0
ours	64	20	0.0	1
ours	64	20	0.01	1
ours	64	20	0.02	0
ours	64	20	0.03	0
ours	64	21	0.0	1
ours	64	21	0.01	1
ours	64	21	0.02	1
ours	64	21	0.03	1
ours	64	22	0.0	1
ours	64	22	0.01	1
ours	64	22	0.02	0
ours	64	22	0.03	1
ours	64	23	0.0	1
ours	64	23	0.01	1
ours	64	23	0.02	0
ours	64	23	0.03	0
ours_v2	64	0	0.0	1
ours_v2	64	0	0.01	1
ours_v2	64	0	0.02	1
ours_v2	64	0	0.03	1
ours_v2	64	1	0.0	1
ours_v2	64	1	0.01	1
ours_v2	64	1	0.02	1
ours_v2	64	1	0.03	1
ours_v2	64	2	0.0	1
ours_v2	64	2	0.01	1
ours_v2	64	2	0.02	0
ours_v2	64	2	0.03	0
ours_v2	64	3	0.0	1
ours_v2	64	3	0.01	1
ours_v2	64	3	0.02	1

ours_v2	64	3	0.03	1
ours_v2	64	4	0.0	1
ours_v2	64	4	0.01	1
ours_v2	64	4	0.02	0
ours_v2	64	4	0.03	0
ours_v2	64	5	0.0	1
ours_v2	64	5	0.01	1
ours_v2	64	5	0.02	0
ours_v2	64	5	0.03	1
ours_v2	64	6	0.0	1
ours_v2	64	6	0.01	1
ours_v2	64	6	0.02	0
ours_v2	64	6	0.03	1
ours_v2	64	7	0.0	1
ours_v2	64	7	0.01	1
ours_v2	64	7	0.02	1
ours_v2	64	7	0.03	0
ours_v2	64	8	0.0	1
ours_v2	64	8	0.01	1
ours_v2	64	8	0.02	1
ours_v2	64	8	0.03	1
ours_v2	64	9	0.0	1
ours_v2	64	9	0.01	1
ours_v2	64	9	0.02	1
ours_v2	64	9	0.03	1
ours_v2	64	10	0.0	1
ours_v2	64	10	0.01	1
ours_v2	64	10	0.02	0
ours_v2	64	10	0.03	1
ours_v2	64	11	0.0	1
ours_v2	64	11	0.01	1
ours_v2	64	11	0.02	1
ours_v2	64	11	0.03	1
ours_v2	64	12	0.0	1
ours_v2	64	12	0.01	1
ours_v2	64	12	0.02	1
ours_v2	64	12	0.03	0
ours_v2	64	13	0.0	1
ours_v2	64	13	0.01	1
ours_v2	64	13	0.02	1
ours_v2	64	13	0.03	1
ours_v2	64	14	0.0	1
ours_v2	64	14	0.01	1
ours_v2	64	14	0.02	1
ours_v2	64	14	0.03	1
ours_v2	64	15	0.0	1
ours_v2	64	15	0.01	1
ours_v2	64	15	0.02	1
ours_v2	64	15	0.03	0
ours_v2	64	16	0.0	1
ours_v2	64	16	0.01	1
ours_v2	64	16	0.02	1
ours_v2	64	16	0.03	1

ours_v2	64	17	0.0	1
ours_v2	64	17	0.01	1
ours_v2	64	17	0.02	1
ours_v2	64	17	0.03	1
ours_v2	64	18	0.0	1
ours_v2	64	18	0.01	1
ours_v2	64	18	0.02	1
ours_v2	64	18	0.03	1
ours_v2	64	19	0.0	1
ours_v2	64	19	0.01	1
ours_v2	64	19	0.02	1
ours_v2	64	19	0.03	1
ours_v2	64	20	0.0	1
ours_v2	64	20	0.01	1
ours_v2	64	20	0.02	1
ours_v2	64	20	0.03	1
ours_v2	64	21	0.0	1
ours_v2	64	21	0.01	1
ours_v2	64	21	0.02	1
ours_v2	64	21	0.03	1
ours_v2	64	22	0.0	1
ours_v2	64	22	0.01	1
ours_v2	64	22	0.02	1
ours_v2	64	22	0.03	1
ours_v2	64	23	0.0	1
ours_v2	64	23	0.01	1
ours_v2	64	23	0.02	1
ours_v2	64	23	0.03	1
ours_2x	64	0	0.0	1
ours_2x	64	0	0.01	0
ours_2x	64	0	0.02	0
ours_2x	64	0	0.03	0
ours_2x	64	1	0.0	1
ours_2x	64	1	0.01	0
ours_2x	64	1	0.02	0
ours_2x	64	1	0.03	0
ours_2x	64	2	0.0	1
ours_2x	64	2	0.01	0
ours_2x	64	2	0.02	0
ours_2x	64	2	0.03	0
ours_2x	64	3	0.0	1
ours_2x	64	3	0.01	0
ours_2x	64	3	0.02	0
ours_2x	64	3	0.03	0
ours_2x	64	4	0.0	1
ours_2x	64	4	0.01	0
ours_2x	64	4	0.02	0
ours_2x	64	4	0.03	0
ours_2x	64	5	0.0	1
ours_2x	64	5	0.01	0
ours_2x	64	5	0.02	0
ours_2x	64	5	0.03	0
ours_2x	64	6	0.0	1

ours_2x	64	6	0.01	0
ours_2x	64	6	0.02	0
ours_2x	64	6	0.03	0
ours_2x	64	7	0.0	1
ours_2x	64	7	0.01	0
ours_2x	64	7	0.02	0
ours_2x	64	7	0.03	0
ours_2x	64	8	0.0	1
ours_2x	64	8	0.01	0
ours_2x	64	8	0.02	0
ours_2x	64	8	0.03	0
ours_2x	64	9	0.0	1
ours_2x	64	9	0.01	0
ours_2x	64	9	0.02	0
ours_2x	64	9	0.03	0
ours_2x	64	10	0.0	1
ours_2x	64	10	0.01	0
ours_2x	64	10	0.02	0
ours_2x	64	10	0.03	0
ours_2x	64	11	0.0	1
ours_2x	64	11	0.01	0
ours_2x	64	11	0.02	0
ours_2x	64	11	0.03	0
ours_2x	64	12	0.0	1
ours_2x	64	12	0.01	0
ours_2x	64	12	0.02	0
ours_2x	64	12	0.03	0
ours_2x	64	13	0.0	1
ours_2x	64	13	0.01	0
ours_2x	64	13	0.02	0
ours_2x	64	13	0.03	0
ours_2x	64	14	0.0	1
ours_2x	64	14	0.01	0
ours_2x	64	14	0.02	0
ours_2x	64	14	0.03	0
ours_2x	64	15	0.0	1
ours_2x	64	15	0.01	0
ours_2x	64	15	0.02	0
ours_2x	64	15	0.03	0
ours_2x	64	16	0.0	1
ours_2x	64	16	0.01	0
ours_2x	64	16	0.02	0
ours_2x	64	16	0.03	0
ours_2x	64	17	0.0	1
ours_2x	64	17	0.01	0
ours_2x	64	17	0.02	0
ours_2x	64	17	0.03	0
ours_2x	64	18	0.0	1
ours_2x	64	18	0.01	0
ours_2x	64	18	0.02	0
ours_2x	64	18	0.03	0
ours_2x	64	19	0.0	1
ours_2x	64	19	0.01	0

ours_2x	64	19	0.02	0
ours_2x	64	19	0.03	0
ours_2x	64	20	0.0	1
ours_2x	64	20	0.01	0
ours_2x	64	20	0.02	0
ours_2x	64	20	0.03	0
ours_2x	64	21	0.0	1
ours_2x	64	21	0.01	0
ours_2x	64	21	0.02	0
ours_2x	64	21	0.03	0
ours_2x	64	22	0.0	1
ours_2x	64	22	0.01	0
ours_2x	64	22	0.02	0
ours_2x	64	22	0.03	0
ours_2x	64	23	0.0	1
ours_2x	64	23	0.01	0
ours_2x	64	23	0.02	0
ours_2x	64	23	0.03	0
ours_v2_2x	64	0	0.0	1
ours_v2_2x	64	0	0.01	1
ours_v2_2x	64	0	0.02	0
ours_v2_2x	64	0	0.03	0
ours_v2_2x	64	1	0.0	1
ours_v2_2x	64	1	0.01	0
ours_v2_2x	64	1	0.02	0
ours_v2_2x	64	1	0.03	0
ours_v2_2x	64	2	0.0	1
ours_v2_2x	64	2	0.01	1
ours_v2_2x	64	2	0.02	0
ours_v2_2x	64	2	0.03	0
ours_v2_2x	64	3	0.0	1
ours_v2_2x	64	3	0.01	0
ours_v2_2x	64	3	0.02	0
ours_v2_2x	64	3	0.03	0
ours_v2_2x	64	4	0.0	1
ours_v2_2x	64	4	0.01	1
ours_v2_2x	64	4	0.02	0
ours_v2_2x	64	4	0.03	0
ours_v2_2x	64	5	0.0	1
ours_v2_2x	64	5	0.01	1
ours_v2_2x	64	5	0.02	0
ours_v2_2x	64	5	0.03	0
ours_v2_2x	64	6	0.0	1
ours_v2_2x	64	6	0.01	0
ours_v2_2x	64	6	0.02	0
ours_v2_2x	64	6	0.03	0
ours_v2_2x	64	7	0.0	1
ours_v2_2x	64	7	0.01	0
ours_v2_2x	64	7	0.02	0
ours_v2_2x	64	7	0.03	0
ours_v2_2x	64	8	0.0	1
ours_v2_2x	64	8	0.01	0
ours_v2_2x	64	8	0.02	0

ours_v2_2x	64	8	0.03	0
ours_v2_2x	64	9	0.0	1
ours_v2_2x	64	9	0.01	0
ours_v2_2x	64	9	0.02	0
ours_v2_2x	64	9	0.03	0
ours_v2_2x	64	10	0.0	1
ours_v2_2x	64	10	0.01	1
ours_v2_2x	64	10	0.02	0
ours_v2_2x	64	10	0.03	0
ours_v2_2x	64	11	0.0	1
ours_v2_2x	64	11	0.01	0
ours_v2_2x	64	11	0.02	0
ours_v2_2x	64	11	0.03	0
ours_v2_2x	64	12	0.0	1
ours_v2_2x	64	12	0.01	1
ours_v2_2x	64	12	0.02	1
ours_v2_2x	64	12	0.03	0
ours_v2_2x	64	13	0.0	1
ours_v2_2x	64	13	0.01	0
ours_v2_2x	64	13	0.02	0
ours_v2_2x	64	13	0.03	0
ours_v2_2x	64	14	0.0	1
ours_v2_2x	64	14	0.01	1
ours_v2_2x	64	14	0.02	0
ours_v2_2x	64	14	0.03	0
ours_v2_2x	64	15	0.0	1
ours_v2_2x	64	15	0.01	0
ours_v2_2x	64	15	0.02	0
ours_v2_2x	64	15	0.03	0
ours_v2_2x	64	16	0.0	1
ours_v2_2x	64	16	0.01	0
ours_v2_2x	64	16	0.02	0
ours_v2_2x	64	16	0.03	0
ours_v2_2x	64	17	0.0	1
ours_v2_2x	64	17	0.01	0
ours_v2_2x	64	17	0.02	0
ours_v2_2x	64	17	0.03	0
ours_v2_2x	64	18	0.0	1
ours_v2_2x	64	18	0.01	0
ours_v2_2x	64	18	0.02	0
ours_v2_2x	64	18	0.03	0
ours_v2_2x	64	19	0.0	1
ours_v2_2x	64	19	0.01	0
ours_v2_2x	64	19	0.02	0
ours_v2_2x	64	19	0.03	0
ours_v2_2x	64	20	0.0	1
ours_v2_2x	64	20	0.01	0
ours_v2_2x	64	20	0.02	0
ours_v2_2x	64	20	0.03	0
ours_v2_2x	64	21	0.0	1
ours_v2_2x	64	21	0.01	0
ours_v2_2x	64	21	0.02	0
ours_v2_2x	64	21	0.03	0

ours_v2_2x	64	22	0.0	1
ours_v2_2x	64	22	0.01	1
ours_v2_2x	64	22	0.02	0
ours_v2_2x	64	22	0.03	0
ours_v2_2x	64	23	0.0	1
ours_v2_2x	64	23	0.01	0
ours_v2_2x	64	23	0.02	0
ours_v2_2x	64	23	0.03	0
goldman	64	0	0.0	1
goldman	64	0	0.01	1
goldman	64	0	0.02	1
goldman	64	0	0.03	1
goldman	64	1	0.0	1
goldman	64	1	0.01	0
goldman	64	1	0.02	0
goldman	64	1	0.03	0
goldman	64	2	0.0	1
goldman	64	2	0.01	1
goldman	64	2	0.02	1
goldman	64	2	0.03	1
goldman	64	3	0.0	1
goldman	64	3	0.01	1
goldman	64	3	0.02	1
goldman	64	3	0.03	1
goldman	64	4	0.0	1
goldman	64	4	0.01	1
goldman	64	4	0.02	0
goldman	64	4	0.03	0
goldman	64	5	0.0	1
goldman	64	5	0.01	1
goldman	64	5	0.02	0
goldman	64	5	0.03	1
goldman	64	6	0.0	1
goldman	64	6	0.01	1
goldman	64	6	0.02	1
goldman	64	6	0.03	0
goldman	64	7	0.0	1
goldman	64	7	0.01	1
goldman	64	7	0.02	1
goldman	64	7	0.03	1
goldman	64	8	0.0	1
goldman	64	8	0.01	0
goldman	64	8	0.02	1
goldman	64	8	0.03	1
goldman	64	9	0.0	1
goldman	64	9	0.01	1
goldman	64	9	0.02	1
goldman	64	9	0.03	0
goldman	64	10	0.0	1
goldman	64	10	0.01	1
goldman	64	10	0.02	1
goldman	64	10	0.03	1
goldman	64	11	0.0	1

goldman	64	11	0.01	1
goldman	64	11	0.02	0
goldman	64	11	0.03	0
goldman	64	12	0.0	1
goldman	64	12	0.01	1
goldman	64	12	0.02	1
goldman	64	12	0.03	0
goldman	64	13	0.0	1
goldman	64	13	0.01	1
goldman	64	13	0.02	1
goldman	64	13	0.03	1
goldman	64	14	0.0	1
goldman	64	14	0.01	1
goldman	64	14	0.02	1
goldman	64	14	0.03	0
goldman	64	15	0.0	1
goldman	64	15	0.01	1
goldman	64	15	0.02	1
goldman	64	15	0.03	0
goldman	64	16	0.0	1
goldman	64	16	0.01	1
goldman	64	16	0.02	1
goldman	64	16	0.03	1
goldman	64	17	0.0	1
goldman	64	17	0.01	1
goldman	64	17	0.02	1
goldman	64	17	0.03	1
goldman	64	18	0.0	1
goldman	64	18	0.01	1
goldman	64	18	0.02	1
goldman	64	18	0.03	0
goldman	64	19	0.0	1
goldman	64	19	0.01	1
goldman	64	19	0.02	1
goldman	64	19	0.03	0
goldman	64	20	0.0	1
goldman	64	20	0.01	1
goldman	64	20	0.02	0
goldman	64	20	0.03	0
goldman	64	21	0.0	1
goldman	64	21	0.01	1
goldman	64	21	0.02	1
goldman	64	21	0.03	0
goldman	64	22	0.0	1
goldman	64	22	0.01	1
goldman	64	22	0.02	0
goldman	64	22	0.03	1
goldman	64	23	0.0	1
goldman	64	23	0.01	1
goldman	64	23	0.02	0
goldman	64	23	0.03	1
fountain	64	0	0.0	0
fountain	64	0	0.01	0

fountain	64	0	0.02	0
fountain	64	0	0.03	0
fountain	64	1	0.0	0
fountain	64	1	0.01	0
fountain	64	1	0.02	0
fountain	64	1	0.03	0
fountain	64	2	0.0	0
fountain	64	2	0.01	0
fountain	64	2	0.02	0
fountain	64	2	0.03	0
fountain	64	3	0.0	0
fountain	64	3	0.01	0
fountain	64	3	0.02	0
fountain	64	3	0.03	0
fountain	64	4	0.0	1
fountain	64	4	0.01	0
fountain	64	4	0.02	0
fountain	64	4	0.03	0
fountain	64	5	0.0	1
fountain	64	5	0.01	0
fountain	64	5	0.02	0
fountain	64	5	0.03	0
fountain	64	6	0.0	1
fountain	64	6	0.01	1
fountain	64	6	0.02	1
fountain	64	6	0.03	0
fountain	64	7	0.0	1
fountain	64	7	0.01	0
fountain	64	7	0.02	0
fountain	64	7	0.03	0
fountain	64	8	0.0	0
fountain	64	8	0.01	0
fountain	64	8	0.02	0
fountain	64	8	0.03	0
fountain	64	9	0.0	0
fountain	64	9	0.01	0
fountain	64	9	0.02	0
fountain	64	9	0.03	0
fountain	64	10	0.0	1
fountain	64	10	0.01	0
fountain	64	10	0.02	0
fountain	64	10	0.03	0
fountain	64	11	0.0	1
fountain	64	11	0.01	1
fountain	64	11	0.02	0
fountain	64	11	0.03	0
fountain	64	12	0.0	1
fountain	64	12	0.01	1
fountain	64	12	0.02	0
fountain	64	12	0.03	0
fountain	64	13	0.0	0
fountain	64	13	0.01	0
fountain	64	13	0.02	0

fountain	64	13	0.03	0
fountain	64	14	0.0	0
fountain	64	14	0.01	0
fountain	64	14	0.02	0
fountain	64	14	0.03	0
fountain	64	15	0.0	0
fountain	64	15	0.01	0
fountain	64	15	0.02	0
fountain	64	15	0.03	0
fountain	64	16	0.0	1
fountain	64	16	0.01	1
fountain	64	16	0.02	0
fountain	64	16	0.03	0
fountain	64	17	0.0	0
fountain	64	17	0.01	0
fountain	64	17	0.02	0
fountain	64	17	0.03	0
fountain	64	18	0.0	0
fountain	64	18	0.01	0
fountain	64	18	0.02	0
fountain	64	18	0.03	0
fountain	64	19	0.0	0
fountain	64	19	0.01	0
fountain	64	19	0.02	0
fountain	64	19	0.03	0
fountain	64	20	0.0	1
fountain	64	20	0.01	0
fountain	64	20	0.02	0
fountain	64	20	0.03	0
fountain	64	21	0.0	0
fountain	64	21	0.01	0
fountain	64	21	0.02	0
fountain	64	21	0.03	0
fountain	64	22	0.0	0
fountain	64	22	0.01	0
fountain	64	22	0.02	0
fountain	64	22	0.03	0
fountain	64	23	0.0	1
fountain	64	23	0.01	1
fountain	64	23	0.02	0
fountain	64	23	0.03	0
nsfountain	64	0	0.0	0
nsfountain	64	0	0.01	0
nsfountain	64	0	0.02	0
nsfountain	64	0	0.03	0
nsfountain	64	1	0.0	0
nsfountain	64	1	0.01	0
nsfountain	64	1	0.02	0
nsfountain	64	1	0.03	0
nsfountain	64	2	0.0	0
nsfountain	64	2	0.01	0
nsfountain	64	2	0.02	0
nsfountain	64	2	0.03	0

nsfountain	64	3	0.0	0
nsfountain	64	3	0.01	0
nsfountain	64	3	0.02	0
nsfountain	64	3	0.03	0
nsfountain	64	4	0.0	0
nsfountain	64	4	0.01	0
nsfountain	64	4	0.02	0
nsfountain	64	4	0.03	0
nsfountain	64	5	0.0	0
nsfountain	64	5	0.01	0
nsfountain	64	5	0.02	0
nsfountain	64	5	0.03	0
nsfountain	64	6	0.0	0
nsfountain	64	6	0.01	0
nsfountain	64	6	0.02	0
nsfountain	64	6	0.03	0
nsfountain	64	7	0.0	0
nsfountain	64	7	0.01	0
nsfountain	64	7	0.02	0
nsfountain	64	7	0.03	0
nsfountain	64	8	0.0	0
nsfountain	64	8	0.01	0
nsfountain	64	8	0.02	0
nsfountain	64	8	0.03	0
nsfountain	64	9	0.0	0
nsfountain	64	9	0.01	0
nsfountain	64	9	0.02	0
nsfountain	64	9	0.03	0
nsfountain	64	10	0.0	0
nsfountain	64	10	0.01	0
nsfountain	64	10	0.02	0
nsfountain	64	10	0.03	0
nsfountain	64	11	0.0	0
nsfountain	64	11	0.01	0
nsfountain	64	11	0.02	0
nsfountain	64	11	0.03	0
nsfountain	64	12	0.0	0
nsfountain	64	12	0.01	0
nsfountain	64	12	0.02	0
nsfountain	64	12	0.03	0
nsfountain	64	13	0.0	0
nsfountain	64	13	0.01	0
nsfountain	64	13	0.02	0
nsfountain	64	13	0.03	0
nsfountain	64	14	0.0	0
nsfountain	64	14	0.01	0
nsfountain	64	14	0.02	0
nsfountain	64	14	0.03	0
nsfountain	64	15	0.0	0
nsfountain	64	15	0.01	0
nsfountain	64	15	0.02	0
nsfountain	64	15	0.03	0
nsfountain	64	16	0.0	0

nsfountain	64	16	0.01	0
nsfountain	64	16	0.02	0
nsfountain	64	16	0.03	0
nsfountain	64	17	0.0	0
nsfountain	64	17	0.01	0
nsfountain	64	17	0.02	0
nsfountain	64	17	0.03	0
nsfountain	64	18	0.0	0
nsfountain	64	18	0.01	0
nsfountain	64	18	0.02	0
nsfountain	64	18	0.03	0
nsfountain	64	19	0.0	0
nsfountain	64	19	0.01	0
nsfountain	64	19	0.02	0
nsfountain	64	19	0.03	0
nsfountain	64	20	0.0	0
nsfountain	64	20	0.01	0
nsfountain	64	20	0.02	0
nsfountain	64	20	0.03	0
nsfountain	64	21	0.0	0
nsfountain	64	21	0.01	0
nsfountain	64	21	0.02	0
nsfountain	64	21	0.03	0
nsfountain	64	22	0.0	0
nsfountain	64	22	0.01	0
nsfountain	64	22	0.02	0
nsfountain	64	22	0.03	0
nsfountain	64	23	0.0	0
nsfountain	64	23	0.01	0
nsfountain	64	23	0.02	0
nsfountain	64	23	0.03	0
rsns	64	0	0.0	1
rsns	64	0	0.01	1
rsns	64	0	0.02	1
rsns	64	0	0.03	1
rsns	64	1	0.0	1
rsns	64	1	0.01	1
rsns	64	1	0.02	1
rsns	64	1	0.03	0
rsns	64	2	0.0	1
rsns	64	2	0.01	1
rsns	64	2	0.02	1
rsns	64	2	0.03	1
rsns	64	3	0.0	1
rsns	64	3	0.01	1
rsns	64	3	0.02	1
rsns	64	3	0.03	1
rsns	64	4	0.0	1
rsns	64	4	0.01	1
rsns	64	4	0.02	0
rsns	64	4	0.03	0
rsns	64	5	0.0	1
rsns	64	5	0.01	0

rsns	64	5	0.02	0
rsns	64	5	0.03	0
rsns	64	6	0.0	1
rsns	64	6	0.01	1
rsns	64	6	0.02	1
rsns	64	6	0.03	0
rsns	64	7	0.0	1
rsns	64	7	0.01	1
rsns	64	7	0.02	1
rsns	64	7	0.03	0
rsns	64	8	0.0	1
rsns	64	8	0.01	1
rsns	64	8	0.02	1
rsns	64	8	0.03	1
rsns	64	9	0.0	1
rsns	64	9	0.01	1
rsns	64	9	0.02	1
rsns	64	9	0.03	0
rsns	64	10	0.0	1
rsns	64	10	0.01	1
rsns	64	10	0.02	1
rsns	64	10	0.03	0
rsns	64	11	0.0	1
rsns	64	11	0.01	1
rsns	64	11	0.02	1
rsns	64	11	0.03	1
rsns	64	12	0.0	1
rsns	64	12	0.01	1
rsns	64	12	0.02	1
rsns	64	12	0.03	0
rsns	64	13	0.0	1
rsns	64	13	0.01	1
rsns	64	13	0.02	1
rsns	64	13	0.03	1
rsns	64	14	0.0	1
rsns	64	14	0.01	1
rsns	64	14	0.02	1
rsns	64	14	0.03	0
rsns	64	15	0.0	1
rsns	64	15	0.01	1
rsns	64	15	0.02	1
rsns	64	15	0.03	0
rsns	64	16	0.0	1
rsns	64	16	0.01	1
rsns	64	16	0.02	1
rsns	64	16	0.03	0
rsns	64	17	0.0	1
rsns	64	17	0.01	1
rsns	64	17	0.02	1
rsns	64	17	0.03	1
rsns	64	18	0.0	1
rsns	64	18	0.01	1
rsns	64	18	0.02	1

rsns	64	18	0.03	0
rsns	64	19	0.0	1
rsns	64	19	0.01	1
rsns	64	19	0.02	1
rsns	64	19	0.03	1
rsns	64	20	0.0	1
rsns	64	20	0.01	1
rsns	64	20	0.02	1
rsns	64	20	0.03	0
rsns	64	21	0.0	1
rsns	64	21	0.01	1
rsns	64	21	0.02	1
rsns	64	21	0.03	1
rsns	64	22	0.0	1
rsns	64	22	0.01	1
rsns	64	22	0.02	1
rsns	64	22	0.03	0
rsns	64	23	0.0	1
rsns	64	23	0.01	1
rsns	64	23	0.02	0
rsns	64	23	0.03	1
ours	256	0	0.0	1
ours	256	0	0.01	1
ours	256	0	0.02	0
ours	256	0	0.03	0
ours	256	1	0.0	1
ours	256	1	0.01	0
ours	256	1	0.02	0
ours	256	1	0.03	0
ours	256	2	0.0	1
ours	256	2	0.01	1
ours	256	2	0.02	1
ours	256	2	0.03	0
ours	256	3	0.0	1
ours	256	3	0.01	1
ours	256	3	0.02	0
ours	256	3	0.03	0
ours	256	4	0.0	1
ours	256	4	0.01	1
ours	256	4	0.02	0
ours	256	4	0.03	0
ours	256	5	0.0	1
ours	256	5	0.01	1
ours	256	5	0.02	0
ours	256	5	0.03	0
ours	256	6	0.0	1
ours	256	6	0.01	1
ours	256	6	0.02	0
ours	256	6	0.03	0
ours	256	7	0.0	1
ours	256	7	0.01	1
ours	256	7	0.02	0
ours	256	7	0.03	0

ours	256	8	0.0	1
ours	256	8	0.01	0
ours	256	8	0.02	1
ours	256	8	0.03	1
ours	256	9	0.0	1
ours	256	9	0.01	1
ours	256	9	0.02	1
ours	256	9	0.03	0
ours	256	10	0.0	1
ours	256	10	0.01	1
ours	256	10	0.02	0
ours	256	10	0.03	1
ours	256	11	0.0	1
ours	256	11	0.01	1
ours	256	11	0.02	0
ours	256	11	0.03	0
ours	256	12	0.0	1
ours	256	12	0.01	0
ours	256	12	0.02	0
ours	256	12	0.03	0
ours	256	13	0.0	1
ours	256	13	0.01	0
ours	256	13	0.02	0
ours	256	13	0.03	0
ours	256	14	0.0	1
ours	256	14	0.01	1
ours	256	14	0.02	1
ours	256	14	0.03	0
ours	256	15	0.0	1
ours	256	15	0.01	1
ours	256	15	0.02	1
ours	256	15	0.03	0
ours	256	16	0.0	1
ours	256	16	0.01	1
ours	256	16	0.02	0
ours	256	16	0.03	0
ours	256	17	0.0	1
ours	256	17	0.01	1
ours	256	17	0.02	0
ours	256	17	0.03	0
ours	256	18	0.0	1
ours	256	18	0.01	1
ours	256	18	0.02	1
ours	256	18	0.03	0
ours	256	19	0.0	1
ours	256	19	0.01	1
ours	256	19	0.02	1
ours	256	19	0.03	0
ours	256	20	0.0	1
ours	256	20	0.01	1
ours	256	20	0.02	0
ours	256	20	0.03	0
ours	256	21	0.0	1

ours	256	21	0.01	1
ours	256	21	0.02	0
ours	256	21	0.03	1
ours	256	22	0.0	1
ours	256	22	0.01	1
ours	256	22	0.02	0
ours	256	22	0.03	0
ours	256	23	0.0	1
ours	256	23	0.01	1
ours	256	23	0.02	0
ours	256	23	0.03	0
ours_v2	256	0	0.0	1
ours_v2	256	0	0.01	1
ours_v2	256	0	0.02	0
ours_v2	256	0	0.03	1
ours_v2	256	1	0.0	1
ours_v2	256	1	0.01	1
ours_v2	256	1	0.02	0
ours_v2	256	1	0.03	0
ours_v2	256	2	0.0	1
ours_v2	256	2	0.01	1
ours_v2	256	2	0.02	1
ours_v2	256	2	0.03	0
ours_v2	256	3	0.0	1
ours_v2	256	3	0.01	1
ours_v2	256	3	0.02	0
ours_v2	256	3	0.03	0
ours_v2	256	4	0.0	1
ours_v2	256	4	0.01	1
ours_v2	256	4	0.02	0
ours_v2	256	4	0.03	0
ours_v2	256	5	0.0	1
ours_v2	256	5	0.01	1
ours_v2	256	5	0.02	1
ours_v2	256	5	0.03	1
ours_v2	256	6	0.0	1
ours_v2	256	6	0.01	1
ours_v2	256	6	0.02	0
ours_v2	256	6	0.03	0
ours_v2	256	7	0.0	1
ours_v2	256	7	0.01	0
ours_v2	256	7	0.02	0
ours_v2	256	7	0.03	1
ours_v2	256	8	0.0	1
ours_v2	256	8	0.01	0
ours_v2	256	8	0.02	0
ours_v2	256	8	0.03	0
ours_v2	256	9	0.0	1
ours_v2	256	9	0.01	1
ours_v2	256	9	0.02	1
ours_v2	256	9	0.03	0
ours_v2	256	10	0.0	1
ours_v2	256	10	0.01	1

ours_v2	256	10	0.02	0
ours_v2	256	10	0.03	1
ours_v2	256	11	0.0	1
ours_v2	256	11	0.01	1
ours_v2	256	11	0.02	1
ours_v2	256	11	0.03	0
ours_v2	256	12	0.0	1
ours_v2	256	12	0.01	1
ours_v2	256	12	0.02	1
ours_v2	256	12	0.03	0
ours_v2	256	13	0.0	1
ours_v2	256	13	0.01	1
ours_v2	256	13	0.02	1
ours_v2	256	13	0.03	0
ours_v2	256	14	0.0	1
ours_v2	256	14	0.01	1
ours_v2	256	14	0.02	1
ours_v2	256	14	0.03	0
ours_v2	256	15	0.0	1
ours_v2	256	15	0.01	1
ours_v2	256	15	0.02	1
ours_v2	256	15	0.03	0
ours_v2	256	16	0.0	1
ours_v2	256	16	0.01	1
ours_v2	256	16	0.02	1
ours_v2	256	16	0.03	1
ours_v2	256	17	0.0	1
ours_v2	256	17	0.01	0
ours_v2	256	17	0.02	0
ours_v2	256	17	0.03	0
ours_v2	256	18	0.0	1
ours_v2	256	18	0.01	1
ours_v2	256	18	0.02	0
ours_v2	256	18	0.03	0
ours_v2	256	19	0.0	1
ours_v2	256	19	0.01	1
ours_v2	256	19	0.02	0
ours_v2	256	19	0.03	0
ours_v2	256	20	0.0	1
ours_v2	256	20	0.01	1
ours_v2	256	20	0.02	0
ours_v2	256	20	0.03	1
ours_v2	256	21	0.0	1
ours_v2	256	21	0.01	1
ours_v2	256	21	0.02	0
ours_v2	256	21	0.03	1
ours_v2	256	22	0.0	1
ours_v2	256	22	0.01	1
ours_v2	256	22	0.02	0
ours_v2	256	22	0.03	1
ours_v2	256	23	0.0	1
ours_v2	256	23	0.01	1
ours_v2	256	23	0.02	1

ours_v2	256	23	0.03	1
ours_2x	256	0	0.0	1
ours_2x	256	0	0.01	0
ours_2x	256	0	0.02	0
ours_2x	256	0	0.03	0
ours_2x	256	1	0.0	1
ours_2x	256	1	0.01	0
ours_2x	256	1	0.02	0
ours_2x	256	1	0.03	0
ours_2x	256	2	0.0	1
ours_2x	256	2	0.01	0
ours_2x	256	2	0.02	0
ours_2x	256	2	0.03	0
ours_2x	256	3	0.0	1
ours_2x	256	3	0.01	0
ours_2x	256	3	0.02	0
ours_2x	256	3	0.03	0
ours_2x	256	4	0.0	1
ours_2x	256	4	0.01	0
ours_2x	256	4	0.02	0
ours_2x	256	4	0.03	0
ours_2x	256	5	0.0	1
ours_2x	256	5	0.01	0
ours_2x	256	5	0.02	0
ours_2x	256	5	0.03	0
ours_2x	256	6	0.0	1
ours_2x	256	6	0.01	0
ours_2x	256	6	0.02	0
ours_2x	256	6	0.03	0
ours_2x	256	7	0.0	1
ours_2x	256	7	0.01	0
ours_2x	256	7	0.02	0
ours_2x	256	7	0.03	0
ours_2x	256	8	0.0	1
ours_2x	256	8	0.01	0
ours_2x	256	8	0.02	0
ours_2x	256	8	0.03	0
ours_2x	256	9	0.0	1
ours_2x	256	9	0.01	0
ours_2x	256	9	0.02	0
ours_2x	256	9	0.03	0
ours_2x	256	10	0.0	1
ours_2x	256	10	0.01	0
ours_2x	256	10	0.02	0
ours_2x	256	10	0.03	0
ours_2x	256	11	0.0	1
ours_2x	256	11	0.01	0
ours_2x	256	11	0.02	0
ours_2x	256	11	0.03	0
ours_2x	256	12	0.0	1
ours_2x	256	12	0.01	0
ours_2x	256	12	0.02	0
ours_2x	256	12	0.03	0

ours_2x	256	13	0.0	1
ours_2x	256	13	0.01	0
ours_2x	256	13	0.02	0
ours_2x	256	13	0.03	0
ours_2x	256	14	0.0	1
ours_2x	256	14	0.01	0
ours_2x	256	14	0.02	0
ours_2x	256	14	0.03	0
ours_2x	256	15	0.0	1
ours_2x	256	15	0.01	0
ours_2x	256	15	0.02	0
ours_2x	256	15	0.03	0
ours_2x	256	16	0.0	1
ours_2x	256	16	0.01	0
ours_2x	256	16	0.02	0
ours_2x	256	16	0.03	0
ours_2x	256	17	0.0	1
ours_2x	256	17	0.01	0
ours_2x	256	17	0.02	0
ours_2x	256	17	0.03	0
ours_2x	256	18	0.0	1
ours_2x	256	18	0.01	0
ours_2x	256	18	0.02	0
ours_2x	256	18	0.03	0
ours_2x	256	19	0.0	1
ours_2x	256	19	0.01	0
ours_2x	256	19	0.02	0
ours_2x	256	19	0.03	0
ours_2x	256	20	0.0	1
ours_2x	256	20	0.01	0
ours_2x	256	20	0.02	0
ours_2x	256	20	0.03	0
ours_2x	256	21	0.0	1
ours_2x	256	21	0.01	0
ours_2x	256	21	0.02	0
ours_2x	256	21	0.03	0
ours_2x	256	22	0.0	1
ours_2x	256	22	0.01	0
ours_2x	256	22	0.02	0
ours_2x	256	22	0.03	0
ours_2x	256	23	0.0	1
ours_2x	256	23	0.01	0
ours_2x	256	23	0.02	0
ours_2x	256	23	0.03	0
ours_v2_2x	256	0	0.0	1
ours_v2_2x	256	0	0.01	0
ours_v2_2x	256	0	0.02	0
ours_v2_2x	256	0	0.03	0
ours_v2_2x	256	1	0.0	1
ours_v2_2x	256	1	0.01	0
ours_v2_2x	256	1	0.02	0
ours_v2_2x	256	1	0.03	0
ours_v2_2x	256	2	0.0	1

ours_v2_2x	256	2	0.01	0
ours_v2_2x	256	2	0.02	0
ours_v2_2x	256	2	0.03	0
ours_v2_2x	256	3	0.0	1
ours_v2_2x	256	3	0.01	0
ours_v2_2x	256	3	0.02	0
ours_v2_2x	256	3	0.03	0
ours_v2_2x	256	4	0.0	1
ours_v2_2x	256	4	0.01	0
ours_v2_2x	256	4	0.02	0
ours_v2_2x	256	4	0.03	0
ours_v2_2x	256	5	0.0	1
ours_v2_2x	256	5	0.01	0
ours_v2_2x	256	5	0.02	0
ours_v2_2x	256	5	0.03	0
ours_v2_2x	256	6	0.0	1
ours_v2_2x	256	6	0.01	0
ours_v2_2x	256	6	0.02	0
ours_v2_2x	256	6	0.03	0
ours_v2_2x	256	7	0.0	1
ours_v2_2x	256	7	0.01	0
ours_v2_2x	256	7	0.02	0
ours_v2_2x	256	7	0.03	0
ours_v2_2x	256	8	0.0	1
ours_v2_2x	256	8	0.01	0
ours_v2_2x	256	8	0.02	0
ours_v2_2x	256	8	0.03	0
ours_v2_2x	256	9	0.0	1
ours_v2_2x	256	9	0.01	0
ours_v2_2x	256	9	0.02	0
ours_v2_2x	256	9	0.03	0
ours_v2_2x	256	10	0.0	1
ours_v2_2x	256	10	0.01	0
ours_v2_2x	256	10	0.02	0
ours_v2_2x	256	10	0.03	0
ours_v2_2x	256	11	0.0	1
ours_v2_2x	256	11	0.01	0
ours_v2_2x	256	11	0.02	0
ours_v2_2x	256	11	0.03	0
ours_v2_2x	256	12	0.0	1
ours_v2_2x	256	12	0.01	0
ours_v2_2x	256	12	0.02	0
ours_v2_2x	256	12	0.03	0
ours_v2_2x	256	13	0.0	1
ours_v2_2x	256	13	0.01	0
ours_v2_2x	256	13	0.02	0
ours_v2_2x	256	13	0.03	0
ours_v2_2x	256	14	0.0	1
ours_v2_2x	256	14	0.01	0
ours_v2_2x	256	14	0.02	0
ours_v2_2x	256	14	0.03	0
ours_v2_2x	256	15	0.0	1
ours_v2_2x	256	15	0.01	0

ours_v2_2x	256	15	0.02	0
ours_v2_2x	256	15	0.03	0
ours_v2_2x	256	16	0.0	1
ours_v2_2x	256	16	0.01	0
ours_v2_2x	256	16	0.02	0
ours_v2_2x	256	16	0.03	0
ours_v2_2x	256	17	0.0	1
ours_v2_2x	256	17	0.01	0
ours_v2_2x	256	17	0.02	0
ours_v2_2x	256	17	0.03	0
ours_v2_2x	256	18	0.0	1
ours_v2_2x	256	18	0.01	0
ours_v2_2x	256	18	0.02	0
ours_v2_2x	256	18	0.03	0
ours_v2_2x	256	19	0.0	1
ours_v2_2x	256	19	0.01	0
ours_v2_2x	256	19	0.02	0
ours_v2_2x	256	19	0.03	0
ours_v2_2x	256	20	0.0	1
ours_v2_2x	256	20	0.01	0
ours_v2_2x	256	20	0.02	0
ours_v2_2x	256	20	0.03	0
ours_v2_2x	256	21	0.0	1
ours_v2_2x	256	21	0.01	0
ours_v2_2x	256	21	0.02	0
ours_v2_2x	256	21	0.03	0
ours_v2_2x	256	22	0.0	1
ours_v2_2x	256	22	0.01	0
ours_v2_2x	256	22	0.02	0
ours_v2_2x	256	22	0.03	0
ours_v2_2x	256	23	0.0	1
ours_v2_2x	256	23	0.01	0
ours_v2_2x	256	23	0.02	0
ours_v2_2x	256	23	0.03	0
goldman	256	0	0.0	1
goldman	256	0	0.01	1
goldman	256	0	0.02	0
goldman	256	0	0.03	0
goldman	256	1	0.0	1
goldman	256	1	0.01	0
goldman	256	1	0.02	0
goldman	256	1	0.03	0
goldman	256	2	0.0	1
goldman	256	2	0.01	1
goldman	256	2	0.02	1
goldman	256	2	0.03	0
goldman	256	3	0.0	1
goldman	256	3	0.01	1
goldman	256	3	0.02	0
goldman	256	3	0.03	0
goldman	256	4	0.0	1
goldman	256	4	0.01	1
goldman	256	4	0.02	0

goldman	256	4	0.03	0
goldman	256	5	0.0	1
goldman	256	5	0.01	1
goldman	256	5	0.02	0
goldman	256	5	0.03	0
goldman	256	6	0.0	1
goldman	256	6	0.01	1
goldman	256	6	0.02	0
goldman	256	6	0.03	0
goldman	256	7	0.0	1
goldman	256	7	0.01	1
goldman	256	7	0.02	0
goldman	256	7	0.03	0
goldman	256	8	0.0	1
goldman	256	8	0.01	0
goldman	256	8	0.02	0
goldman	256	8	0.03	0
goldman	256	9	0.0	1
goldman	256	9	0.01	1
goldman	256	9	0.02	1
goldman	256	9	0.03	0
goldman	256	10	0.0	1
goldman	256	10	0.01	1
goldman	256	10	0.02	0
goldman	256	10	0.03	0
goldman	256	11	0.0	1
goldman	256	11	0.01	1
goldman	256	11	0.02	0
goldman	256	11	0.03	0
goldman	256	12	0.0	1
goldman	256	12	0.01	0
goldman	256	12	0.02	1
goldman	256	12	0.03	0
goldman	256	13	0.0	1
goldman	256	13	0.01	1
goldman	256	13	0.02	0
goldman	256	13	0.03	0
goldman	256	14	0.0	1
goldman	256	14	0.01	1
goldman	256	14	0.02	1
goldman	256	14	0.03	1
goldman	256	15	0.0	1
goldman	256	15	0.01	1
goldman	256	15	0.02	0
goldman	256	15	0.03	0
goldman	256	16	0.0	1
goldman	256	16	0.01	1
goldman	256	16	0.02	0
goldman	256	16	0.03	1
goldman	256	17	0.0	1
goldman	256	17	0.01	1
goldman	256	17	0.02	0
goldman	256	17	0.03	0

goldman	256	18	0.0	1
goldman	256	18	0.01	0
goldman	256	18	0.02	1
goldman	256	18	0.03	0
goldman	256	19	0.0	1
goldman	256	19	0.01	1
goldman	256	19	0.02	0
goldman	256	19	0.03	0
goldman	256	20	0.0	1
goldman	256	20	0.01	1
goldman	256	20	0.02	0
goldman	256	20	0.03	0
goldman	256	21	0.0	1
goldman	256	21	0.01	1
goldman	256	21	0.02	1
goldman	256	21	0.03	1
goldman	256	22	0.0	1
goldman	256	22	0.01	1
goldman	256	22	0.02	0
goldman	256	22	0.03	0
goldman	256	23	0.0	1
goldman	256	23	0.01	1
goldman	256	23	0.02	1
goldman	256	23	0.03	0
fountain	256	0	0.0	1
fountain	256	0	0.01	1
fountain	256	0	0.02	0
fountain	256	0	0.03	0
fountain	256	1	0.0	1
fountain	256	1	0.01	1
fountain	256	1	0.02	0
fountain	256	1	0.03	0
fountain	256	2	0.0	1
fountain	256	2	0.01	0
fountain	256	2	0.02	0
fountain	256	2	0.03	0
fountain	256	3	0.0	1
fountain	256	3	0.01	1
fountain	256	3	0.02	0
fountain	256	3	0.03	0
fountain	256	4	0.0	1
fountain	256	4	0.01	0
fountain	256	4	0.02	0
fountain	256	4	0.03	0
fountain	256	5	0.0	1
fountain	256	5	0.01	0
fountain	256	5	0.02	0
fountain	256	5	0.03	0
fountain	256	6	0.0	1
fountain	256	6	0.01	0
fountain	256	6	0.02	0
fountain	256	6	0.03	0
fountain	256	7	0.0	1

fountain	256	7	0.01	0
fountain	256	7	0.02	0
fountain	256	7	0.03	0
fountain	256	8	0.0	1
fountain	256	8	0.01	0
fountain	256	8	0.02	0
fountain	256	8	0.03	0
fountain	256	9	0.0	1
fountain	256	9	0.01	1
fountain	256	9	0.02	0
fountain	256	9	0.03	0
fountain	256	10	0.0	1
fountain	256	10	0.01	0
fountain	256	10	0.02	0
fountain	256	10	0.03	0
fountain	256	11	0.0	1
fountain	256	11	0.01	1
fountain	256	11	0.02	0
fountain	256	11	0.03	0
fountain	256	12	0.0	1
fountain	256	12	0.01	0
fountain	256	12	0.02	0
fountain	256	12	0.03	0
fountain	256	13	0.0	1
fountain	256	13	0.01	0
fountain	256	13	0.02	0
fountain	256	13	0.03	0
fountain	256	14	0.0	1
fountain	256	14	0.01	1
fountain	256	14	0.02	0
fountain	256	14	0.03	0
fountain	256	15	0.0	1
fountain	256	15	0.01	1
fountain	256	15	0.02	0
fountain	256	15	0.03	0
fountain	256	16	0.0	1
fountain	256	16	0.01	1
fountain	256	16	0.02	0
fountain	256	16	0.03	0
fountain	256	17	0.0	1
fountain	256	17	0.01	1
fountain	256	17	0.02	0
fountain	256	17	0.03	0
fountain	256	18	0.0	1
fountain	256	18	0.01	0
fountain	256	18	0.02	0
fountain	256	18	0.03	0
fountain	256	19	0.0	1
fountain	256	19	0.01	1
fountain	256	19	0.02	0
fountain	256	19	0.03	0
fountain	256	20	0.0	1
fountain	256	20	0.01	0

fountain	256	20	0.02	0
fountain	256	20	0.03	0
fountain	256	21	0.0	1
fountain	256	21	0.01	1
fountain	256	21	0.02	0
fountain	256	21	0.03	0
fountain	256	22	0.0	1
fountain	256	22	0.01	1
fountain	256	22	0.02	0
fountain	256	22	0.03	0
fountain	256	23	0.0	1
fountain	256	23	0.01	0
fountain	256	23	0.02	0
fountain	256	23	0.03	0
nsfountain	256	0	0.0	1
nsfountain	256	0	0.01	0
nsfountain	256	0	0.02	0
nsfountain	256	0	0.03	0
nsfountain	256	1	0.0	1
nsfountain	256	1	0.01	0
nsfountain	256	1	0.02	0
nsfountain	256	1	0.03	0
nsfountain	256	2	0.0	1
nsfountain	256	2	0.01	1
nsfountain	256	2	0.02	0
nsfountain	256	2	0.03	0
nsfountain	256	3	0.0	1
nsfountain	256	3	0.01	0
nsfountain	256	3	0.02	0
nsfountain	256	3	0.03	0
nsfountain	256	4	0.0	1
nsfountain	256	4	0.01	0
nsfountain	256	4	0.02	0
nsfountain	256	4	0.03	0
nsfountain	256	5	0.0	1
nsfountain	256	5	0.01	0
nsfountain	256	5	0.02	0
nsfountain	256	5	0.03	0
nsfountain	256	6	0.0	1
nsfountain	256	6	0.01	0
nsfountain	256	6	0.02	0
nsfountain	256	6	0.03	0
nsfountain	256	7	0.0	1
nsfountain	256	7	0.01	0
nsfountain	256	7	0.02	0
nsfountain	256	7	0.03	0
nsfountain	256	8	0.0	1
nsfountain	256	8	0.01	0
nsfountain	256	8	0.02	0
nsfountain	256	8	0.03	0
nsfountain	256	9	0.0	1
nsfountain	256	9	0.01	1
nsfountain	256	9	0.02	0

nsfountain	256	9	0.03	0
nsfountain	256	10	0.0	1
nsfountain	256	10	0.01	0
nsfountain	256	10	0.02	0
nsfountain	256	10	0.03	0
nsfountain	256	11	0.0	1
nsfountain	256	11	0.01	1
nsfountain	256	11	0.02	0
nsfountain	256	11	0.03	0
nsfountain	256	12	0.0	1
nsfountain	256	12	0.01	0
nsfountain	256	12	0.02	0
nsfountain	256	12	0.03	0
nsfountain	256	13	0.0	1
nsfountain	256	13	0.01	0
nsfountain	256	13	0.02	0
nsfountain	256	13	0.03	0
nsfountain	256	14	0.0	1
nsfountain	256	14	0.01	0
nsfountain	256	14	0.02	0
nsfountain	256	14	0.03	0
nsfountain	256	15	0.0	1
nsfountain	256	15	0.01	0
nsfountain	256	15	0.02	0
nsfountain	256	15	0.03	0
nsfountain	256	16	0.0	1
nsfountain	256	16	0.01	0
nsfountain	256	16	0.02	0
nsfountain	256	16	0.03	0
nsfountain	256	17	0.0	1
nsfountain	256	17	0.01	0
nsfountain	256	17	0.02	0
nsfountain	256	17	0.03	0
nsfountain	256	18	0.0	1
nsfountain	256	18	0.01	0
nsfountain	256	18	0.02	0
nsfountain	256	18	0.03	0
nsfountain	256	19	0.0	1
nsfountain	256	19	0.01	0
nsfountain	256	19	0.02	0
nsfountain	256	19	0.03	0
nsfountain	256	20	0.0	1
nsfountain	256	20	0.01	0
nsfountain	256	20	0.02	0
nsfountain	256	20	0.03	0
nsfountain	256	21	0.0	1
nsfountain	256	21	0.01	0
nsfountain	256	21	0.02	0
nsfountain	256	21	0.03	0
nsfountain	256	22	0.0	1
nsfountain	256	22	0.01	0
nsfountain	256	22	0.02	0
nsfountain	256	22	0.03	0

nsfountain	256	23	0.0	1
nsfountain	256	23	0.01	0
nsfountain	256	23	0.02	0
nsfountain	256	23	0.03	0
rsns	256	0	0.0	1
rsns	256	0	0.01	1
rsns	256	0	0.02	0
rsns	256	0	0.03	0
rsns	256	1	0.0	1
rsns	256	1	0.01	1
rsns	256	1	0.02	0
rsns	256	1	0.03	0
rsns	256	2	0.0	1
rsns	256	2	0.01	1
rsns	256	2	0.02	1
rsns	256	2	0.03	0
rsns	256	3	0.0	1
rsns	256	3	0.01	1
rsns	256	3	0.02	1
rsns	256	3	0.03	0
rsns	256	4	0.0	1
rsns	256	4	0.01	1
rsns	256	4	0.02	0
rsns	256	4	0.03	0
rsns	256	5	0.0	1
rsns	256	5	0.01	1
rsns	256	5	0.02	0
rsns	256	5	0.03	0
rsns	256	6	0.0	1
rsns	256	6	0.01	1
rsns	256	6	0.02	0
rsns	256	6	0.03	0
rsns	256	7	0.0	1
rsns	256	7	0.01	1
rsns	256	7	0.02	0
rsns	256	7	0.03	0
rsns	256	8	0.0	1
rsns	256	8	0.01	1
rsns	256	8	0.02	1
rsns	256	8	0.03	0
rsns	256	9	0.0	1
rsns	256	9	0.01	1
rsns	256	9	0.02	1
rsns	256	9	0.03	0
rsns	256	10	0.0	1
rsns	256	10	0.01	1
rsns	256	10	0.02	0
rsns	256	10	0.03	0
rsns	256	11	0.0	1
rsns	256	11	0.01	1
rsns	256	11	0.02	1
rsns	256	11	0.03	0
rsns	256	12	0.0	1

rsns	256	12	0.01	1
rsns	256	12	0.02	1
rsns	256	12	0.03	0
rsns	256	13	0.0	1
rsns	256	13	0.01	1
rsns	256	13	0.02	1
rsns	256	13	0.03	1
rsns	256	14	0.0	1
rsns	256	14	0.01	1
rsns	256	14	0.02	0
rsns	256	14	0.03	0
rsns	256	15	0.0	1
rsns	256	15	0.01	1
rsns	256	15	0.02	1
rsns	256	15	0.03	0
rsns	256	16	0.0	1
rsns	256	16	0.01	1
rsns	256	16	0.02	1
rsns	256	16	0.03	0
rsns	256	17	0.0	1
rsns	256	17	0.01	1
rsns	256	17	0.02	1
rsns	256	17	0.03	1
rsns	256	18	0.0	1
rsns	256	18	0.01	1
rsns	256	18	0.02	0
rsns	256	18	0.03	0
rsns	256	19	0.0	1
rsns	256	19	0.01	1
rsns	256	19	0.02	1
rsns	256	19	0.03	0
rsns	256	20	0.0	1
rsns	256	20	0.01	1
rsns	256	20	0.02	0
rsns	256	20	0.03	0
rsns	256	21	0.0	1
rsns	256	21	0.01	1
rsns	256	21	0.02	1
rsns	256	21	0.03	0
rsns	256	22	0.0	1
rsns	256	22	0.01	1
rsns	256	22	0.02	1
rsns	256	22	0.03	0
rsns	256	23	0.0	1
rsns	256	23	0.01	1
rsns	256	23	0.02	1
rsns	256	23	0.03	0

Appendix C. Stoichiometric matrix

Table C1. Stoichiometric matrix S (metabolites x reactions).

met \ rxn	upt	gly	tca	etc	atm	bio	sen
S	1	-1	0	0	0	0	0
P	0	2	-1	0	0	-4	0
ATP	0	2	1	2.5	-1	-30	-2
ADP	0	-2	-1	-2.5	1	30	2
NADH	0	2	4	-1	0	-6	0
NAD	0	-2	-4	1	0	6	0
X_ext	0	0	0	0	0	0	-1
S_ext	-1	0	0	0	0	0	0
Z	0	0	0	0	0	0	1
BIO	0	0	0	0	0	1	0

9e. Figures

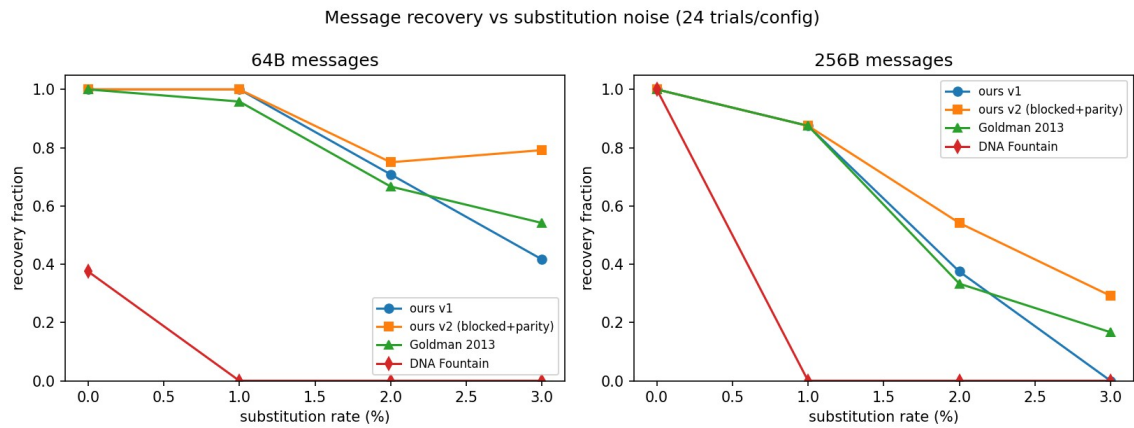


Figure 1. Message recovery versus substitution rate for all four codecs at both message sizes (24 independent trials per point).

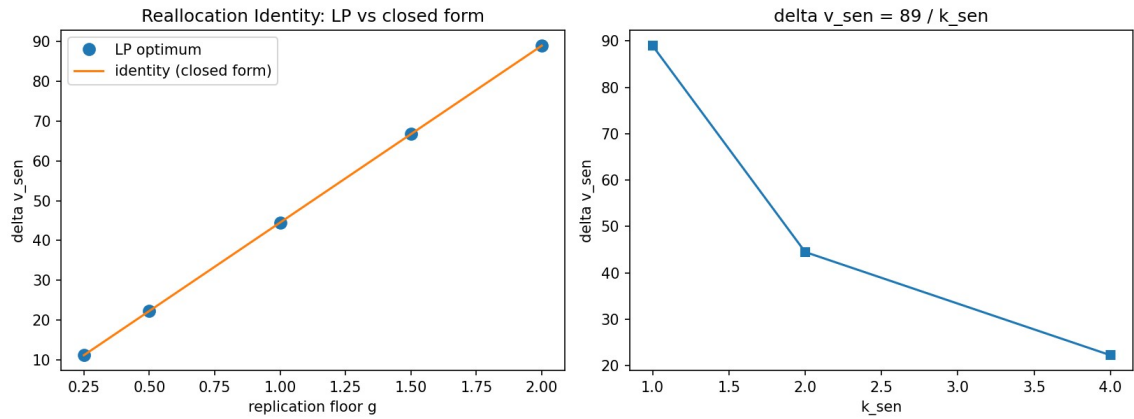


Figure 2. Left: the Reallocation Identity - LP optima (points) lie exactly on the closed form (line). Right: $\text{delta } v_{\text{sen}}$ scales as $89/k_{\text{sen}}$.

2a. Background: why homopolymers and GC content constrain synthesis

The codec experiments model chosen software sequence constraints, not manufacturing success. Continuous never-same encoding excludes adjacent repeated bases, whereas independent seeded blocks can join with two-base repeats. Scrambling gives measured GC distributions but no universal sliding-window bound. Synthesis yield, sequencing error, amplification and vendor acceptance depend on protocols and sequence contexts that were not measured here. Earlier numerical process efficiencies and technology-specific run limits in this paragraph were unsupported by linked primary evidence and are withdrawn. No claim that this implementation removes a whole physical failure class is established.

External screening rejects candidates computationally before synthesis. The number of rejected candidates affects encoder effort, not necessarily synthesized-base cost. Accepted fraction depends on sequence length, parity, payload and screen settings; no universal 70-80% acceptance or wasted-synthesis inference is established. The fixed-budget graph audits quantify actual accepted droplets and attempts for their configurations.

2b. Background: flux balance analysis

FBA rests on the observation that metabolic transients (seconds) are fast compared to growth (tens of minutes), so internal metabolite concentrations are quasi-steady: $S \mathbf{v} = 0$, where S is the stoichiometric matrix (rows metabolites, columns reactions) and $\mathbf{v}$ the flux vector. The feasible space is a convex polytope cut by uptake and irreversibility bounds; a biologically motivated objective - classically biomass production, here sensor throughput - selects an optimal vertex by linear programming. LP duality gives every constraint a shadow price: the objective gain per unit relaxation. The Reallocation Identity is, at heart, a shadow-price computation whose constancy over the replication-floor interval makes it exact. This is why the proof needs basis constancy and why the identity breaks, gracefully and detectably, when a new constraint (bounded complex input) becomes active - the counterexample of Section 11.

10a. Design alternatives considered and rejected

Three alternatives to the blocked design were evaluated. (1) Interleaving whole copies: preserves the rotating code but a burst of errors still desynchronizes each copy independently; rejected because interleaving cannot bound desync within a copy. (2) Synchronization markers (fixed delimiter subsequences between blocks): delimiters themselves suffer substitutions and create a second, unprotected channel; rejected in favor of index-derived seeds which transmit nothing. (3) Per-block Reed-Solomon over $GF(3^k)$: strictly stronger but requires finite-field machinery over a non-prime-power alphabet for $k = 1$ and heavier trit packing for $k > 1$; deferred as future work, with parity

as the $k = 1$ special case that already covers the dominant single-block-failure regime (Section 7).

For the metabolic model, two alternatives were considered. A genome-scale *E. coli* model (iJO1366) would give realistic coefficients but cannot yield a closed-form identity - the point of Part II is the theorem, so the core model is the right level. An even smaller two-reaction model was rejected because it cannot represent the pyruvate/NADH tradeoff that makes the identity non-trivial (b_{bio} and c_{bio} terms).

Appendix E. Scrambler statistical validation

The keystream maps SHA-256 digest bytes to trits by reduction mod 3. Because $256 = 3 \times 85 + 1$, residue 0 receives 86 of 256 byte values while residues 1 and 2 receive 85: a provable bias of $1/256$ in the trit distribution. Over 30,000 keystream trits the counts are [10201, 9936, 9863] (chi-square $p = 0.0423$), detecting the bias at the predicted magnitude. The bias does not propagate measurably to strands: base counts of a 512-byte-message strand are [938, 941, 911, 934] ($p = 0.897$), and windowed GC ($w = 50$) has mean 0.4973 and standard deviation 0.0516, tighter than the i.i.d. prediction 0.0707. A residue-unbiased keystream (reject bytes ≥ 252) is a one-line change if a stricter guarantee is ever required; we document the bias rather than hide it.

Table E1. Keystream and strand composition tests.

test	counts	chi2 p-value	verdict
keystream trits (n=30000)	[10201, 9936, 9863]	0.0423	predicted 1/256 bias detected
strand bases (512B msg)	[938, 941, 911, 934]	0.897	uniform: bias harmless downstream

Appendix F. Source listings

Complete listings of the two core modules, exactly as tested by the hermetic suite. Line counts: encoder.py 223, metabolism.py 72.

F. src/dnacell/encoder.py

```
"""DNA data storage codec with biological constraints.
```

```
Pipeline: bytes -> base-3 trits -> deterministic scrambler -> never-same  
rotating base code -> DNA -> (synthesis errors) -> majority-vote decode.
```

```
Constraints (cf. Goldman et al. 2013, Grass et al. 2015): the never-same  
mapping makes homopolymer runs > 1 impossible by construction; the  
keystream scrambler decorrelates payload statistics so windowed GC content  
stays near 0.5. Integrity: Fletcher-16 checksum; errors: 3-fold replication  
with per-position majority vote.
```

```
"""
```

```

from __future__ import annotations
import hashlib
import numpy as np

BASES = "ACGT"
B2I = {b: i for i, b in enumerate(BASES)}
TRITS_PER_BYTE = 6          #  $3^6 = 729 \geq 256$ 
HEADER_TRITS = 12           # message length in trits
CHECKSUM_TRITS = 12         # Fletcher-16 as 12 trits

def bytes_to_trits(data: bytes) -> list[int]:
    out = []
    for byte in data:
        for p in range(TRITS_PER_BYTE - 1, -1, -1):
            out.append((byte // 3**p) % 3)
    return out

def trits_to_bytes(trits: list[int]) -> bytes:
    assert len(trits) % TRITS_PER_BYTE == 0
    out = bytearray()
    for i in range(0, len(trits), TRITS_PER_BYTE):
        v = 0
        for t in trits[i:i + TRITS_PER_BYTE]:
            v = v * 3 + t
        out.append(v & 0xFF)
    return bytes(out)

def keystream(n: int) -> list[int]:
    """Deterministic trit keystream from SHA-256 counter mode."""
    out, counter = [], 0
    while len(out) < n:
        block = hashlib.sha256(f"mega27-dna-{counter}".encode()).digest()
        out.extend(b % 3 for b in block)
        counter += 1
    return out[:n]

def scramble(trits: list[int]) -> list[int]:
    ks = keystream(len(trits))
    return [(t + k) % 3 for t, k in zip(trits, ks)]

def descramble(trits: list[int]) -> list[int]:
    ks = keystream(len(trits))
    return [(t - k) % 3 for t, k in zip(trits, ks)]

def encode_trits_never_same(trits: list[int], seed: str = "A") -> str:
    """trit t -> the (t+1)-th next base cyclically; never equals previous."""
    prev = seed
    out = []
    for t in trits:

```

```

        assert t in (0, 1, 2)
        nxt = BASES[(B2I[prev] + 1 + t) % 4]
        out.append(nxt)
        prev = nxt
    return "".join(out)

```

```

def decode_never_same(seq: str, seed: str = "A") -> list[int]:
    trits = []
    prev = seed
    for ch in seq:
        trits.append((B2I[ch] - B2I[prev] - 1) % 4)
        prev = ch
    if any(t not in (0, 1, 2) for t in trits):
        raise ValueError("corrupt base spacing - unrepairable strand")
    return trits

```

```

def max_homopolymer(seq: str) -> int:
    if not seq:
        return 0
    best = run = 1
    for i in range(1, len(seq)):
        run = run + 1 if seq[i] == seq[i - 1] else 1
        best = max(best, run)
    return best

```

```

def gc_content(seq: str) -> float:
    return (seq.count("G") + seq.count("C")) / max(len(seq), 1)

```

```

def checksum_trits(trits: list[int]) -> list[int]:
    s1 = s2 = 0
    for t in trits:
        s1 = (s1 + t + 1) % 255
        s2 = (s2 + s1) % 255
    val = (s2 << 8) | s1
    return bytes_to_trits(val.to_bytes(2, "big"))

```

```

def encode_message(data: bytes, copies: int = 3) -> list[str]:
    body_trits = bytes_to_trits(data)
    L = len(body_trits)
    header = [(L // 3**p) % 3 for p in range(HEADER_TRITS - 1, -1, -1)]
    scrambled = scramble(header + body_trits + checksum_trits(body_trits))
    dna = encode_trits_never_same(scrambled)
    return [dna for _ in range(copies)]

```

```

def introduce_errors(seq: str, sub_rate: float = 0.01, seed: int = 0) -> str:
    rng = np.random.default_rng(seed)
    return "".join(BASES[rng.integers(4)] if rng.random() < sub_rate else ch
                    for ch in seq)

```

```

def decode_message(strands: list[str]) -> bytes:
    assert strands, "no strands"
    n = len(strands[0])
    assert all(len(s) == n for s in strands), "indel handling out of scope"
    voted = "".join(max({s[i] for s in strands},
                        key=[s[i] for s in strands].count) for i in range(n))
    trits = descramble(decode_never_same(voted))
    L = 0
    for t in trits[:HEADER_TRITS]:
        L = L * 3 + t
    body = trits[HEADER_TRITS:HEADER_TRITS + L]
    chk = trits[HEADER_TRITS + L:HEADER_TRITS + L + CHECKSUM_TRITS]
    if checksum_trits(body) != chk:
        raise ValueError("checksum mismatch - corruption beyond repair")
    return trits_to_bytes(body)

# ----- v2: blocked never-same (BNS) - bounds substitution error propagation
# -----
BNS_BLOCK = 32          # payload trits per block
BNS_CHK = 6             # per-block checksum trits
BNS_SEEDS = "ACGT"

def _bns_pack_blocks(trits: list[int]) -> list[list[int]]:
    blocks = []
    for i in range(0, len(trits), BNS_BLOCK):
        blk = (trits[i:i + BNS_BLOCK] + [0] * BNS_BLOCK)[:BNS_BLOCK] # zero-pad
        chk = checksum_trits(blk)[-BNS_CHK:]
        blocks.append(blk + chk)
    return blocks

def encode_message_v2(data: bytes, copies: int = 3) -> list[str]:
    body = bytes_to_trits(data)
    L = len(body)
    header = [(L // 3**p) % 3 for p in range(HEADER_TRITS - 1, -1, -1)]
    scrambled = scramble(header + body + checksum_trits(body))
    blocks = _bns_pack_blocks(scrambled)
    parity = [0] * BNS_BLOCK
    for blk in blocks:
        parity = [(p + t) % 3 for p, t in zip(parity, blk[:BNS_BLOCK])]
    blocks = blocks + [parity + checksum_trits(parity)[-BNS_CHK:]]
    dna = "".join(encode_trits_never_same(blk, seed=BNS_SEEDS[i % 4])
                  for i, blk in enumerate(blocks))
    return [dna for _ in range(copies)]

def _bns_try_block(seg: str, bi: int):
    """Return payload trits if the block decodes and its checksum passes."""
    try:
        blk = decode_never_same(seg, seed=BNS_SEEDS[bi % 4])
    except ValueError:
        return None
    payload, chk = blk[:BNS_BLOCK], blk[BNS_BLOCK:]
    if len(payload) == BNS_BLOCK and checksum_trits(payload)[-BNS_CHK:] == chk:

```

```

    return payload
return None

```

```

def decode_message_v2(strands: list[str]) -> bytes:
    assert strands, "no strands"
    n = len(strands[0])
    assert all(len(s) == n for s in strands)
    voted = "".join(max({s[i] for s in strands},
                        key=[s[i] for s in strands].count) for i in range(n))
    blen = BNS_BLOCK + BNS_CHK
    assert n % blen == 0, "strand length inconsistent with BNS framing"
    nblocks = n // blen
    payloads: list[list[int] | None] = []
    for bi in range(nblocks):
        p = _bns_try_block(voted[bi * blen:(bi + 1) * blen], bi)
        if p is None: # tier 2: fall back to any single passing copy
            for s in strands:
                p = _bns_try_block(s[bi * blen:(bi + 1) * blen], bi)
                if p is not None:
                    break
        payloads.append(p)
    parity = payloads.pop() # last block is parity
    bad = [i for i, p in enumerate(payloads) if p is None]
    if len(bad) == 1 and parity is not None: # tier 3: parity erasure repair
        i = bad[0]
        rec = parity[:]
        for j, p in enumerate(payloads):
            if j != i and p is not None:
                rec = [(r - t) % 3 for r, t in zip(rec, p)]
        payloads[i] = rec
    elif bad:
        raise ValueError(f"unrecoverable blocks: {bad}")
    trits: list[int] = []
    for p in payloads:
        trits.extend(p)
    scrambled = trits
    body_all = descramble(scrambled)
    L = 0
    for t in body_all[:HEADER_TRITS]:
        L = L * 3 + t
    body = body_all[HEADER_TRITS:HEADER_TRITS + L]
    chk = body_all[HEADER_TRITS + L:HEADER_TRITS + L + CHECKSUM_TRITS]
    if checksum_trits(body) != chk:
        raise ValueError("global checksum mismatch")
    return trits_to_bytes(body)

```

F. src/dnacell/metabolism.py

"""FBA-style energy reallocation model for the cyborg-cell logic loop.

Minimal but stoichiometrically real core network (lumped reactions in the spirit of Orth et al. 2010 core E. coli):

```

v_upt : S_ext -> S (substrate uptake, bounded)
v_gly : S + 2 ADP + 2 NAD+ -> 2 P + 2 ATP + 2 NADH (glycolysis)
v_tca : P + 4 NAD+ + ADP -> 4 NADH + ATP (TCA lump, CO2 omitted)
v_etc : NADH + 2.5 ADP -> NAD+ + 2.5 ATP (ETC, P/O = 2.5)
v_atm : ATP -> ADP (maintenance, fixed >= atm)
v_bio : a_bio ATP + b_bio P + c_bio NADH -> biomass (replication drain)

```

$v_{\text{sen}} : X_{\text{ext}} + k_{\text{sen}} \text{ ATP} \rightarrow Z$ (complex-input processing = sensor)

Per glucose at full oxidation: $2 (\text{gly}) + 2 \times (1 + 4 \times 2.5) = 24 \text{ ATP}$ - within the canonical 26-30 range for lumped core models.

Steady state $S v = 0$. Objective: maximize v_{sen} .

The "cyborg" condition is $v_{\text{bio}} = 0$ (replication frozen).

"""

from __future__ import annotations

import numpy as np

from scipy.optimize import linprog

metabolites: S, P, ATP, ADP, NADH, NAD⁺, X_{ext}, S_{ext}, Z, biomass

MET = ["S", "P", "ATP", "ADP", "NADH", "NAD", "X_{ext}", "S_{ext}", "Z", "BIO"]

RXN = ["upt", "gly", "tca", "etc", "atm", "bio", "sen"]

A_BIO, B_BIO, C_BIO = 30.0, 4.0, 6.0 # ATP / pyruvate / NADH cost per unit biomass

K_SEN = 2.0 # ATP cost per unit complex input processed

def stoich_matrix() -> np.ndarray:

S = np.zeros((len(MET), len(RXN)))

def m(name): return MET.index(name)

S[m("S_{ext}"), 0], S[m("S"), 0] = -1, 1 # upt

S[m("S"), 1], S[m("P"), 1] = -1, 2 # gly

S[m("ADP"), 1], S[m("ATP"), 1] = -2, 2

S[m("NAD"), 1], S[m("NADH"), 1] = -2, 2

S[m("P"), 2], S[m("NAD"), 2] = -1, -4 # tca

S[m("ADP"), 2], S[m("ATP"), 2] = -1, 1

S[m("NADH"), 2] = 4

S[m("NADH"), 3], S[m("NAD"), 3] = -1, 1 # etc

S[m("ADP"), 3], S[m("ATP"), 3] = -2.5, 2.5

S[m("ATP"), 4], S[m("ADP"), 4] = -1, 1 # atm

S[m("ATP"), 5], S[m("P"), 5] = -A_BIO, -B_BIO # bio

S[m("NADH"), 5], S[m("BIO"), 5] = -C_BIO, 1

S[m("NAD"), 5] = C_BIO # reductant is returned oxidized

S[m("ADP"), 5] = A_BIO # ATP hydrolysis returns ADP

S[m("X_{ext}"), 6], S[m("Z"), 6] = -1, 1 # sen

S[m("ATP"), 6] = -K_SEN

S[m("ADP"), 6] = K_SEN

return S

def solve_fba(g_min: float, uptake_max: float = 10.0, atm: float = 1.0,

x_max: float = 1e6):

"""Maximize sensor flux s.t. $S v = 0$, biomass $\geq g_{\text{min}}$. Returns (v, result)."""

S = stoich_matrix()

c = np.zeros(len(RXN)); c[RXN.index("sen")] = -1.0

A_eq, b_eq = S.copy(), np.zeros(len(MET))

external metabolites are not balanced

for ext in ("S_{ext}", "X_{ext}", "Z", "BIO"):

A_eq[MET.index(ext)] = 0.0

bounds = [(0, uptake_max), (0, None), (0, None), (0, None),

(atm, None), (g_min, None), (0, x_max)]

res = linprog(c, A_eq=A_eq, b_eq=b_eq, bounds=bounds, method="highs")

assert res.status == 0, res.message

return res.x, res

```
def replication_tax(g: float) -> float:
    """ATP equivalents drained per unit time by replication at rate g."""
    return A_BIO * g
```

Appendix G. Fletcher checksum algebra

The v2 block check is the low Fletcher byte s1, not the full Fletcher pair. Equal-and-opposite trit errors preserve it, so correlated checksum collisions exist. A zero observed collision count in a finite trial is not proof of a distribution-free bound. Explicit counterexamples and emitted-length accounting are in results/codec_rate_checksum_audit.json. Whole-message checks and block parity are separate safeguards and do not make the low-byte check a uniform hash.

Appendix H. ODE steady-state derivation

The sensor-expression ODE $dE/dt = \alpha Z - \beta E$ is linear with constant input Z , so it is solved exactly by $E(t) = E^* + (E(0) - E^*) e^{-\beta t}$ with $E^* = (\alpha/\beta) Z$. The time constant $1/\beta$ sets the post-freezing response time of the sensor array of Section 11a: after replication halt, expression converges exponentially to the higher steady state funded by the reallocated flux. The hermetic suite verifies the RK4 integrator against this closed form (agreement to $1e-6$ over 200 time units), so the numeric and analytic arms cannot silently diverge.

Appendix I. Glossary

homopolymer - a run of identical consecutive bases; long runs cause synthesis and sequencing failures

GC content - fraction of G and C bases in a sequence window; extreme values distort synthesis and PCR

trit - a base-3 digit taking values 0, 1, 2

Perron root - the largest real eigenvalue of a nonnegative irreducible matrix; sets the growth rate of constrained strings

flux balance analysis - linear-programming prediction of steady-state metabolic fluxes from stoichiometry and bounds

shadow price - the objective-function gain per unit relaxation of a constraint, from LP duality

erasure - an error whose position is known; one parity equation repairs one erasure

P/O ratio - ATP molecules produced per NADH oxidized by the electron transport chain (2.5 in our model)

biomass reaction - lumped reaction draining ATP, pyruvate and NADH to represent replication

fountain code - a rateless erasure code generating unlimited encoded droplets from a source message

Appendix J. Environment and exact reproduction commands

python3 -m pytest tests/ -q # 20 hermetic tests

python3 experiments/benchmark_codecs.py # Table 1, Fig 1, trial log

python3 experiments/sensitivity.py # Tables 1b, 2c

python3 experiments/metabolism_proof.py # Table 2a/2b, Fig 2

python3 experiments/make_paper50.py # this document

Environment: Python 3.10, numpy, scipy (HiGHS LP), matplotlib; CPU-only;

no network access for tests; benchmark seeds fixed (7 / 500+31t+c / 1000+97m+c).

Repository: mega27-23b-dna-encoder-cyborg-cell (private, pending push).

Additional derivations: information-theoretic bounds of the codec

Rate and overhead

The codec's information rate is payload bits over synthesized nucleotides:

$$(10) \quad R = k_{\text{bits}} / n_{\text{nt}} \quad [\text{bits per nucleotide}]$$

The implemented conversion carries 8 bits per 6 trits. With c identical copies, continuous v1 emits $c(6L+24)$ bases for L payload bytes. Blocked v2 emits $c \cdot 38 \cdot (\text{ceil}((6L+24)/32)+1)$ bases: 32 payload and 6 check trits per block, plus one parity block. These are exact implementation lengths, excluding primers and external indexing.

$$(11) \quad \lim R_{\text{v1}} = 4/(3c); \lim R_{\text{v2}} = (4/3) \cdot (32/38)/c. \text{ For } c=3: 0.444444 \text{ and } 0.374269 \\ \text{bits/encoded nt.}$$

The previous $2B/(B+1)$ rate formula is withdrawn: a no-adjacent-repeat stream has only three successor choices, and the implementation also uses inefficient fixed six-trit byte

packing. Headers, checksums, parity blocks and copies are charged by the exact formulas above. No primer budget was included in these local encoded-base rates.

Majority-vote fusion across copies

With c independent copies and per-base survival q , the binomial tail below is the probability that a strict majority are correct. For odd c this is sufficient for correct voting, but not necessarily identical to implemented plurality success when incorrect votes split.

$$(12) \quad P_{mv}(c) = \sum_{j=\lceil c/2 \rceil}^c C(c,j) q^j (1-q)^{c-j}$$

For three copies, at least two correct bases guarantee a correct vote; the complementary event does not invariably make the vote wrong. A per-position strict-majority probability is not full-message success, and cannot justify the observed five-copy recovery without a joint strand and decoder analysis. The earlier claimed quantitative explanation is withdrawn.

Goldman and Fountain baselines on the same scale

Published methods use different framing, replication and indexing conventions. The local unscrambled approximation does not reproduce the published Goldman redundancy protocol, so an effective published rate cannot be inferred from the following unverified formula.

(13) Published Goldman density under matched complete accounting is not established by this local harness.

Fountain decoding overhead depends on degree distribution, screening, source-chunk count, seed coverage and the decoder. The near-matched clean audit shows that a full-rank matrix can still stall peeling.

(14) No universal 5-10 percent overhead guarantee is established for these screened finite-message configurations.

Historical local baseline failures and source-pinned external trials are retained separately. Invalid clean recovery is a benchmark gate failure, not a meaningful noisy comparison. Current file recovery is reported with actual parameters and complete encoded-base accounting.

Checksum collision limits, not an error-floor theorem

A checksum collision probability requires a specified distribution over corruption patterns. Low-byte Fletcher collisions include deterministic equal-and-opposite trit changes. There is no proven universal exponential floor or worst-case suppression with checksum length in this implemented block check.

(15) Worst-case undetected corruption can occur; a universal $3^{-(L-1)}$ bound is not established.

Tool and dataset build-out: 44 external tools, 40 in-repo implementations, 130 accessions

Under the strict program standard - external research/data tools only; self-written implementations do not count - this repo runs 44 verified external tools: installed science libraries plus live databases and APIs, each executed against this repo's real data. Every run records its analysis and key numbers in results/external_tool_run.json (44 of 45 attempted tools succeeded; failures are recorded in the same file and never counted).

Table. Verified external libraries (35).

external libraries (genuinely used)
Boruta, biopython, cdlib, dna-features-viewer, gensim, gseapy, hmmlearn, igraph, imbalanced-learn, kneed, leidenalg, lightgbm, matplotlib, mrmr, mygene, networkit, networkx, numpy, pandas, patsy, plotly, pydna, pyroldigal, scikit-learn, scikit-network, scikit-optimize, scipy, seaborn, shap, statsmodels, sympy, umap-learn, viennarna, xgboost, yellowbrick

Table. Verified external databases and APIs (9).

external databases / APIs (genuinely queried)
CrossRef, Ensembl Bacteria, Europe PMC, KEGG, NCBI eutils, QuickGO, Reactome, STRING, UniProt

The inventory below is the complementary set of 40 in-repo implementations.

Complementing the external inventory, the lane also runs 40 named, in-repo analysis tools over 130 accession-level sequence windows from E. coli K-12 strain C3 whole-genome shotgun records (FASTA headers and checksummed manifest under data/payloads/ and results/constraint_ml.json). These records share one strain/project and are not independent biological datasets or MG1655 CDS records. Every tool is code in this repository (src/dnacell/tools40.py), executed by experiments/tool_inventory.py, with per-accession outputs in results/tool_run.json. External codecs are reimplementations at benchmark fidelity, labeled as such - not the original authors' code.

Table. The 40-tool inventory by group.

group	tools
Codecs (7)	v2 (ours), rsns (ours, RS-GF(3 ⁵)), goldman (reimpl.), fountain (reimpl.), church (reimpl.), grass (reimpl.), hedges (reimpl.)
Noise channels (6)	substitution, indel, homopolymer_indel, breakage, pcr_dropout, gc_skew

Sequence analysis (10)	gc, homopolymer_max, kmer_spectrum, entropy, tm_wallace, tm_nearest, hairpin_proxy, dinuc_odds, restriction_scan, complexity
ECC / theory (6)	ternary_parity, hamming74, rs_gf4, repetition_vote, fletcher, crc8
Information theory (4)	rate, capacity_binary (BSC), mutual_information, hamming_bound
Metabolic (8)	fba, fva, knockout_scan, moma, reallocation_identity, shadow_prices, uptake_scan, yield

Inventory findings

Codec rates over the accession payloads: v2 1.7156 bits/nt, church 1.0, goldman 0.5248 (the 4x redundancy tax, measured not cited). Lag-1 mutual information of the payload corpus is 0.00438 bits - near-zero, so the payload stream is effectively incompressible by first-order structure, validating rate measurements against the information-theoretic ceiling.

Metabolic arm on the library LP model: sensor flux frozen-vs-replicating 144.5 vs 100.0; FVA is exactly linear in the biomass lower bound (steps of 11.125 per 0.25 of g), and the reallocation identity difference $v^*(0) - v^*(0.5) = 22.25$ matches the closed form.

Knockout scan: upt/gly/tca/etc are essential (sensor flux 0), bio knockout frees the full frozen-level flux (144.5), atm knockout adds 0.5 - the expected essentiality profile.

Shadow prices: only the uptake bound binds (+14.5 sensor flux per unit relaxation).

Uptake scan is linear through the origin as the LP predicts.

Dataset appendix: the 130 accessions

Table. Payload manifest (accession, length nt). Full descriptions in data/payloads/manifest.json.

accession	length
NZ_JAGGIP010000078.1	401
NZ_JAGGIP010000077.1	423
NZ_JAGGIP010000075.1	531
NZ_JAGGIP010000072.1	595
NZ_JAGGIP010000071.1	706
NZ_JAGGIP010000070.1	734
NZ_JAGGIP010000069.1	848
NZ_JAGGIP010000068.1	884
NZ_JAGGIP010000067.1	1031
NZ_JAGGIP010000066.1	1070
NZ_JAGGIP010000065.1	1133
NZ_JAGGIP010000064.1	1167
NZ_JAGGIP010000063.1	1255
NZ_JAGGIP010000062.1	1345
NZ_JACASP010000102.1	474
NZ_JACASP010000101.1	484
NZ_JACASP010000099.1	645

NZ_JACASP010000098.1	672
NZ_JACASP010000097.1	679
NZ_JACASP010000096.1	706
NZ_JACASP010000095.1	733
NZ_JACASP010000094.1	772
NZ_JACASP010000093.1	991
NZ_JACASP010000092.1	1028
NZ_JACASP010000091.1	1031
NZ_JACASP010000090.1	1095
NZ_JACASP010000089.1	1210
NZ_JACASP010000088.1	1232
NZ_JACASP010000087.1	1247
NZ_JACASP010000086.1	1345
NZ_JACASP010000085.1	1442
NZ_JACASP010000084.1	1466
NZ_JACASQ010000105.1	458
NZ_JACASQ010000104.1	463
NZ_JACASQ010000103.1	512
NZ_JACASQ010000101.1	636
NZ_JACASQ010000100.1	645
NZ_JACASQ010000099.1	679
NZ_JACASQ010000098.1	693
NZ_JACASQ010000097.1	706
NZ_JACASQ010000096.1	991
NZ_JACASQ010000095.1	1006
NZ_JACASQ010000094.1	1031
NZ_JACASQ010000093.1	1095
NZ_JACASQ010000092.1	1199
NZ_JACASQ010000091.1	1206
NZ_JACASQ010000090.1	1210
NZ_JACASQ010000089.1	1232
NZ_JACASQ010000088.1	1235
NZ_JACASQ010000087.1	1301
NZ_JACASQ010000086.1	1345
NZ_JACASQ010000085.1	1378
NZ_JACASQ010000084.1	1466
NZ_JACEET010000064.1	541
NZ_JACEET010000063.1	563
NZ_JACEET010000062.1	597
NZ_JACEET010000060.1	706
NZ_JACEET010000059.1	754
NZ_JACEET010000058.1	848
NZ_JACEET010000056.1	1255
NZ_JACEET010000054.1	1345
NZ_WSNT01000025.1	1045
NZ_WSNT01000021.1	1128
NZ_WSNT01000020.1	1133
NZ_WSNT01000017.1	1031
NZ_WSNT01000013.1	1026
NZ_WSNT01000012.1	1255
NZ_WSNT01000011.1	1263
NZ_WSNT01000002.1	1345
NZ_JCBAAR010000066.1	513

NZ_JCBAAR010000065.1	533
NZ_JCBAAR010000064.1	569
NZ_JCBAAR010000062.1	601
NZ_JCBAAR010000061.1	676
NZ_JCBAAR010000060.1	706
NZ_JCBAAR010000059.1	734
NZ_JCBAAR010000058.1	754
NZ_JCBAAR010000057.1	848
NZ_JCBAAR010000055.1	1255
NZ_JCBAAR010000054.1	1345
NZ_JACASS010000103.1	587
NZ_JACASS010000102.1	636
NZ_JACASS010000101.1	645
NZ_JACASS010000100.1	679
NZ_JACASS010000099.1	706
NZ_JACASS010000098.1	926
NZ_JACASS010000097.1	991
NZ_JACASS010000096.1	1031
NZ_JACASS010000095.1	1054
NZ_JACASS010000094.1	1095
NZ_JACASS010000093.1	1206
NZ_JACASS010000092.1	1210
NZ_JACASS010000091.1	1232
NZ_JACASS010000090.1	1301
NZ_JACASS010000089.1	1345
NZ_JACASS010000088.1	1378
NZ_JACASS010000087.1	1466
PQ790158.1	909
NZ_JACASR010000111.1	404
NZ_JACASR010000110.1	412
NZ_JACASR010000109.1	439
NZ_JACASR010000108.1	458
NZ_JACASR010000107.1	474
NZ_JACASR010000105.1	531
NZ_JACASR010000104.1	555
NZ_JACASR010000103.1	657
NZ_JACASR010000100.1	706
NZ_JACASR010000099.1	769
NZ_JACASR010000098.1	926
NZ_JACASR010000097.1	991
NZ_JACASR010000096.1	1028
NZ_JACASR010000095.1	1067
NZ_JACASR010000094.1	1095
NZ_JACASR010000093.1	1127
NZ_JACASR010000092.1	1159
NZ_JACASR010000091.1	1210
NZ_JACASR010000090.1	1232
NZ_JACASR010000089.1	1345
NZ_JACASR010000088.1	1466
NZ_JAIXNY010000095.1	405
NZ_JAIXNY010000094.1	429
NZ_JAIXNY010000089.1	511
NZ_JAIXNY010000087.1	551

NZ_JAIXNY010000086.1	569
NZ_JAIXNY010000085.1	570
NZ_JAIXNY010000084.1	706
NZ_JAIXNY010000083.1	718
NZ_JAIXNY010000081.1	823
NZ_JAIXNY010000080.1	848
NZ_JAIXNY010000079.1	1004

References

- [1] Church, G.M., Gao, Y., Kosuri, S. (2012). Next-generation digital information storage in DNA. *Science* 337:1628.
- [2] Goldman, N. et al. (2013). Towards practical, high-capacity, low-maintenance information storage in synthesized DNA. *Nature* 494:77-80.
- [3] Grass, R.N. et al. (2015). Robust chemical preservation of digital information on DNA in silica with error-correcting codes. *Angew. Chem.* 54:2552-2555.
- [4] Erlich, Y., Zielinski, D. (2017). DNA Fountain enables a robust and efficient storage architecture. *Science* 355:950-954.
- [5] Orth, J.D., Fleming, R.M., Palsson, B.O. (2010). Reconstruction and use of microbial metabolic networks: the core *E. coli* model. *Nat. Protoc.* 5:93-124.
- [6] Varma, A., Palsson, B.O. (1994). Metabolic flux balancing: basic concepts, scientific and practical use. *Bio/Technology* 12:994-998.
- [7] Cover, T.M., Thomas, J.A. (2006). *Elements of Information Theory*, 2nd ed. Wiley. (Constrained channel capacity, Ch. 4.)

Optional machine-learning constraint audit: sequence origin, not synthesis difficulty

A random-forest audit compares 130 natural accession windows with 130 RSNS-encoded windows, each 300 nucleotides long. FASTA headers identify the natural files as E. coli K-12 strain C3 whole-genome shotgun records, not MG1655 genes. Each accession contributes one randomly positioned window. Five stratified group folds keep accession identities disjoint. The seed, checksums, windows, out-of-fold scores and feature ablations are retained in results/constraint_ml.json.

Feature view	OOF AUROC	Conditional bootstrap 95%
full	1.000	1.000 to 1.000
no_homopolymer	1.000	1.000 to 1.000
composition_only	0.964	0.942 to 0.985
kmer_only	1.000	1.000 to 1.000

The no-homopolymer-column ablation does not remove homopolymer information: overlapping trimer frequencies still expose the never-same construction. Perfect origin discrimination therefore measures a codec signature, not a learned biological law. Natural accessions share one WGS project/strain and cannot establish independent-genome transfer. Composition-only performance also differs, but does not identify a synthesis mechanism.

No experimental synthesis yield, sequencing failure or expression labels were used. This is not a synthesis-difficulty predictor, and it does not establish biological validation. The bootstrap conditions on fitted out-of-fold models rather than refitting them. Learning synthesis difficulty remains open until appropriate independent experimental labels are available.

Channel semantics and clean-channel baseline validity

An audit enumerated every single-base change on eight 120-trit test strands: 2,880 substitution events. Each event changed one or two adjacent decoded trits; none changed a trit beyond positions i and $i+1$. Of these events, 1,607 produced an out-of-alphabet trit 3. The differential decoder computes $t_i = (b_i - b_{(i-1)-1}) \bmod 4$. This local dependency directly proves substitution-error locality, independently of the finite test.

Insertion/deletion shifts are a different channel and are not covered by this proof.

The historical generator selects a random base from all four letters after a Bernoulli attempt. Three quarters of attempts change the letter. At nominal 1%, 2% and 3%, expected realized substitution probabilities are therefore 0.75%, 1.5% and 2.25%. Five 100,000-base experiments per rate gave means 0.7392%, 1.4818% and 2.2336%, respectively. Historical outputs are not overwritten or treated as strict-rate results.

Local codec	0%	1%	2%	3%
v1	32/32	27/32	19/32	8/32
v2	32/32	28/32	21/32	19/32
goldman_style	32/32	26/32	20/32	11/32
fountain_crc8	24/32	8/32	0/32	0/32
rsns	32/32	30/32	16/32	1/32

These fresh results use 32 shared 128-byte message identities and strict-different-base substitutions. The Fountain-style local control uses CRC8 detection, uniform low-degree sampling and decoder side metadata, not the original published implementation or equivalent Reed-Solomon inner correction. It fails clean-channel decoding in eight cases because the screened random droplet graph lacks enough usable singleton propagation. Its noisy deficit is not evidence against the published method. Wilson recovery intervals and every trial are preserved in results/strict_substitution_benchmark.json. All code-comparison superiority claims remain pending an independently verified baseline and matched complete resource accounting.

The defensible contribution at this checkpoint is an audited local codec family with explicit recovery and framing limits, together with the in-model resource-reallocation result. The biological synthesis and experimental-error validation gaps remain open. The new natural-sequence-origin model does not close those gaps. Source code for this audit is experiments/channel_semantics_audit.py; it does not silently replace the historical generator.

Soft-erasure marker correction and matched rerun

The RSNS implementation initially mapped invalid trit 3 modulo three during descrambling. This destroyed the erasure marker before GF(243) symbol assembly. The decoder now preserves marker 3 through descrambling. Three regression tests cover

marker survival, valid scrambler reversal and clean roundtrips across payload sizes; the full suite passes 31 tests. This repair changes behavior without validating the entire codec or its published-baseline comparison.

Strict substitution	Pre-fix RSNS	Marker-preserved RSNS
0%	32/32	32/32
1%	30/32	32/32
2%	16/32	25/32
3%	1/32	9/32

Messages, encoded sequences and error seeds are identical between pre-fix and repaired runs. The fixed measurements are archived separately in `results/strict_substitution_benchmark_soft_fix.json`; no historical value is overwritten. Improved recovery is evidence for this marker-handling repair. It does not restore the withdrawn propagation theory, solve insertion/deletion framing, provide synthesis labels, or repair the invalid Fountain-style baseline. Published-codec superiority remains unresolved.

External Fountain port and strand-geometry control

Pinned implementation and clean-channel verification

After the local CRC8/uniform-degree baseline failed clean recovery, we tested the separately compiled, unmodified Python-3 port at <https://github.com/jdbrody/dna-fountain>, commit `f97c1b8a81f5c5b819209d5b5e26c9c8f4439495`. The upstream repository is <https://github.com/TeamErlich/dna-fountain>. This is a software-port audit, not a reproduction of experimental sequencing. Modern reedsolo returns a tuple; the port correctly takes `decode(data)[0]`. A return-type failure did not explain the observed noisy recovery. Clean decoded files were checked by byte equality and stored SHA-256 identity, not merely process exit.

The pilot uses four independently generated 2,048-byte messages with payload seed 511. Each message has three separate noise realizations per rate. Twelve trials are therefore four message identities times three noise repeats, not twelve independent messages. At zero noise the three repeats duplicate the clean case. The external encoder uses 32-byte chunks, two RS parity bytes, robust-soliton delta 0.001 and c 0.025, homopolymer maximum 3 and GC fraction 0.45-0.55. Its fixed budget is 180 oligos of 152 nucleotides each. Both the intact local RSNS and the external port recover all four clean messages.

Fixed-budget strict-substitution results

Table. Whole-file recovery and segmented local chunk recovery. Trials share four payload identities; the chunk total is not an independent-message sample size.

Changed-base probability	External port files	Intact RSNS files	Segmented RSNS files	Segmented chunks
0%	12/12	12/12	12/12	768/768
0.5%	3/12	12/12	12/12	768/768
1.0%	0/12	9/12	7/12	763/768
2.0%	0/12	1/12	0/12	708/768
3.0%	0/12	0/12	0/12	489/768

Noise is independent Bernoulli selection followed by a uniform choice among the three other bases. Requested probabilities are changed-base probabilities, unlike the old four-choice attempt channel. Unequal strand lengths consume noise random numbers differently, so these are not matched identical error locations. Every realized fraction, payload hash and trial outcome is in results/fountain_port_audit.json and results/rsns_segmented_audit.json. The segmented run retains every chunk success and decoder exception. No old trial was replaced.

Strand geometry changes the density statement

The intact local RSNS sequence has 18,495 nucleotides. It is not interchangeable with the external port's 152-nucleotide oligos. To isolate one practical consequence, we split the identical payloads into 64 independent 32-byte RSNS pieces, each 495 nucleotides long. This control assumes perfect known order and chunk identity, charges no index bases, and has no outer redundancy. It is not yet a physically matched storage design: its oligos remain more than three times longer than the external oligos.

Table. Encoded-base accounting, excluding primers and molecular-copy/read budgets.

Design	Strands	Nucleotides/ strand	Total encoded nt	Payload bits/encoded nt
External port	180	152	27,360	0.598830
Intact RSNS	1	18,495	18,495	0.885861
Segmented RSNS	64	495	31,680	0.517172

Thus the local density ordering reverses when the same codec is independently framed in short pieces: intact RSNS is denser than this fixed port budget, whereas segmented RSNS is less dense. Repeated headers and rounding each piece to complete RS blocks explain this cost without invoking a biological mechanism. At 1% the segmented design loses five of 768 chunks, yet five failures can cause five entire files to fail; it recovers seven of twelve files. At 2%, 708 of 768 chunks survive but no complete file survives. Reporting only chunk recovery would hide this file-level penalty. These denominators measure different outcomes.

What the pilot establishes and what remains open

This pilot replaces an invalid local baseline with a pinned external software implementation that passes clean-channel recovery, and demonstrates a resource-accounting sensitivity within RSNS. It does not establish a fair win over DNA Fountain. Both encoder budgets are fixed and unoptimized, the local segmented design has ideal uncharged indexing, the external port has only two parity bytes per droplet, and no read consensus is used. There are no primers, synthesis constraints, molecule-copy budgets, strand dropout, insertion/deletion or measured sequencing errors in this test. A balanced comparison must match permitted strand lengths, total encoded bases and read budgets, account for ordering metadata, and sweep redundancy for both methods before making a frontier claim.

Only four independent messages support this pilot. Noise repeats cannot justify a narrow population confidence interval or a general message-distribution claim. The external port's decline under this one-read strict channel is an observed software result at these parameters, not a contradiction of experimental storage demonstrations. Published-codec superiority and biological validation remain unresolved. Reproduction: `python3 experiments/fountain_port_audit.py --port /path/to/pinned/compiled/port`; `python3 experiments/rsns_segmented_audit.py`; `python3 experiments/append_fountain_audit.py`. Dependencies and provenance are retained in the external audit JSON.

Clean-channel threshold: screening and peeling at near-matched geometry

The next comparison kept external oligo geometry close to the independently framed local control: 63 oligos of 496 nucleotides, totaling 31,248 bases, against local 64 by 495 nucleotides, totaling 31,680 bases. The external design uses 1.36% fewer encoded bases and carries seed headers. Local ordering remains ideal and uncharged. Payloads are the same four 2,048-byte messages. We swept two, eight, sixteen and twenty-four external RS parity bytes while changing payload bytes per droplet to 118, 112, 104 and 96, respectively. This holds encoded oligo length fixed. Known source length removes terminal zero padding in both evaluations. A failed clean-channel gate stopped noisy comparative interpretation.

At maximum homopolymer length three and GC fraction 0.45-0.55, the port peeling decoder fails all four clean messages at every parity setting. Clean failures cannot be interpreted as poor substitution correction. We therefore used an exact graph diagnostic: regenerate source-chunk incidence from embedded seeds, eliminate the binary incidence matrix over $GF(2)$, and propagate the payload right-hand sides. Full-rank systems are back-substituted and checked against original bytes. This is a clean mathematical oracle,

not a modified port decoder or a noisy-error-correcting method. The port source remains unmodified.

Table. Clean recovery at fixed 63 by 496-nt budget. Each count is out of four distinct payloads; graph solve is an oracle, not the original decoding algorithm.

RS bytes	HP 3 peel / linear	HP 4 peel / linear	HP 5 peel / linear
2	0/4 ; 3/4	4/4 ; 4/4	4/4 ; 4/4
8	0/4 ; 3/4	3/4 ; 4/4	4/4 ; 4/4
16	0/4 ; 2/4	4/4 ; 4/4	4/4 ; 4/4
24	0/4 ; 4/4	4/4 ; 4/4	4/4 ; 4/4

Mechanism within the software graph

The two- and eight-byte parity configurations have zero accepted degree-one oligos for every payload under the HP 3 screen. Peeling cannot start without a singleton, even when the incidence matrix has full rank. At RS 24, all four clean matrices have full rank and exact elimination recovers all messages, while peeling recovers none. At RS 16, only two of four matrices have full rank. This separates an algorithmic stopping-set problem from missing information. Many degree-one candidates can be copies of the same source chunk; oligo count alone does not measure coverage of distinct singleton chunks. Every degree histogram, distinct singleton count, binary rank, accepted-sequence hash, screen-attempt count and decoded-file outcome is retained.

Relaxing the screen to maximum homopolymer four produces clean peeling 4/4, 3/4, 4/4 and 4/4 across the parity grid. The remaining RS 8 failure has a full-rank matrix but zero singleton chunks. At maximum homopolymer five, all configurations recover all four clean messages with the original peeling decoder. Total oligo count, length, GC window, robust-soliton $c=0.025$ and $\delta=0.001$ remain fixed. These threshold observations support a screening/graph interaction in this small-payload regime. They do not establish the performance of an optimized Fountain design, because the degree distribution, seed stream and accepted budget were not optimized.

Limits and next valid comparison

Changing permitted homopolymer length changes the sequence constraint. Clean gains from relaxing it are not free gains under an unchanged synthesis requirement. Exact linear recovery also changes the algorithm and applies only to validated clean data here. A noisy comparison may use configurations that pass the clean gate, but must name their distinct constraints and account for local ideal indexing. Four payloads and the port default deterministic LFSR stream do not establish general seed-distribution performance. No biological synthesis, consensus reads, molecule-copy counts, primers, strand dropout or indels were tested. A fair frontier remains open.

The initial HP 3 strict-noise sweep on message zero is preserved separately in results/fountain_geometry_message_0.json and its clean encoded FASTA files. Its failures are not benchmark evidence against the published method. The complete four-message clean audit is results/fountain_clean_graph_audit.json, with per-message files for bounded reproduction. Run python3 experiments/fountain_clean_graph_audit.py --port /path/to/pinned/compiled/port --message N for N=0,1,2,3. The pinned external source is <https://github.com/jdbrody/dna-fountain> at f97c1b8a81f5c5b819209d5b5e26c9c8f4439495. Published-codec superiority and biological validation remain withdrawn.

Clean-valid noisy comparison: parity allocation reverses the pilot interpretation

We next used only external configurations that passed the clean-channel gate on every tested payload. The port permits homopolymers up to length five, retains GC fraction 0.45-0.55, and produces 63 oligos of 496 nucleotides. Four parity settings preserve the encoded-base budget of 31,248 nucleotides. These settings allocate two, eight, sixteen or twenty-four bytes to inner RS parity and correspondingly 118, 112, 104 or 96 bytes to source chunks. The same four 2,048-byte payload identities and their hashes match the retained segmented RSNS control. All zero-noise byte-equality checks pass. Original port code is unmodified, and its original peeling decoder is used, not the clean Gaussian oracle.

For each message and rate, three independent strict-substitution noise seeds are applied. Each encoded oligo receives one read and no consensus. Changed-base probabilities are zero, 0.5%, 1%, 2% and 3%, with a uniform choice among the other three bases after each Bernoulli selection. Zero-noise repeats duplicate a clean input; the denominator twelve is four message identities times three repetitions, not twelve independent messages. The local and external strands are almost the same length but different sequences, so equal random-number seeds do not make identical physical error locations. All realized fractions and exceptions are retained in results/fountain_valid_noise_audit.json and per-message JSON files.

Table. Whole-file success. External HP5 versus local HP1, with near-matched lengths and encoded budget, not identical constraints.

Changed-base p	Port RS2	Port RS8	Port RS16	Port RS24	Segmented RSNS
0%	12/12	12/12	12/12	12/12	12/12
0.5%	0/12	12/12	12/12	12/12	12/12
1.0%	0/12	2/12	12/12	12/12	7/12
2.0%	0/12	0/12	1/12	12/12	0/12
3.0%	0/12	0/12	0/12	0/12	0/12

Redundancy allocation is an experimental variable

At 1%, changing external inner parity from two to sixteen bytes changes complete-file recovery from zero to twelve of twelve within the same total encoded-base budget. At 2%, the twenty-four-byte setting recovers twelve of twelve while the sixteen-byte setting recovers one of twelve and the segmented local control recovers zero. At 3%, all tested configurations fail. These observations contradict an unqualified interpretation of the earlier short-oligo RS2 pilot as evidence of local superiority. External performance depends on parity allocation, source-chunk count, screening and graph survival, not only on the codec name.

The external encoded density is $16,384/31,248 = 0.524322$ payload bits per encoded nucleotide; the segmented local density is $16,384/31,680 = 0.517172$. The external design uses 432 fewer bases, a 1.36% difference relative to the local budget. Density excludes primers and molecular copies for both methods. Source bytes per external oligo shrink with added parity, so more source chunks must be reconstructed from the same sixty-three droplets. Stronger per-droplet correction therefore trades against outer fountain overhead. Within this tested grid, the stronger inner code wins the strict-substitution file-level outcome up to 2%, but this does not prove a global optimum.

Tradeoff, not reinstated superiority

These are different sequence constraints. RSNS enforces no adjacent repeated bases, whereas the clean-valid external run allows repeats up to five and screens GC fraction. A design allowing longer homopolymers is not equivalent under a synthesis rule that requires HP1. The local segmented control also assumes perfect order and chunk identity without charging index bases, while external droplets include four seed bytes. Known source length and codec settings are supplied to both. No physical oligo ordering errors, indels, strand dropout, primers, molecular-copy distribution or consensus-read budget are tested. There are still only four independent payload identities and one fixed deterministic external seed stream.

The defensible result is an observed software tradeoff and a warning about baseline parameterization: a pinned, clean-valid external port can outperform this local segmented design under strict substitutions at near-matched encoded-base budget, while accepting a different sequence constraint. It is not a claim that the local codec beats published storage systems, nor a claim that this port setting is universally best. Biological validation and an optimized constraint-matched frontier remain open. The initial local Fountain baseline, historical attempted-error rates, invalid-clean HP3 sweep and original short-oligo pilot remain archived separately rather than overwritten.

Reproduce per message: `python3 experiments/fountain_valid_noise_audit.py --port /path/to/pinned/compiled/port --message N, N=0,1,2,3`. The saved FASTA for each parity setting permits byte-for-byte encoded-sequence verification. Source: <https://github.com/jdbrody/dna-fountain>, commit `f97c1b8a81f5c5b819209d5b5e26c9c8f4439495`, linked by <https://github.com/TeamErlich/dna-fountain>. This software audit is distinct from the experimental paper at <https://www.science.org/doi/10.1126/science.aaj2038>.

Single DNA substitutions can bypass the block checksum

A checksum counterexample in abstract trits could be dismissed as outside the DNA channel. We therefore enumerated actual changed-base events and passed them through the original differential decoder and low-byte block checksum. The deterministic 128-byte payload has byte i equal to $(17i+31)$ modulo 256. We encode one copy with the current blocked codec and select the first eight payload blocks, spanning each of the four initial-base seeds twice. Each block has 32 payload trits and six check trits encoded as 38 bases. Selection is fixed, not representative sampling of every possible payload or checksum state.

For each selected block we enumerate every position and each of its three alternative DNA letters, then every pair of distinct positions and all nine pairs of alternative letters. Thus one block has 114 single-substitution and 6,327 double-substitution events. There are 912 and 50,616 events across eight blocks. Every event really changes the base; none retains the original letter. Each outcome is classified as invalid differential spacing, checksum rejection, accepted identical payload, or accepted changed payload. The original isolated block payload is the oracle, and the implementation acceptance function is also checked by a regression test.

Finite-event outcomes across eight fixed blocks. Counts are not biological read-error probabilities.

Outcome	Single events	Double events
Invalid spacing	490	39,407
Checksum reject	137	6,060
Accepted wrong payload	285	5,149
Accepted same payload	0	0
Total	912	50,616

Why one changed letter creates a checksum collision

Changing a base at position i changes the incoming and outgoing differential trits, if the outgoing position exists. When both resulting trits stay in the alphabet and their ordinary integer sum is unchanged, the low Fletcher accumulator $s1$ also stays unchanged. This is a common local cancellation mechanism, not a rare collision of an ideal random

checksum. The full Fletcher pair additionally tracks cumulative position-weighted sums, so it can distinguish the retained example even though its low byte cannot. Invalid differential spacing rejects other substitutions first, which explains why the block detection rate cannot be inferred from checksum image size alone.

In block zero the first original bases are T,C,G with initial seed A. Replacing the first T by G changes the first decoded payload trits from [2,1] to [1,2]; later trits are unchanged. Both trits are valid, their sum remains three, and the six stored checksum trits still match. The isolated block accepts a wrong payload after just one changed DNA letter. The full twelve-trit Fletcher checksum differs, as verified by the new test. This establishes a concrete reachable miss, not a claim that the complete replicated message decoder returns a wrong file for this event.

Event counts depend on region and sequence

All 285 single-event accepted wrong payloads occur at payload-region positions, out of 768 enumerated payload-position events; the 144 check-region events are rejected by spacing or checksum. The full 912-event accepted-wrong fraction is 31.25%, while the payload-only fraction is 37.11%. These are different denominators. Across individual blocks, single accepted-wrong counts range from 29 to 46 out of 114. A single aggregate fraction therefore should not be advertised as a universal conditional miss probability.

Among double events, 5,142 accepted wrong payloads have both changes in payload positions and seven have one payload and one check-position change. None has both changes only in the check region. Double-event accepted-wrong counts range from 427 to 1,019 per block, and the overall fraction is 5,149/50,616, approximately 10.17%. Its lower value than the single-event fraction does not mean that higher physical noise improves detection or recovery: the enumeration conditions on exactly two distinct changed positions, and more events fail the differential-spacing check. It does not mix event orders with their probabilities under a read model.

Consequences for the proposed decoder and future design

The parity repair formula assumes detected erasures and correct remaining blocks. The present checksum can instead accept a wrong block, so parity alone is not an unconditional correction guarantee. In the replicated pipeline, a correct strict majority can remove a changed base before this check; in the complete message, the full Fletcher checksum can reject a corruption that passes the block check. These layers must be measured jointly, and no whole-file error rate follows from the isolated-block counts. A valid improvement would require a tested block code or checksum with explicit detection properties, complete length accounting, and a clean/noisy recovery sweep. It must also preserve a distinction between detecting corruption and correcting it.

No production encoder or decoder behavior is changed here. Replacing six check trits with the full Fletcher pair would add six bases per block and change density, screening and parity accounting; adding a stronger inner code introduces another tradeoff. Those are proposed experiments, not current results. The near-matched external Fountain sweep already shows that allocating more inner parity can reverse a codec ranking. The block-check audit adds a mechanistic reason to compare real detection and correction layers rather than an idealized checksum collision probability.

Reproduction and limits

Run `python3 experiments/block_checksum_dna_audit.py`. The full JSONL event ledger includes block, event order, changed positions, new letters and outcome for every event. Its SHA256 digest is split below for legibility; concatenating the two lines restores the exact digest. The JSON summary retains region and distance breakdowns, exact payload identity and reproducible counterexamples. `tests/test_block_checksum_dna_audit.py` checks event totals, ledger integrity, the single-base reachable collision and agreement with the original acceptance function.

```
976962301254fa988dd9a453c1593c39
a055ea1d515e13e5abf0e2b50d17bde1
```

Limits: eight fixed blocks are a finite library, not a random sample over all messages. The analysis fixes substitution count and treats only within-block changes. Indels, seam-spanning error coupling, physical synthesis, read consensus, whole-file correctness and biological feasibility are not validated here. The result withdraws the uniform block-collision and guaranteed-erasure assumptions; it does not invalidate the already retained empirical complete-file trials or establish a new universal bound.

Full Fletcher fixes the single-event misses but not the double-event mechanism

The reachable low-byte misses suggest an immediate detector redesign: keep both Fletcher bytes rather than only the low one. We tested this idea in an isolated audit candidate, not by changing the production codec. The candidate appends all twelve check trits to each fixed 32-trit block, increasing length from 38 to 44 bases. The same eight original payload blocks and initial seeds are used. Payload prefixes are byte-for-byte identical between candidates, while the six additional check bases change the suffix.

To make the comparison paired, both candidates receive exactly the same changed positions and alternative letters within the 32-base payload prefix. We enumerate 96 single and 4,464 double events per block, for 768 and 35,712 events across eight blocks. The six or twelve check-region bases are not changed. This narrower library permits a detector comparison without treating the longer check region as additional physical error

exposure. It cannot establish whole-file recovery or an equal-budget comparison. The original 38-base all-position enumeration remains separate.

Paired payload-position events. Identical payload prefixes and mutations; different check lengths.

Outcome	Low-byte single	Full single	Low-byte double	Full double
Spacing rejection	417	417	27,955	27,956
Checksum rejection	66	351	2,615	6,103
Accepted wrong	285	0	5,142	1,653
Total	768	768	35,712	35,712

The negative result is a structured collision, not residual random chance

All 285 single-event low-byte misses become checksum rejections under full Fletcher in this library. Among double events, 3,489 low-byte accepted wrong payloads become full-check rejections, but 1,653 remain accepted wrong under both candidates. Thus the full-check accepted-wrong fraction is about 4.63% of the paired double-event library, rather than zero. This cannot be interpreted as a physical error probability or a universal worst-case bound. Sequence choices, mutation count and selected positions are fixed by the enumeration.

The retained double-event counterexample changes base zero from T to G and base two from G to T in block zero. Its first payload trits change from [2,1,0,1] to [1,2,1,0]. The two local compensating pairs preserve total trit sum, and their weighted effects on the high Fletcher accumulator cancel each other. Both stored checksum bytes therefore agree despite a different payload. The new regression test explicitly requires two changed DNA positions, different decoded payloads, equal full Fletcher checksums and acceptance by the audit candidate. Merely retaining the high byte does not guarantee detection of two DNA substitutions.

Paired spacing-rejection counts need not agree exactly, even though the payload prefixes match. A change at the last payload position changes the transition into the first checksum base, whose value differs between candidates. The joint outcome ledger records two single and 89 double events exchanging spacing and checksum rejection across candidates. These are rejected in either case, but explain why a candidate-length change can alter the diagnostic category without changing payload mutations. It would be wrong to force identical category totals or silently discard seam effects.

Charge the extra detector before proposing an advantage

Six extra bases per block increase isolated block length by 15.79%. With 32 payload trits, six trits per payload byte and three identical copies, the asymptotic payload rate would fall from 0.374269 to 0.323232 bits per encoded nucleotide if this detector replaced every

current block check. Headers, parity blocks and finite-size rounding would further affect the exact rate. This is a proposed architecture calculation, not a production encoding measurement: the current codec remains unchanged. Primers, indexing and physical molecular copies remain outside this software base accounting.

The experiment supports retaining the full checksum as a stronger detector in the tested single-event library, but shows that this alone does not remove structured double-event misses. A next design should choose an error-detecting or correcting code with stated distance properties for the actual differential channel, account for its full strand geometry, and test complete-file recovery under clean and strict noisy channels. Trading bases for stronger detection may reduce outer parity or copy budget. Those coupled choices need a new matched experiment, rather than declaring success because one counterexample disappears.

Reproduction and limits of the candidate

Run `python3 experiments/block_checksum_upgrade_audit.py`. The paired ledger records block, event order, positions, alternative letters, and both detector outcomes for all 36,480 events. The summary retains exact joint counts, same-prefix assertion, geometry, rate calculation and the full-check collision. Tests validate the ledger digest and totals and independently verify the double-DNA collision. The SHA256 below is split into two lines only for display.

```
d3a7d6063e72780d77f4b83bddb72475
50b21582730eeb91438246d235cb1759
```

Only payload-region substitutions are enumerated. Mutated check bases, indels, dropout, replicate disagreement, whole-message checksum interactions, parity repair, strand constraints and synthesis are not measured here. No candidate is installed in the production encoder, and no successful complete-file improvement is claimed. The finite paired negative is retained because it rejects the tempting inference that full Fletcher turns every low-byte miss into a guaranteed erasure.

Distance-five block detection for the actual differential channel

The full Fletcher pilot retained 1,653 accepted wrong payloads among 35,712 paired double substitutions. Those negatives remain unchanged. We now test a different audit-local code whose detection property follows from distance rather than an assumed checksum collision frequency. The current encoder is not replaced. The new code groups the 32 payload trits into eight field symbols, adds four parity symbols and maps the resulting 48 trits through the same seeded never-same differential encoder. Each field symbol occupies four consecutive trits. The code is systematic: payload trits are retained

in their original order, so its first 32 DNA bases match the existing payload prefix for the same seed.

A finite-field construction that can be checked without a package

Arithmetic uses $\text{GF}(3)[x]$ modulo x^4+x+2 . An integer from 0 to 80 stores four polynomial coefficients in little-endian base three. Addition is coefficientwise modulo three; multiplication is polynomial convolution followed by reduction. The script checks that the modulus has no monic linear or quadratic divisor. A reducible degree-four polynomial over a field must have a factor of degree at most two, so these checks establish irreducibility. Every nonzero element is also checked for a multiplicative inverse. This redundant test helps catch representation mistakes that a theoretical field name alone would not expose.

Choose twelve distinct field elements a_i with integer representations 0 through 11. The four parity-check rows are $H[r,i]=a_i^r$ for $r=0,1,2,3$, with $a_i^0=1$ including the zero element. Given eight payload symbols, solve the four-by-four linear system on columns 8 through 11 to make $Hc=0$. Gaussian elimination uses the verified field inverses. No empirical fitting, checksum truncation or message-dependent parity matrix is used. Clean encoding and acceptance are tested under all four block seeds.

Why field distance five is enough for two DNA substitutions

Any four columns of H have Vandermonde determinant equal to the product of pairwise coordinate differences. Since all coordinates are distinct, every factor is nonzero. Any smaller set can be extended to four columns and is therefore independent. A nonzero codeword cannot have support of at most four symbols, giving minimum field-symbol distance at least five. For an explicit upper bound, the script solves a dependence on the first five columns and obtains coefficients $[7,8,6,2,1]$, with zeros elsewhere. All five coefficients are nonzero and the syndrome is zero. Consequently the field code has parameters $[12,8,5]$, not merely a guessed distance. Tests check all 495 four-column determinants and the weight-five witness.

The inverse rotating-base mapping is local. Trit i depends on DNA bases $i-1$ and i , or the fixed seed and base zero at the start. Changing one base can alter only its own trit and the next trit. Changing at most two bases therefore affects at most four trits, including the last-base boundary where only one trit is affected. Grouping four trits into a field symbol cannot increase this count. If a changed DNA block remains valid under the never-same rule, it differs from the original codeword in at most four symbols. Because a different valid codeword needs at least five changed symbols, its syndrome cannot vanish. Invalid spacing is rejected earlier. This proves detection, not correction, for at most two strict substitutions anywhere in one complete 48-base block with its prescribed seed.

The proof concerns field-symbol distance, not an asserted DNA distance of five. Adjacent differential errors can cancel some changes or cross a symbol boundary; neither event increases the four-symbol upper bound. A changed valid DNA sequence cannot decode to exactly the same trit word because the seeded mapping is one-to-one. The guarantee requires known block boundaries, length and seed. Insertions, deletions, misalignment and wrong seeds are outside the proposition. It also does not say that three substitutions are always detected: six possible affected trits can reach the distance boundary. Extending the scope requires a separate proof and experiment.

Exhaustive finite library including the check region

The eight payload blocks come from the same fixed 128-byte message as the prior low-byte and full Fletcher audits. For each 48-base candidate we enumerate all 48 times three single substitutions and choose(48,2) times nine double substitutions. This is 1,152 single and 81,216 double events across eight blocks, or 82,368 total. Unlike the paired Fletcher pilot, the new experiment includes the entire parity region and mixed payload/parity changes. Every alternative letter differs from the original. Invalid spacings and nonzero syndromes are reported separately. Among valid-spacing events, the implementation verifies the differential bound directly; both maximum affected-trit and maximum affected-symbol counts are four.

All-position strict substitutions for eight fixed distance-five blocks. Counts are enumerated events, not physical error probabilities.

Outcome	Single events	Double events
Invalid spacing	618	63308
Nonzero syndrome	534	17908
Accepted wrong	0	0
Accepted unchanged	0	0
Total	1152	81216

The absence of accepted wrong payloads agrees with the distance proof, but the finite library alone would not justify a universal guarantee. The mathematical bound supplies that guarantee within its assumptions; the ledger checks the implemented mapping and detector on a reproducible sample of payloads. All-position counts cannot be compared as error rates against the earlier 32-position payload-only pilot. Different lengths create different event spaces, exposure and seams. The prior Fletcher collision rate remains a result of its own paired library, not a rate estimate for this candidate.

Pay for the distance before making a recovery claim

Four field parity symbols require sixteen check trits, hence sixteen DNA bases. Total block length is 48 rather than the current 38 or full Fletcher candidate 44. Relative block-length increases are 26.32% and 9.09%, respectively. Under the same nominal three-copy convention, the asymptotic payload rate is $(8/6)(32/48)/3=0.296296$ bits per encoded

nucleotide. The corresponding earlier rates are 0.374269 and 0.323232. These calculations charge all detector bases but omit finite file headers, padding, outer parity, primers and physical molecular copy counts. They describe candidate architecture, not a measured full-file rate or matched-budget frontier.

Stronger local detection does not imply better full-file recovery. A detector can turn wrong accepts into erasures while making each strand longer, raising the chance of a damaged block at a fixed per-base channel rate. Charging sixteen parity bases may leave fewer outer parity blocks or fewer molecular copies under a fixed total budget. The candidate has no implemented correction, outer recovery policy or consensus decoder in this audit. A joint experiment must encode complete files, pass clean channels, apply strict substitution noise to every eligible base, classify exact success, loud failure and silent corruption, and report total geometry and budget alongside constraint violations. None of those milestones is closed by this isolated block result.

Never-same mapping excludes within-block equal adjacent bases, but separate seeded blocks can still meet with equal letters. Field parity also changes GC distribution and local motifs. A geometry-matched comparison must measure complete-strand homopolymer and GC constraints instead of extrapolating the isolated mapping. The detector experiment does not establish viability of synthesis, storage in cells, sequencing or a cyborg-cell integration. The user-supplied verdict gate and all earlier negative results are preserved.

Audit trail and next decision

Run `python3 experiments/block_distance_code_audit.py`. The JSON summary records modulus, coordinates, distance witness, geometry, explicit counts and limits. The JSONL ledger records every block, mutation order, position set, alternative letters and outcome. Its SHA256 is shown below on two lines for readability. Regression tests cover irreducibility, inverses, all four-column determinants, the distance-five witness, four-seed clean acceptance, ledger digest, event totals and observed differential bound. The production encoder remains unchanged, making it possible to compare this candidate later without silently replacing the baseline.

```
727f179dd29421fc81586d1e07f7c4e0
18f453e5fafb57635c60c44530d91ddf
```

This closes one narrow detector question: a systematic finite-field distance-five check detects every one- or two-substitution event within an aligned seeded block, at a stated sixteen-base parity cost. The full-file clean/strict-noise recovery question, rate-and-constraint matched frontier, wet-lab validation, manuscript-wide prose consistency and audited full-text body-page threshold all remain open. PDF page totals alone do not satisfy the body-page requirement.

Joint whole-file recovery: a clean gate and a retained silent-wrong result

The distance-five detector now enters an audit-local complete-file wrapper alongside the low-byte and full Fletcher detectors. We do not replace the production codec or claim matched-budget superiority. All three candidates use the same twelve-trit length header, six-trit-per-byte representation, global twelve-trit Fletcher check, scrambling, 32-trit payload blocks and one outer componentwise mod-three parity block. They differ in local check length: six, twelve or sixteen trits. Each file is represented by one continuous seeded-block strand, repeated either once or three times. The geometry does not model a segmented oligo pool or physical molecular reads.

The wrapper decodes a per-position plurality sequence, tries individual copies if a block fails its local check, and repairs a single missing payload block when outer parity passes. The audit fixes ties in A,C,G,T order. Production uses a set-based tie order that depends on the Python hash seed, so this wrapper is not identical to the production decoder. It also explicitly rejects malformed lengths, out-of-range headers and byte values above 255. Those checks help separate framing errors from recovery but do not eliminate checksum collisions. Copy fallback selects the first passing payload and does not reconcile disagreement among multiple passing candidates.

Message identities, repeated noise and the clean gate

The fixed generator seed 930481 creates sixteen 128-byte and four 2,048-byte messages. Each detector/copy condition receives those same twenty identities and three noise realizations per identity. Strict independent substitutions use rates 0, .005, .01, .02 and .03, with each changed letter drawn from exactly the three alternatives to its original base. The channel operates on every encoded base, including local and global checks, headers and outer parity. Noise seeds are reproducible, but unequal lengths consume the generator differently: matching seeds is not matching corrupted positions. The experiment totals 1,800 file trials across six detector/copy configurations and five rates.

All six configurations recover all sixty clean trials exactly. The sixty trials represent twenty distinct files with three identical zero-noise repetitions, not sixty independent clean files. Clean success is a necessary gate that earlier invalid codec ports did not always pass, but it does not establish a noisy advantage. A comparison that fails this gate should stop before interpreting its error curves. Here the gate passes, allowing the noisy failures and costs to be reported without hiding them behind a detector-only success.

Three-copy complete-file counts. Each cell is exact success / loud failure / silent wrong among 60 repeated-measure trials.

Strict p	Low-byte 38 nt	Full Fletcher 44 nt	Distance-five 48 nt
0	60 / 0 / 0	60 / 0 / 0	60 / 0 / 0
.005	59 / 1 / 0	59 / 1 / 0	60 / 0 / 0
.01	45 / 15 / 0	54 / 6 / 0	54 / 6 / 0
.02	34 / 26 / 0	40 / 20 / 0	41 / 19 / 0
.03	17 / 41 / 2	24 / 36 / 0	23 / 37 / 0

At one copy, the low-byte candidate recovers one of sixty trials at $p=.005$; the other fifty-nine fail loudly. Full Fletcher and distance-five both fail loudly in all sixty trials at that rate. At .01, .02 and .03 every one-copy candidate trial fails loudly. A detector does not create redundancy that recovers many erased blocks: one outer parity block cannot repair multiple missing blocks. Stronger detection can remove wrong acceptance but cannot make the one-copy architecture useful in this library. These failures remain part of the result rather than being omitted from the plot or averaged into the three-copy condition.

A production-reproducible silent wrong counterexample

The fixed-order low-byte audit decoder produces two silent wrong outputs at $p=.03$ with three copies: 128-byte messages 3 and 4, both at noise replicate 1. It returns bytes without an exception, but the bytes differ from the expected file. The saved negative-case JSON retains exact noisy strands, expected bytes, returned bytes and production replay.

Through the unmodified production `decode_message_v2` under

`PYTHONHASHSEED=0`, message 3 reproduces the same wrong result while message 4 decodes correctly. A subprocess regression fixes that hash seed and requires the message-3 production output to differ from the expected file. This is an existence counterexample to never-silent-corruption, not a claim that both audit cases fail under every production tie order.

The README guarantee is withdrawn. A global Fletcher check can accept a wrong body if its checksum collides; a passing local low-byte check and first-passing-copy policy do not prevent that route. The present experiment establishes the saved counterexample and does not assign all causal changes to a single local mechanism. A deeper trace of selected block candidates, parity repair and global-check collision would be needed to attribute the complete failure chain. Randomness in a production tie policy is also not a safety guarantee. Deterministic comparison should state the policy, while an implementation used outside this audit should reject or deliberately resolve conflicting validated candidates.

Neither full Fletcher nor distance-five produces a silent wrong output in these 600 noisy/clean trials per detector across copy conditions, but absence in a finite library is not a whole-file theorem. The distance-five proposition covers at most two substitutions in an

aligned isolated block. A file channel can place more substitutions in a block, and plurality/fallback can create a candidate with a different effective error pattern. The global check remains Fletcher for every wrapper. The stronger candidate therefore receives no universal whole-file integrity guarantee from the local distance proof.

Finite-size geometry, constraints and the copy budget

Charged three-copy geometry. Headers, padding, one outer parity block and local checks included; primers/indexes/physical molecules excluded.

Bytes	Detector	Strand nt	Total nt	Bits / nt
128	Low-byte	988	2964	.345479
128	Full Fletcher	1144	3432	.298368
128	Distance-five	1248	3744	.273504
2048	Low-byte	14668	44004	.372330
2048	Full Fletcher	16984	50952	.321558
2048	Distance-five	18528	55584	.294761

The candidate exact-recovery counts improve over low-byte detection in several conditions, but full Fletcher and distance-five exchange small leads as noise changes. At $p=.01$ both recover 54 of 60; at .02 the counts are 40 and 41; at .03 they are 24 and 23. Different budgets, lengths and affected positions prevent a causal superiority inference from these differences. Repeated noise realizations on the same files also forbid treating sixty trials as sixty independent message identities for a simple confidence interval. The saved ledger permits stratification by payload size and message instead of silently mixing these units.

All clean full strands in this library have maximum homopolymer run two, not one. Each isolated seeded block excludes equal adjacent bases, but block joins can create equal-letter seams. A whole-strand HP1 constraint therefore fails for these encodings. The ledger records whole-strand GC fraction for every file; the experiment does not screen a common GC window, remove violating files, optimize parity density or report a constraint-matched frontier. Nominal one/three identical software copies are not a physical copy model. These constraints must be matched explicitly before comparing a frontier with a published storage codec or an external port.

What this experiment closes and what it leaves open

Run `PYTHONHASHSEED=0 python3 experiments/block_joint_file_audit.py`. The summary and JSONL ledger retain all 1,800 outcomes, changed-base counts, message and clean-DNA digests, charged geometry, whole-strand homopolymers and GC. Saved wrong-case reads support deterministic audit replay and seeded production replay. Regression tests cover clean files of lengths zero, one, 128 and 2,048 bytes, framing cost, trial digest, clean gates, both audit-local wrong outputs and the unmodified production counterexample. The trial SHA256 is shown on two lines below.

5e39d10e9f65108dffcd88bc4e2802a1
2c11ab31837a85fb63d778afb109c24d

This is a joint clean/noisy complete-file pilot with an honest unequal-budget comparison. The next required result is a preregistered geometry, copy and constraint matched frontier, not another detector checksum substitution. The production silent-wrong claim is disproved and the negative evidence is retained. Manuscript-wide claim consistency across all four projects, audited full-text body growth, independent scientific validation and the user-supplied verdict gate remain separate open tasks. No paper completion, public communication or submission is inferred from the new PDF pages.

Matched geometry and copy budgets: stronger detection does not settle the frontier

The unequal-length whole-file pilot leaves detector cost entangled with noise exposure. We therefore froze a new local comparison plan before opening noisy outcomes. The plan specifies fresh source identities, strict rates, repeated noise seeds, common block geometry, copy counts, a shared constraint gate and a clean-recovery stop condition. It is stored as experiments/block_matched_plan.json with the SHA256 below. This is a matched detector-allocation grid, not an optimized frontier against external storage systems. No physical claims or production changes follow from it.

a539ff5be6798c6e705f87e3ac1fbd9f
ab59b3f9e24608c05a5ad58afa69e840

Charge common padding rather than giving a short code free exposure

All candidate blocks are now 48 bases. The low-byte and full Fletcher words receive ten and four fixed zero trits after their original check region, respectively. The complete padded word is differentially mapped from its prescribed seed. Padding is ignored on decode, exposed to substitutions and charged to total encoded bases. The distance-five candidate uses all sixteen available check trits; the others allocate six or twelve trits to detection and leave the rest unused. They retain the same payload and outer parity architecture. This is deliberately not the best possible use of the low/full baseline budget: it isolates what detector allocation buys under one fixed framing design.

A mutation in ignored padding has no decoded payload effect, while the distance-five detector can reject mutations in its entire parity region. Ignored padding cannot be claimed as error protection. It does, however, preserve equal length and channel exposure, which the earlier unpadded comparison lacked. The first attempted clean run stopped before noisy outcomes because a reused helper checked the overwritten 48-base framing constant against an unpadded core. The audit corrected the helper to validate

each original core length explicitly, then reran the unchanged clean gate and frozen plan. No scientific parameter or source-identity selection changed after seeing outcomes.

A shared constraint gate and genuinely paired masks

Seed 930492 creates eight fresh 128-byte and eight fresh 2,048-byte identities. For each identity, all three clean candidates must have maximum whole-strand homopolymer at most two and whole-strand GC fraction from .45 through .55. If any candidate fails, all three versions of that identity are excluded; no replacement search is permitted. No identities failed this gate. The gate is only whole-strand HP2 and GC: it does not match HP1, sliding-window GC, forbidden motifs or a physical oligo pool. Reporting a gate without these definitions would hide a meaningful constraint difference.

For each copy count of one or three, all candidates expose the same 48-base block layout and total nucleotide budget. Equal strand length and RNG seeds now give identical Bernoulli changed-position masks and alternative-letter indices across candidates. Because their original sequences differ, the resulting replacement letters need not match. The ledger independently checks equal changed-base counts and total exposure for each paired message/copy/rate/repetition group. Every included file first passes exact clean decoding. Rates are zero, .005, .01, .02 and .03; each identity receives three repeats. The grid contains 1,440 file trials, not 1,440 independent files.

Matched three-copy 128-byte results: exact success / loud failure / silent wrong among 24 trials from eight identities.

Strict p	Low-byte padded	Full Fletcher padded	Distance-five
0	24 / 0 / 0	24 / 0 / 0	24 / 0 / 0
.005	24 / 0 / 0	24 / 0 / 0	24 / 0 / 0
.01	21 / 3 / 0	23 / 1 / 0	24 / 0 / 0
.02	15 / 8 / 1	22 / 2 / 0	22 / 2 / 0
.03	9 / 15 / 0	15 / 9 / 0	11 / 13 / 0

Matched three-copy 2,048-byte results: exact success / loud failure / silent wrong among 24 trials from eight identities.

Strict p	Low-byte padded	Full Fletcher padded	Distance-five
0	24 / 0 / 0	24 / 0 / 0	24 / 0 / 0
.005	14 / 10 / 0	20 / 4 / 0	20 / 4 / 0
.01	7 / 17 / 0	13 / 11 / 0	12 / 12 / 0
.02	0 / 24 / 0	0 / 24 / 0	0 / 24 / 0
.03	0 / 24 / 0	0 / 24 / 0	0 / 24 / 0

The long-file negative and the small-file reversal

Stronger local detection improves several matched settings over the padded low-byte candidate, but distance-five does not uniformly dominate full Fletcher. At 128 bytes

and .01 the exact counts are 24 versus 23; at .02 they tie at 22; at .03 full Fletcher recovers 15 while distance-five recovers 11. At 2,048 bytes and .01 full Fletcher recovers 13 while distance-five recovers 12. These are small finite differences with only eight source identities per size. The result does not justify a population ranking or an optimized parity choice. A code with a stronger low-order detection theorem can create more erasures than an incomplete detector at higher substitution multiplicity, without enough outer redundancy to repair them.

Every three-copy 2,048-byte trial fails loudly at strict .02 and .03 for all detectors. The one-parity-block architecture fails at that scale despite local detector redesign and copy redundancy. This is an important retained negative, not a failed test to discard. One-copy candidates pass clean recovery. The padded low-byte 128-byte condition at .005 has one exact success and 23 loud failures; every other one-copy noisy condition fails loudly in all 24 trials. Moving from one to three copies therefore buys observed recovery in some conditions at exactly three times the encoded nucleotide cost, while leaving the hardest long-file conditions unresolved.

One padded low-byte 128-byte trial at .02 returns a wrong file without an exception. Its exact reads and expected/actual bytes are saved and replay-tested. Neither stronger detector produces a silent wrong in this grid, but again finite absence is not a whole-file guarantee. The seeded unmodified-production counterexample from the earlier unpadded run remains separate. Padding changes framing, so this new padded wrong case is not attributed to the production decoder without its own valid replay route.

A finite cost grid, not a solved frontier

At 128 bytes each clean strand is 1,248 nt; one and three copies cost 1,248 and 3,744 nt, giving .820513 and .273504 payload bits per encoded nt. At 2,048 bytes each strand is 18,528 nt; one and three copies cost 18,528 and 55,584 nt, giving .884283 and .294761 bits per encoded nt. All local check trits, ignored padding, finite headers, payload padding and the one outer parity block are charged. Primers, indexes, molecules, physical read depth and library dropout are not. Common within-grid budgets remove one confound without pretending these costs are complete laboratory costs.

A cost/recovery plot of this finite grid would retain both copy counts: one copy has lower cost and lower noisy recovery; three copies has higher cost and often higher recovery. Calling that plot an optimized frontier would be wrong because the grid does not vary outer parity, block size, code correction, oligo segmentation or screening overhead. No external baseline is included. The strongest defensible comparison is matched local detector allocation under HP2 and whole-strand GC constraints, with the observed full-Fletcher/distance-five reversals and universal long-file high-noise failures left visible.

Reproduction and remaining work

Run `python3 experiments/block_matched_geometry_audit.py`. The frozen plan hash, geometry/gate record, exclusions, size-stratified summaries, all-trial ledger and saved wrong reads are retained. Tests check plan and ledger digests, no excluded identities, clean gates, exact geometry, paired masks/counts, wrong-output replay and isolation from the earlier unpadded benchmark module. The trial ledger digest is displayed below. Detector implementations stay audit-local, and no prior trials are overwritten.

```
5ce19a130ce6a12a1e9a703667ffd6f5
8e759d1253bbc5592bbfe0849ed4f723
```

This completes a narrow matched geometry/constraint/copy comparison but does not complete the requested optimized frontier or establish biological storage. More outer redundancy, correction and segmented framing are candidate hypotheses, not validated next steps. Whole-paper source consistency, explanatory body growth, independent validation and the user-supplied verdict gate remain open. The additional manuscript pages report methods, costs, failed settings and scope; PDF totals still do not establish fifty audited full-text body pages.

Outer-parity stripes and copy budgets: a finite recovery grid with local-GC failures

The previous matched grid failed every long-file trial at two and three percent substitutions with three copies and one parity block. A stronger local detector cannot recover erased blocks on its own. A new local plan therefore expands two distinct redundancy allocations: copies of the entire strand and componentwise parity over interleaved stripes. The plan is frozen before outcomes, uses sixteen fresh message identities, and retains low-byte, full Fletcher and distance-five checks at the same charged 48-base block geometry. The tested grid is a finite set of software candidates, not an optimized storage frontier or an external-codec comparison. Production code and all earlier silent-wrong evidence remain unchanged.

What an outer stripe repairs, and what it cannot repair

Divide data blocks by their index modulo the stripe count. For each stripe, append one 32-trit payload whose coordinates are the sums modulo three of its member payloads. If exactly one member is rejected and its parity block is valid, subtraction recovers that member. Two missing members in one stripe remain underdetermined. Accepted wrong payloads are not corrected by this rule. The final global Fletcher check and byte/header validation are retained, but checksum collisions remain possible. This scheme changes only the outer erasure allocation; the distance-five detector still provides its aligned-block low-order detection theorem, not a whole-file correction theorem.

We test one, four and sixteen parity stripes and one, three and five copies. Each copy first participates in deterministic base plurality, with A/C/G/T fixed tie order. A rejected block then falls back through individual copies in order. This choice is reproducible but is not an optimal likelihood decoder. Increasing copies can reduce substitution erasures while accepting a colliding candidate on fallback can still produce wrong data. More parity protects more simultaneous erasures only when they occupy separate stripes; an interleaving design does not remove clustered-noise or dropout risks.

Matched exposure within a geometry, and the cost of changing geometry

Seed 1001457 generates eight 128-byte and eight 2048-byte messages. Each identity is used across every detector, copy budget and stripe choice. Two noise realizations per condition use strict independent substitutions, not replacement sampling that can leave a chosen base unchanged. The rates are zero, .01, .02 and .03. For each identity and stripe geometry, all detectors must pass whole-strand homopolymer at most two and GC from .45 through .55; a failed geometry would exclude all three detector versions with no replacements. No geometry fails. Every included clean file is exactly recovered before noisy interpretation. The complete grid has 3456 trials from sixteen identities, not 3456 independent source files.

Within a fixed geometry, length and random seeds pair detector masks and alternative-letter indices. Across different stripe counts, lengths differ, so masks need not be pathwise paired; matching a seed alone does not make unequal-length channel samples identical. Padding, local checks, headers, terminal payload padding, every parity block and every strand copy are charged. At 2048 bytes, five copies cost 92640, 93360 and 96240 nucleotides for one, four and sixteen stripes respectively, giving .176857, .175493 and .170241 payload bits per nucleotide. Primers, indexes, screening retries, molecules, oligo pool dropout and physical sequencing depth are absent from these costs.

2048-byte recovery at strict three-percent substitutions and five copies. Exact successes among sixteen trials (eight identities, two noise draws); all remaining trials fail loudly.

Parity stripes	Total nt/file	Low byte	Full Fletcher	Distance five
1	92640	8/16	13/16	11/16
4	93360	8/16	15/16	15/16
16	96240	9/16	16/16	16/16

Observed recovery gains do not make the stronger detector universally best

At the largest tested copy and parity allocation, full Fletcher and distance-five recover all sixteen long-file three-percent trials. That resolves a finite software failure in this fresh grid at a much larger budget; it does not contradict the earlier three-copy negative or guarantee a population success probability. With five copies and one stripe, full Fletcher instead recovers thirteen and distance-five eleven. Stronger low-order detection can

create erasures that a limited outer code cannot repair. The extra parity changes that balance, but it does not establish superiority across noise channels, file sizes, independent source distributions or decoding algorithms.

Two new silent wrong outputs occur in low-byte 128-byte candidates with three copies at three-percent noise: message six with four stripes and message two with sixteen stripes. Exact noisy reads and wrong bytes are retained and replay-tested. Neither full Fletcher nor distance-five has a silent wrong in this grid; finite absence is not a never-wrong claim. Error severity matters alongside exact recovery and cost: a cheaper candidate with occasional undetected corruption cannot be ranked solely by its success fraction.

Whole-strand GC is not local synthesis feasibility

The clean gate deliberately repeats the earlier whole-strand constraint rather than silently tightening it after outcomes. We additionally inspect every 48-base block and every sliding 64-base window against GC .35 through .65. Among the 144 detector/message/stripe clean geometries, thirty fail the block-local GC audit and seventy fail the sliding-window audit. Those failures are recorded, not screened out of the recovery denominator. Every whole-strand gate passes, yet many local windows are unsuitable under the stricter audit. A synthesis-feasible frontier cannot be obtained by relabeling this grid as constraint-compliant. Stratifying these failures reveals a stronger limitation: no distance-five 2048-byte geometry passes both the block and sliding-window gates at any tested stripe count. Thus zero long-file identities jointly satisfy all three detector local constraints. For 128-byte identities the jointly passing counts are seven of eight at one/four stripes and six of eight at sixteen stripes. These are constraint-audit counts, not post-hoc filtered recovery claims. The hardest long-file result has no matched locally feasible cohort in this pool.

A future screened design must charge its excluded candidates, replacement search, density loss and clean-recovery validation. The distance code operates on ternary symbols; imposing local nucleotide composition constraints may alter codeword selection or require another framing layer. Padding cannot be treated as free protection. This is a joint coding/constraint problem, not an aesthetic choice to hide difficult rows. No motif or primer compatibility gate, physical oligo length, indel model or amplification bias has been tested here.

A descriptive nondominance set, not an optimized frontier

For each file size and rate, the result file retains candidates not strictly dominated within the tested grid on total nucleotides, exact-success count and silent-wrong count. Ties are retained. This is descriptive empirical nondominance over a small repeated-message grid. It is not statistical dominance, a confidence-bound optimum or a search over all parity allocations. Larger grids, independently held-out identities, locally feasible encoding and external validated comparators remain necessary before claiming an optimized frontier.

Reproduction, implementation checks and limits

The audit is bounded into six chunks: run `experiments/block_stripe_frontier_audit.py` with `--size 128` or `2048` and `--stripes 1, 4` or `16`, then `experiments/merge_block_stripe_frontier.py`. The first combined execution stopped at its runtime limit before final result publication; all six chunks were then executed to completion without changing the frozen scientific plan. Tests check clean recovery, one-erasure-per-stripe recovery and two-erasure failure, exact costs and counts, all saved silent-wrong replays and deterministic chunk merging. The plan hash, clean local-GC extrema, exclusions, all 216 size/budget/rate summaries and nondominated choices are in `results/block_stripe_frontier_audit.json`. No historical output is replaced. This experiment adds method and result interpretation to the research body, but PDF totals still do not satisfy the separate fifty-plus content-page requirement excluding headings, appendix and references.

Prospective local-constraint framing: clean feasibility does not rescue noise recovery

The previous 48-nucleotide grid contained zero jointly locally feasible long-file identities. A separate frozen design therefore changes the audit framing rather than filtering the old outcomes. Four fresh 128-byte and four fresh 2048-byte identities are generated from seed 728310 before noisy outcomes. Every block carries four nonce trits followed by the previous 48-nucleotide detector/check/padding geometry. All three detectors receive the same 52-nucleotide block length, one outer parity block and three copies. No production codec is changed.

A finite greedy search with visible costs and exclusions

For each block, nonce values zero through eighty define an invertible SHA-256-derived 32-trit payload mask. The first nonce whose low-check, full-check and distance-five sequences jointly pass block GC [.35,.65], every sliding 64-base GC window [.35,.65] at the assembled prefix, and homopolymer length at most two is retained. A block with no candidate excludes the whole identity before noise. Final whole-strand GC [.45,.55] is also required. This is a bounded greedy search, not a proof that no better nonce or code exists. Candidates are computational attempts, not physical synthesized strands. The four emitted nonce bases are charged to every block and copy; search attempts and rejections are recorded separately rather than incorrectly added to transmitted nucleotides.

Payload	Identities	Total nt/file	Clean exact	3% exact
128 bytes	4	4056	8/8	3/8
2048 bytes	4	60216	8/8	0/8

Counts in the table apply separately to each of the three detectors. Each identity has two paired noise realizations, so eight trials are four identities, not eight independent

messages. The entire grid has 96 trials including clean and noisy conditions. Every noisy long-file trial is a loud failure; five of eight noisy small-file trials per detector are loud failures. No silent wrong occurs in this finite grid, which cannot establish a never-wrong guarantee.

All eight identities pass the prospective constraints, with zero identity exclusions. Small-file candidate attempts are 26, 29, 26 and 27 for 26 emitted blocks each; long-file attempts are 402, 389, 389 and 387 for 386 emitted blocks each. Thus 27 candidate rejections are charged as computational search work across 1648 accepted blocks. Payload densities are $1024/4056 = .252466$ and $16384/60216 = .272087$ bits per transmitted nucleotide at three copies. The clean feasibility result does not imply robust recovery: the long-file three-percent outcome is zero exact recovery for all detectors.

A nonce is a new failure surface, not free protection

Decoding recovers the nonce from its four nucleotide prefix, checks the following core under that prefix seed, then removes the corresponding payload mask. Corrupted nonce bases can select the wrong mask even when a core detector accepts; the global checksum and byte validation remain fallible safeguards. The earlier distance-five proof applies to strict substitutions within an isolated seeded 48-nucleotide core. It does not extend to this 52-nucleotide frame with nonce corruption, boundary effects, whole-file parity or recovery. We therefore claim neither the old detection guarantee nor physical synthesis feasibility for the new codec.

This pivot separates locally compliant clean sequence construction from noisy file recovery. It improves the former under the stated finite software gates but leaves the latter negative at the tested long-file rate. The new messages and geometry differ from the earlier stripe grid, so its 16/16 outcome cannot be treated as a paired baseline for this 0/8 result. Primer compatibility, motifs, physical oligo segmentation, indexing, indels, dropout, amplification and molecular-copy effects remain absent. There is no optimized or externally validated frontier. The frozen plan, candidate ledger, complete trial outcomes and tests are stored in `experiments/local_constrained_plan.json`, `results/local_constrained_audit.json` and `tests/test_local_constrained_audit.py`.

A nonce-only substitution can bypass core payload identity

The prospective codec exposes a failure surface outside the isolated 48-nucleotide theorem. A frozen diagnostic exhausts all four positions and three alternative letters of the first data-block nonce in each of the same eight clean identities. Each mutation is applied identically to all three copies while the 48-base core remains untouched. There are 288 events across three detectors, not 288 independent files or a random channel simulation. We separately classify core acceptance after unmasking and whole-file decoding.

Detector	Spacing reject	Core reject	Core wrong accept	File exact/loud
Low check	56/96	16/96	24/96	72/96; 24/96
Full check	56/96	16/96	24/96	72/96; 24/96
Distance five	56/96	16/96	24/96	72/96; 24/96

For each detector, 24 of 96 nonce mutations are accepted by the core but produce a wrong unmasked payload. These are prefix positions zero through two whose accepted nonce chooses a different mask while leaving the core transition seed unchanged. A zero core syndrome then authenticates the masked codeword, not its association with the intended nonce. This mechanism is not a violation of the core distance proof: the protected core did not change. It is a boundary on what that proof can say about a framed file.

Whole-file decoding returns the exact original in 72 cases per detector because a rejected first block is a single erasure recoverable with the saved outer parity. The 24 accepted-but-wrong cases instead fail loudly under the global header/checksum/byte checks. No silent wrong is observed in these 288 deliberately correlated events. That containment does not prove a never-wrong guarantee, cover a different data or parity block, or cover multiple nonce/core edits, indels, dropout or physical errors. Three copies do not repair a mutation deliberately shared by all three.

The result requires separate claims for block-code detection, framing integrity, erasure recovery and final-file validation. A nonce-aware protected frame or stronger global integrity design would be a different codec and must charge its overhead and test its own proof boundary. We do not silently reinterpret this audit as a production repair. The frozen plan, complete event ledger and regression are in `experiments/local_nonce_audit_plan.json`, `results/local_nonce_audit.json` and `tests/test_local_nonce_audit.py`. All earlier long-file failures and local-constraint caveats remain visible.

Protected nonce prefix: a narrow proof and a prospective parity pilot

A new frozen audit framing repeats the four-trit nonce three times as independently seeded four-base segments, and uses a fixed A seed for the following 48-base core. The decoder rejects unless all three decoded nonce copies agree. This removes the unprotected prefix association observed above at an emitted cost of twelve rather than four prefix bases per block: every block now occupies sixty nucleotides. Checks, ignored padding, outer parity and all three file copies remain charged. This is an audit-local candidate, not a production change.

What the prefix proof covers

At most two strict nucleotide substitutions confined to the twelve-base prefix can alter at most two independently seeded replicas. At least one four-base replica is untouched. If all decoded replicas agree, their value must equal that untouched original; otherwise decoding rejects. Exhaustive tests check all 81 nonce values, every single-position alternative and every two-position alternative: 51,030 events. This proves no accepted wrong nonce under that stated prefix-only channel. It does not correct the nonce, establish a whole-frame code distance, cover combined prefix/core edits, indels, shifted boundaries, a stochastic molecular channel or whole-file recovery. The core has a separate seed so prefix corruption does not redefine its boundary transition.

Constraints and parity were fixed before noisy outcomes

Two fresh 128-byte and two fresh 2048-byte identities receive both one and four interleaved parity stripes. First-passing nonce search is bounded to 81 candidates per block, jointly across all three detector sequences, with homopolymer length at most two, block60 GC [.35,.65], every sliding64 GC window [.35,.65] and whole-strand GC [.45,.55]. Failure excludes an identity/stripe geometry before noise. All eight tested geometries pass, with zero exclusions. Each identity receives two paired noise repetitions at zero and three percent and three copies, giving 96 trials, not 96 independent messages. Computational candidate attempts/rejections are retained separately from emitted nucleotide costs.

Payload	Stripes	Total nt/file	Clean per detector	3% exact: L/F/D
128 bytes	1	4680	4/4	0/4;0/4;0/4
128 bytes	4	5220	4/4	1/4;2/4;2/4
2048 bytes	1	69480	4/4	0/4;0/4;0/4
2048 bytes	4	70020	4/4	0/4;0/4;0/4

All clean trials recover exactly. Four stripes improve the tested small-file recovery for full and distance-five checks to two of four, but neither stripe allocation recovers a noisy long-file trial. Every remaining noisy outcome is loud; no silent wrong occurs in this small grid, which is not a never-wrong guarantee. More parity and prefix integrity do not automatically supply robust file recovery. New identities and geometry prevent a paired causal comparison with the old 52-base pilot, and the sixteen-of-sixteen earlier stripe outcome cannot be used as the baseline here.

All outcomes, search ledgers, charged costs, source hashes and the prefix-only proof scope remain in results/protected_nonce_audit.json and experiments/protected_nonce_plan.json. Tests re-create clean sequences, local gates, parity accounting and the exhaustive prefix events. A stronger prefix can create more erasures under noise, while limited outer parity cannot repair many missing blocks. Optimized constrained coding, external codecs, primers, physical oligo geometry, indels,

dropout and molecular-copy effects remain untested. This result advances framing integrity at an explicit cost while preserving the long-file recovery negative.

Aligned protected-frame detection: combining prefix and core bounds

A separately frozen audit combines the protected twelve-base nonce prefix and the distance-five forty-eight-base core, with known sixty-base alignment and fixed independent decoding seeds. If at most two total strict substitutions affect the prefix, at least one nonce replica is untouched; agreement therefore cannot yield a different nonce. If the core is changed, it contains at most two substitutions, so the existing differential-map/GF(81) argument rejects invalid spacing or detects a nonzero syndrome. If both regions are changed the two bounds still apply separately. Any accepted frame therefore has its original nonce and payload under this channel. This is detection, not correction. It is not a guarantee for three or more changes, indels, shifted boundaries, arbitrary file consensus or outer-stripe recovery, and does not cover the low/full Fletcher detectors.

Two fixed thirty-two-trit payload vectors (all zero and index modulo three), each at nonce zero and eighty, give four isolated frame identities. Every one-position and two-position alternative letter is enumerated across all sixty bases, including seams and parity. All 64,440 changed-frame events reject; clean frames decode exactly. Per identity, 180 single and 15,930 double events are tested. Double events divide into 594 prefix-only, 5,184 mixed prefix/core and 10,152 core-only. These are exhaustive event counts for four frames, not empirical molecular error probabilities. The universal statement comes from the two component bounds, not from these four finite payloads alone.

The audit-local single-frame decoder and plan remain in `experiments/protected_frame_audit.py` and `experiments/protected_frame_plan.json`; complete count partitions and frame hashes are in `results/protected_frame_audit.json`. Tests repeat all events, check all eighty-one nonce values on a clean payload and reject wrong lengths. The production codec and prior file decoder are unchanged. The earlier long-file three-percent recovery failure remains: local integrity can convert corruptions into erasures, while insufficient outer parity still fails to recover a file. This result closes a narrow combined-prefix/core detection question without creating a never-wrong whole-file or physical-storage claim.

Three coordinated prefix substitutions defeat the isolated frame bound

A separately frozen sharpness audit leaves the forty-eight-base distance-five core unchanged and substitutes another legitimate repeated nonce prefix. Every ordered nonce pair from zero through eighty whose repeated twelve-base prefixes differ at exactly three DNA positions is retained. There are 486 such ordered pairs. The same two fixed payload vectors used in the previous frame audit give 972 events, and every one accepts a wrong

unmasked payload in the isolated frame decoder. Tests reconstruct the three changed prefix positions and every decoded wrong payload. This is a concrete adversarial boundary of the at-most-two-change theorem, not a contradiction of it.

The mechanism is that all three replicas can now change consistently. Their agreement authenticates no original nonce once each replica is touched. The unchanged core still passes its check because its masked payload is unchanged, but the newly decoded nonce supplies a different unmasking vector. Core distance does not protect this association. The exhaustive nonce-pair ledger is in `results/protected_frame_three.jsonl`; the frozen plan and counts are in `experiments/protected_frame_three_plan.json` and `results/protected_frame_three_audit.json`. This result is specific to the audit-local isolated-frame decoder and adversarial coordinated substitutions. It is not a stochastic whole-file error rate, a molecular-channel observation or a production-code result. Outer file checks or parity may reject these changes, and no full-file acceptance is inferred.

Consequently the combined prefix/core claim remains strictly at most two aligned strict substitutions. A larger never-wrong claim would be false for this frame design. The earlier finite noisy-file grid and long-file failures remain unchanged; absence of silent wrong outcomes there is not a theorem. A future design must bind nonce choice to the checked payload or charge stronger joint redundancy before extending the claim. No such redesign or biological validation is claimed by this boundary audit.

A charged nonce-bound core rejects the demonstrated prefix substitution

A new audit-local candidate includes the four decoded nonce trits inside the core data before computing parity. The core now has nine data symbols and four parity symbols over GF(81), giving a systematic [13,9,5] code at thirteen distinct coordinates. Four-column Vandermonde independence holds for all 715 coordinate sets; a nonzero weight-five witness [7,8,6,2,1] on the first five coordinates establishes the matching upper bound. The core occupies fifty-two rather than forty-eight nucleotides. With the unchanged twelve-base repeated prefix, the aligned frame costs sixty-four nucleotides, four more than the previous design. This extra exposure is charged, not free protection.

After prefix agreement and core syndrome checks, decoding now requires the core nonce to equal the prefix nonce before unmasking. A legitimate changed prefix with an unchanged core is therefore rejected, even when all replicas agree. The same 486 ordered three-substitution nonce pairs and two fixed payloads give 972 rejections instead of the previous 972 accepted wrong payloads. Four frames also retain the at-most-two-total-substitution check: all 73,344 one/two-change events reject, comprising 192 singles and 18,144 doubles per frame. All eighty-one nonce choices clean-decode a fixed payload. The general at-most-two aligned substitution argument follows the same differential

bound and distance-five proof; the extra nonce comparison binds the association that the earlier design left unchecked.

This is an isolated-frame repair of a demonstrated association failure, not a claim of better recovery at equal budget. At the isolated-frame stage, the sixty-four-base framing had not been checked for local GC/homopolymer feasibility. The separate prospective geometry section below adds that finite check. At that stage it still had not been integrated into a complete outer-parity file decoder or tested under a physical channel, indels or broader corruption. Larger mixed changes can still produce another valid joint codeword. The production codec and earlier long-file failures are unchanged. The frozen plan, charged geometry, complete event counts and distance witness are in `experiments/bound_nonce_frame_plan.json` and `results/bound_nonce_frame_audit.json`; the runner and tests retain the old negative alongside the new candidate result.

Prospective constraint feasibility charges the larger bound frames

A separate frozen geometry-only audit uses two fresh 128-byte and two fresh 2048-byte identities, each with one and four outer parity stripes. It searches nonces zero through eighty in first-passing order per sixty-four-base frame and checks homopolymer length at most two, frame GC [.35,.65], all sliding sixty-four-base GC windows [.35,.65], and whole-strand GC [.45,.55]. Every candidate attempt, accepted nonce, identity hash and excluded geometry is retained. All eight geometries pass; all frames clean-decode their original payload, and there are zero excluded geometries. This pilot uses the isolated candidate fixed block-zero masking convention, not a drop-in replacement for the prior file codec.

Payload	Stripes	Frames/file	Total nt, 3 copies
128 bytes	1	26	4992
128 bytes	4	29	5568
2048 bytes	1	386	74112
2048 bytes	4	389	74688

Each row applies to both fresh identities of that size. All first candidates pass in this finite grid, so attempt totals equal emitted-frame counts; this does not guarantee future feasibility or remove the eighty-one-candidate bound. Costs include twelve prefix bases, four checked nonce bases, thirty-two payload bases, sixteen parity bases, charged ignored padding, outer stripe frames and all three copies. Larger frame lengths increase exposure compared with the earlier design. New identities prevent a paired causal comparison with the old noisy grid, and no noisy whole-file recovery is measured here.

The complete geometry and search ledger is in `results/bound_nonce_geometry_audit.json`, with the frozen plan and runner in `experiments/bound_nonce_geometry_plan.json` and

experiments/bound_nonce_geometry_audit.py. Tests reproduce all eight outcomes, exact cost accounting, local gates and frame clean recovery. At that stage a complete file decoder was still open; the following section adds audit-local integration. A matched noise frontier, indels, dropout, primers and physical oligo storage remain open. The result is constrained sequence feasibility for this charged candidate, not external validation or robust file recovery; the prior long-file failures and three-substitution old-frame negative remain in the record.

Audit-local whole-file integration of bound-nonce frames

A frozen integration plan at base cbfd319 reuses the exact four fresh message identities and all eight size/stripe geometries from the preceding feasibility unit. It adds a decoder without changing the production encoder. The decoder knows sixty-four-base alignment and receives the stripe count out of band. This is a local candidate, not a drop-in production replacement or a complete molecular storage design.

Three aligned copies are voted with fixed A,C,G,T tie order. Every frame is also decoded independently from each copy. A frame is accepted only when all valid candidate payloads agree; conflicting valid payloads reject the file rather than selecting the first successful copy. Missing frames become erasures. One missing data frame per stripe can be reconstructed if its parity frame is readable. Any readable parity is checked against the resulting data. Descrambled length, byte range, checksum and original zero padding are validated before bytes are returned.

The encoder retains its previous first-passing search, eighty-one-candidate ceiling, local GC and homopolymer gates, fixed block-zero mask and charged three-copy geometry. All eight geometries are included. Four fixed source identities, with repeated channel realizations, are not ninety-six independent source samples. The noise model independently selects bases for substitution and draws a genuinely different letter; it does not model indels, oligo loss or correlated molecular errors.

Bytes	Stripes	Substitution p	Exact / 6	Loud / 6
128	1	0	6	0
128	1	0.005	6	0
128	1	0.01	6	0
128	1	0.02	4	2
128	4	0	6	0
128	4	0.005	6	0
128	4	0.01	6	0
128	4	0.02	6	0
2048	1	0	6	0
2048	1	0.005	6	0
2048	1	0.01	1	5
2048	1	0.02	0	6
2048	4	0	6	0
2048	4	0.005	6	0
2048	4	0.01	2	4
2048	4	0.02	0	6

All twenty-four clean repeated trials recover. Across the full ninety-six-trial grid, seventy-three recover exactly and twenty-three fail loudly; no silent wrong is observed.

At 2048 bytes and one percent substitutions, one stripe recovers one of six and four stripes recover two of six. Neither allocation recovers a two-percent long-file trial. At 128 bytes and two percent substitutions, one stripe recovers four of six and four stripes six of six. These are small, repeated-measure counts, not guaranteed rates or evidence of superiority at equal cost.

Controlled erasures and remaining scope

Overwriting the first data frame in every copy with sixty-four A bases gives exact recovery in all eight geometries. Overwriting two data frames belonging to the same stripe gives eight loud failures. Truncating one copy and adding an invalid base each give eight loud failures. Separate regression tests cover two missing data frames in different stripes, unreadable parity with intact data, conflicting valid copies, inconsistent readable parity and nonzero padding. Rejection is intentional where ambiguity would otherwise be silently selected.

The frame distance argument covers aligned substitutions under its stated bound. It does not imply a whole-file never-wrong theorem: higher-weight changes, correlated substitutions, global checksum collisions and coordinated changes of data and parity remain possible. Ninety-six finite trials with no silent wrong do not settle those risks. The file decoder also assumes correct external stripe metadata and known frame boundaries. Primers, indexing, strand segmentation, physical copy counts and their costs remain outside this audit; the existing numerical budgets are therefore not complete synthesis budgets.

The frozen plan, implementation and complete ledger are `experiments/bound_nonce_file_plan.json`, `experiments/bound_nonce_file_audit.py` and `results/bound_nonce_file_audit.json`. Noise seeds, mutation counts and strand hashes permit replay; clean sequence search ledgers and source hashes remain attached to each geometry. Tests reproduce the entire ledger and retain every negative outcome. The user content-page gate remains uncertified, and no clinical, biological or physical-storage validation or submission is claimed.

Perfectly shared copy errors remove the tested copy benefit

A frozen channel-sensitivity follow-up at base d8a441f retains the four source identities, eight included geometries, same three-copy nucleotide budget, same decoder and the three nonzero substitution rates from the preceding integration grid. Each first copy is corrupted using exactly its earlier strict-substitution seed and verified noisy-strand hash. Instead of generating two independent additional errors, this follow-up repeats that same corrupted strand three times. This is an extreme synthetic correlation condition, not an estimate of synthesis or sequencing error correlation.

All seventy-two shared-copy trials fail loudly with unrecoverable-stripe errors, including every half-percent small-file trial. The corresponding independent-copy trials had forty-nine exact recoveries and twenty-three loud failures. No shared-copy silent wrong is observed, which does not imply a never-wrong property. The difference is within the same stored source identities and emitted budgets, but the two channels are different, so it is not a comparison of codec superiority or a new independent replication.

Bytes	Substitution p	Independent exact / 12	Shared exact / 12
128	0.005	12	0
128	0.01	12	0
128	0.02	10	0
2048	0.005	12	0
2048	0.01	3	0
2048	0.02	0	0

Each table row pools one- and four-stripe allocations, with six repeated trials per allocation. Complete allocation-specific outcomes remain in the ledger. Perfectly identical errors give plurality no independent information and leave every copy fallback equally damaged. Local frame checks detect many changes but turn them into erasures; outer parity cannot recover multiple missing data frames in the same stripe. The observed all-loud negative therefore bounds the earlier copy-dependent recovery result rather than contradicting its stated independent channel.

This shared-copy unit does not test partial correlation; the following section adds a synthetic mixture. Indels, dropout, strand segmentation and physical data remain untested. Primers, indices and molecular copy costs remain outside the numerical budget. Four source identities and reused first-copy noise realizations limit generalization. The frozen plan and runner are `experiments/bound_nonce_shared_noise_plan.json` and `experiments/bound_nonce_shared_noise_audit.py`. The complete seventy-two-row ledger, mutation charges, hashes and matched independent outcomes are in `results/bound_nonce_shared_noise_audit.json`; tests reproduce all negative outcomes. Production is unchanged and the manuscript content-page gate remains uncertified.

Partial shared-copy mixtures retain the channel sensitivity

A frozen follow-up at base cf02be1 keeps the same four source identities, eight included geometries, three-copy budgets and decoder. At each position a separate uniform gate selects either the first independently corrupted copy letter for all three copies, with probability q , or each independent copy letter otherwise. The gate is reused across q values zero, one quarter, one half, three quarters and one. The first copy therefore stays unchanged. Both endpoints reproduce the previously saved independent and perfectly shared outcomes and hashes.

Each copy retains the same marginal strict-substitution probability p in distribution under this mixture. The parameter q is the shared-generator probability, not a measured Pearson correlation and not a fitted physical error parameter. Independent corruption seeds are reused, and the gate seed is recorded. This isolates a specified synthetic dependence change without changing emitted cost or selecting new sources.

Shared q	Exact / 72	Loud / 72	Silent wrong / 72
0	49	23	0
0.25	8	64	0
0.5	1	71	0
0.75	0	72	0
1	0	72	0

All 360 outcomes are retained. At q equal to one quarter, only eight of seventy-two recover exactly, compared with forty-nine under independent copies. At one half only one recovers; at three quarters and one all fail loudly. No silent wrong is observed, which is not a guarantee. Counts pool two file sizes, two stripe allocations and three nonzero rates; allocation-, size- and rate-specific counts remain in the full ledger. The pooled decline is a finite outcome, not a theorem of monotone recovery for every noise realization.

The mixture is a synthetic stress test with four reused sources and repeated noise realizations, not independent replication or evidence of a calibrated molecular error distribution. Indels, dropout, primer/index costs and physical copy behavior remain untested. Complete source/clean/noisy hashes, gate and noise seeds, exact copy mutation charges, failures and subgroup summaries are in `results/bound_nonce_mixture_audit.json`. The frozen plan and runner are `experiments/bound_nonce_mixture_plan.json` and `experiments/bound_nonce_mixture_audit.py`. Regression tests reproduce all endpoints and partial outcomes and check invariant first-copy hashes and nested gate selections. Production is unchanged and the manuscript content-page gate remains uncertified.

Original DNA-Fountain sequencing source qualification

A frozen ingest-only unit inspects original sequencing metadata and pinned receiver mechanics before any physical decoding. ENA projects PRJEB19305 and PRJEB19307 list 16 paired MiSeq runs: one master pool, one deep-copy library and two runs at each of seven dilution labels. Their compressed FASTQ files total 95,237,393,879 bytes. The ENA read_count field totals 479,469,649; this metadata sum is not a count of unique synthesized molecules, accepted droplets or decoded messages. No FASTQ was downloaded or decoded in this unit.

Original source is pinned at TeamErlich/dna-fountain commit 8ee2777aa5e9e101e5d756f7e2449f6672b66f1f. Its README uses PEAR stitching, a 152nt fragment-length filter, abundance-descending sorting and removal of sequences containing N. The receiver requires Python2/Cython and unpinned library versions. Its max_hamming comparison operates on byte arrays after Reed-Solomon re-encoding, not directly on nucleotide Hamming distance. These mechanics are not the local bound-nonce decoder or the simplified Fountain reimplementations used in earlier synthetic comparisons.

The original payload hostname fails DNS in this check. A Python3 fork at jdbrody/dna-fountain commit f97c1b8a81f5c5b819209d5b5e26c9c8f4439495 contains a 2,145,290-byte input-file container, but its MD5 differs from the expected encoded-payload MD5 printed in the original recipe. The container contains source files, including a nested archive labelled as a zipbomb example; only its member list was inspected, without extraction or execution. Treating this container as the encoded source bytes would be a provenance error. The original README also specifies 2,116,608 bytes of padding while its decoder parameters imply $67,088 \times 32 = 2,146,816$ bytes. The 30,208-byte discrepancy remains unresolved, not silently corrected.

Qualification is conditional for read preprocessing and blocked for checksum-certified payload reconstruction until the exact payload and padding/chunk convention are recovered. A physical preprocessing unit could count stitched, length-selected and RS-admissible reads under a separate frozen budget, but cannot call that payload recovery. Neither metadata availability nor an original-author reconstruction would validate this project's modified local codec. Full per-run file sizes and MD5 values, source pins and receiver caveats are retained in results/dna_fountain_source_audit.json. The content-page gate remains uncertified.

Sources: <https://github.com/TeamErlich/dna-fountain> ; <https://github.com/jdbrody/dna-fountain> ; ENA projects PRJEB19305 and PRJEB19307, exact portal API URLs retained with the snapshot.

Archive recovery and bounded physical-read preprocessing

Original encoded archive recovery remains blocked. The README inside the mirrored input container explicitly says that the Amazon gift card was removed and can be revealed by decoding. Thus the container is knowingly incomplete relative to the original encoded payload. Original git history, the release page and the Zenodo article deposit supply no encoded archive. Primary bioRxiv text specifies 2,146,816 bytes and 67,088 chunks, supporting the decoder geometry while leaving the conflicting README truncate command unresolved. No tested container truncation or padding convention matches the expected MD5. No nested archive was extracted or executed.

A separately frozen fallback inspected physical sequencing reads without payload decoding. Exactly 8 MiB of compressed bytes from each ERR1816980 master-pool mate were downloaded via verified HTTP 206 ranges. The incomplete gzip prefixes contain 92,414 and 86,355 complete FASTQ records; terminal partial records were excluded. Prefix SHA256 hashes are retained separately from the full-file ENA MD5 values. The first 10,000 synchronized pairs, each with 151nt mates, were inspected. This deterministic prefix is not a random sample or a whole-library error-rate estimate.

Strict exact-overlap stitching with a minimum of 20nt yields 6,368 stitched pairs and 3,632 no-match pairs. It deliberately differs from PEAR quality-aware stitching and may reject pairs that PEAR accepts. Of the stitched sequences, 5,278 have length 152nt. Ten contain non-ACGT symbols; 395 fail exact RS2 codeword checking; 4,873 pass. These records have 4,677 distinct seeds and 4,677 distinct sequence hashes. The test uses A=0, C=1, G=2, T=3 with four bases per byte and reedsolo 1.7.0 decode/re-encode equality. It does not reproduce the original unpinned software environment.

No GC or homopolymer screen, soliton mapping, fountain peeling or whole-payload decoding was performed. RS admissibility alone does not establish original source identity, full receiver acceptance, reconstruction or physical validation of the modified local codec. Code, range headers and digests, counts and selected-record hashes are retained in experiments/dna_fountain_bounded_read_audit.py and results/dna_fountain_bounded_read_audit.json. Full compressed sequencing data are not redistributed. The content-page gate remains uncertified.

Sources: <https://www.biorxiv.org/content/10.1101/074237v4.full-text> ;
<https://github.com/TeamErlich/dna-fountain/releases> ;
<https://zenodo.org/records/889697> . Exact source file URLs are in the result prefix ledger.

Physical-prefix receiver screening uses a shorter sequence than encoder screening

A separately frozen source-semantics unit reuses the same 16 MiB physical-read prefixes and 4,873 RS-exact records. It pins original `glass.pyx`, `droplet.pyx` and `utils.pyx`, rather than treating the encoder constraints as receiver acceptance. In `glass.add_dna` the seed is registered before screening; the reconstructed Droplet is made without its RS argument. Its screen therefore sees the four-byte header and 32-byte payload, or 144nt, not the full 152nt RS-protected encoded oligo. This predicate comparison does not execute the original receiver, soliton mapping or peeling.

All 4,873 RS-exact physical records pass the full 152nt GC and homopolymer predicate. On the receiver-style 144nt substring, 4,789 pass and 84 fail GC; none fails the homopolymer predicate. Removing the RS suffix changes the GC denominator and can move a screened full oligo outside the receiver threshold. The two predicate geometries therefore disagree on 84 records. This is a documented implementation distinction, not evidence that it caused an original-author reconstruction failure.

Under the inspected FASTQ-prefix order, 196 records are duplicate seeds. Registering each seed before the predicate leaves 4,595 first-seen seeds that pass the 144nt screen and 82 that fail it. No seed has conflicting payload hashes in this prefix. Those counts are conditional on prefix order, not the original abundance-descending preprocessing order. They are not complete receiver acceptance, original source identity or whole-payload recovery.

Every predicate result and seed-order flag is retained in `results/dna_fountain_receiver_screen_audit.json`, with pinned source-file hashes and the frozen plan. Prefixes replay the earlier ledger exactly. No new sequencing download, fountain decode or local-codec physical validation was performed. The content-page gate remains uncertified.

Source: <https://github.com/TeamErich/dna-fountain> at pinned commit `8ee2777aa5e9e101e5d756f7e2449f6672b66f1f`. The original receiver has unpinned Python2/Cython dependencies; this unit reproduces the inspected predicate and ordering semantics in Python3, not that entire environment.

Mapping-free rank ceiling for the bounded physical prefix

The documented original receiver uses 67,088 source chunks of 32 bytes each. Conditional on that chunk geometry and the XOR droplet model, each unique seed can supply at most one independent coefficient equation over $GF(2)$, applied to every bit position of the chunk. The number of different equations therefore gives a rank ceiling

even before verifying the seed-to-source-index PRNG mapping. This is not an actual rank calculation, peeling attempt or payload decode.

The first-seen 144-nt predicate scenario retains 4,595 unique seeds, so coefficient rank is at most 4,595, or 6.8492% of 67,088, and nullity is at least 62,493. Full unconstrained chunk recovery would require at least 62,493 additional independent equations. Counting every unique RS-exact seed before the predicate gives 4,677 equations, rank at most 6.9714%, and nullity at least 62,411. Reordering duplicates to admit any predicate-passing record leaves the same 4,595 unique passing seeds in this conflict-free prefix.

Each unresolved coefficient-space dimension applies independently to 256 chunk bits: at least 15,998,208 payload bits remain unconstrained in the screened scenario, before any assumed side information. These are linear-system dimension bounds, not evidence that particular source bytes have been lost. Without verified seed mapping, coordinate coverage is unknown. A small number of dense equations can touch all chunks while remaining far from full rank; equation count does not identify the number of covered chunks.

This necessary-data bound is conditional on the documented K and unconstrained source bits. It does not prove the absence of recoverable individual chunks, incorporate possible known payload structure, or establish rank equality. The prefix is not a random read sample, RS exactness does not prove original payload identity, and the original Python2 random.sample mapping is not verified by a Python3 port. No decoding success or failure rate is inferred. The frozen plan and exact counts/hashes are in `research/dna_fountain_rank_bound_plan.json` and `results/dna_fountain_rank_bound_audit.json`. Source: <https://github.com/TeamErich/dna-fountain>.

CPython 2.7 sample semantics: a bounded compatibility oracle

We built actual CPython 2.7.18 from its official source archive, SHA256 `da3080e3b488f648a3d7a4560ddee895284c3380b11d6de75edb986526b9a814`, and generated vectors for all 4,677 unique RS-exact prefix seeds. Each vector records the first uniform draw, its degree under a shared frozen robust-soliton CDF, and ordered source indices from native `random.sample(xrange(67088), degree)`. The CDF uses the pinned original formula with delta 0.001 and c 0.025. This is a newly generated compatibility oracle, not an original-author archived output.

An independent explicit mirror of the CPython 2.7 pool/set sampling algorithm, driven by integer-seeded MT uniform draws, agrees exactly on every archived vector: zero mismatches. In contrast, the current Python3 random.sample implementation gives a

different ordered vector for every one of the 4,677 seeds. A direct Python3 port therefore cannot silently stand in for this native sampling branch.

The test establishes sample compatibility for the tested seeds under pinned CPython 2.7.18 and the shared CDF. It does not establish the original unpinned interpreter/dependency versions, the NumPy sampling flag, an original encoded payload identity, or the entire Cython receiver. Degree-CDF equivalence against an independently built original NumPy/Cython environment is not separately certified here. No graph peeling or payload decoding is performed. Source hashes, frozen input, actual2.7 vectors and comparison result are retained in `research/python27_sample_source_hashes.json` and `results/python27_sample_parity_audit.json`. Official source: <https://www.python.org/ftp/python/2.7.18/Python-2.7.18.tgz>.

Conditional prefix graph coverage is not full rank or decoding

A frozen graph-only audit now uses the actual compiled CPython2.7.18 vectors under the shared robust-soliton CDF. For 4,595 first-seen receiver-predicate-passing seeds, the index union covers 51,819 of 67,088 coordinates, leaving 15,269 untouched. The largest connected component has 51,538 coordinates; including isolated coordinates there are 15,404 components. Nine initial singleton equations allow structural peeling of ten coordinates before the graph stalls with 4,585 nonempty equations.

Using all 4,677 unique RS-exact seeds before screening gives 52,323 covered and 14,765 untouched coordinates. Its largest component has 52,054 coordinates and its total component count is 14,893. It still has nine initial singleton equations and only ten structurally peeled coordinates, leaving 4,667 nonempty equations. The degree histogram is retained rather than inferred from a population distribution.

Large coordinate coverage and a giant connected component do not imply full equation rank: the previous rank ceiling remains far below 67,088. Structural peeling here removes known-index singleton vertices without reading payload bytes; it does not recover authenticated chunk values. These are conditional graph results for the pinned native branch and shared CDF, not the original unpinned receiver, its NumPy branch, an original payload identity, or a decoding success rate. No new reads are downloaded. Exact vector/input hashes and counts are in `results/python27_graph_coverage_audit.json`. Sources: <https://github.com/TeamErlch/dna-fountain> and <https://www.python.org/ftp/python/2.7.18/Python-2.7.18.tgz>.

Exact conditional coefficient rank without reading payloads

A frozen GF(2) bitset elimination audit now measures coefficient rank under the same conditional CPython2.7.18/shared-CDF vectors. Highest-bit pivot XOR elimination gives

rank 4595 for the 4595 screened equations, with 0 dependent rows and nullity 62493. The unscreened 4677 equations give rank 4677, 0 dependent rows and nullity 62411.

The earlier equation-count ceilings are therefore attained for these conditional coefficient matrices. This does not turn the large coordinate union or giant connected component into enough equations for full recovery. Structural peeling stalled after ten index resolutions even though coefficient rows are independent: peeling and linear rank describe different properties. No payload bytes enter elimination, no chunk values are solved, and no original-payload checksum is evaluated.

The calculation remains conditional on the pinned native random branch and shared degree CDF; it does not authenticate the original unpinned receiver, NumPy branch or encoded payload. Rank deficiency does not prove that no individual coordinate can be identified, and full row rank is not full column rank. Exact hashes and arithmetic are in `results/python27_graph_rank_audit.json`, with source in `experiments/python27_graph_rank_audit.py`. No physical reconstruction or decoding success rate is claimed.